TEC | Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

EL5409 – Laboratorio de control automático

Model-Based Design para la planta térmica TCLab

Modelado, identificación, diseño, verificación e implementación de un sistema de control de temperatura

Profesor: Ing. Juan Luis Guerrero Fernández, PhD

II semestre de 2026

Índice

1 Introducción **1**
1.1 Diseño basado en modelos . . . 1
1.2 Alcance de la práctica . . . 2

2 Objetivos **3**
2.1 Objetivo general . . . 3
2.2 Objetivos específicos . . . 3

3 Recursos y requisitos **4**
3.1 Equipo y software . . . 4
3.2 Archivos suministrados . . . 4
3.3 Archivos desarrollados durante la práctica . . . 5

4 Modelo físico de la TCLab **6**
4.1 Descripción de la planta . . . 6
4.2 Suposiciones . . . 6
4.3 Balance de energía . . . 7
4.3.1 Energía almacenada . . . 7
4.3.2 Potencia del calentador . . . 7
4.3.3 Pérdidas por convección . . . 8
4.3.4 Pérdidas por radiación . . . 8
4.3.5 Ecuación dinámica . . . 8
4.4 Modelo no lineal . . . 9

5 Linealización en $Q = 50\,\%$ **10**
5.1 Punto de operación . . . 10
5.2 Variables de desviación . . . 10
5.3 Expansión de Taylor . . . 11
5.4 Función de transferencia . . . 11

6 Verificación previa de la planta **13**
6.1 Condiciones iniciales . . . 13
6.2 Prueba de conexión y del LED . . . 14
6.3 Prueba de los sensores . . . 14

6.4 Prueba del calentador Q_1 . . . 14
6.5 Lista de comprobación . . . 14

7 Experimentos de identificación 16
7.1 Condiciones experimentales . . . 16
7.2 Ensayo escalón . . . 16
7.3 Enfriamiento entre ensayos . . . 17
7.4 Ensayo multinivel . . . 17
7.5 Comprobación de los datos . . . 18

8 Identificación de modelos FOPDT 20
8.1 Estructura del modelo . . . 20
8.2 Identificación con el ensayo escalón . . . 21
8.3 Identificación con el ensayo multinivel . . . 21
8.4 Comparación de los parámetros . . . 22
8.5 Identificación con System Identification Toolbox . . . 22

9 Validación cruzada 24
9.1 Modelo escalón frente a datos multinivel . . . 24
9.2 Modelo multinivel frente a datos escalón . . . 24
9.3 Gráficas y residuos . . . 25
9.4 Indicadores de validación . . . 25
9.5 Selección del modelo nominal . . . 27

10 Diseño del controlador PI mediante SIMC 30
10.1 Fundamento del método . . . 30
10.2 Estructura del controlador PI . . . 30
10.3 Reglas de sintonización SIMC . . . 31
10.4 Cálculo en MATLAB . . . 32
10.5 Implementación discreta . . . 33
10.6 Alcance del diseño . . . 33

11 Simulación y evaluación Model-in-the-Loop 35
11.1 Arquitectura del modelo . . . 35
11.2 Implementación discreta del PI . . . 36
11.3 Condiciones de simulación . . . 37

11.4 Métricas de desempeño . . . 37
11.5 Análisis comparativo . . . 39

12 Diseño alternativo con Control System Designer 41
12.1 Problema de diseño . . . 41
12.2 Resultado del diseño . . . 41
12.3 Comparación final en MIL . . . 42

13 Transición de MIL a SIL 44
13.1 Controlador seleccionado . . . 44
13.2 Definición del sistema bajo prueba . . . 44
13.3 Frontera del SUT . . . 45
13.4 Estrategia de comparación Normal–SIL . . . 46

14 Preparación del controlador para generación de código 48
14.1 Creación del modelo de trabajo . . . 48
14.2 Comprobación del tiempo de muestreo . . . 49
14.3 Revisión de la frontera del controlador . . . 49
14.4 Conversión a modelo referenciado . . . 50
14.5 Comprobación de `Controller.slx` . . . 50
14.6 Datos propios del controlador . . . 51
14.7 Separación de la inicialización . . . 52
14.8 Configuración para Embedded Coder . . . 52
14.8.1 Parámetros calibrables . . . 52
14.8.2 Optimización de parámetros . . . 53
14.8.3 Solver . . . 53
14.8.4 Target y lenguaje . . . 54
14.8.5 Hardware utilizado durante SIL . . . 54
14.8.6 Interfaz del modelo referenciado . . . 55
14.9 Verificación para generación de código . . . 55
14.9.1 Code Generation Advisor . . . 55
14.9.2 Model Advisor . . . 56
14.10Comprobación funcional . . . 58

15 Generación e inspección del código C 60

15.1 Generación del código 60
15.2 Localización de los archivos generados 60
15.3 Inspección de la interfaz 62
15.4 Estado interno del controlador 63
15.5 Parámetros calibrables 65
15.6 Saturación de la salida 66
15.7 Lógica de anti-windup 69
15.8 Separación entre controlador y planta 71
15.9 Criterio para continuar 72

16 Banco de pruebas Normal–SIL **74**
16.1 Arquitectura del banco de pruebas 74
16.2 Creación del modelo de prueba 75
16.3 Rama de referencia Normal 75
16.4 Creación de la rama SIL 76
16.5 Condiciones comunes de simulación 77
16.6 Inicialización del modelo padre 77
16.7 Registro de las señales 78
16.8 Comprobación de la arquitectura 78

17 Ejecución y comparación Normal–SIL **80**
17.1 Ejecución inicial del banco de pruebas 80
17.2 Comprobaciones previas 81
17.3 Diferencias muestra a muestra 81
17.4 Criterio de equivalencia 81
17.5 Script de comparación 82
17.6 Comparación gráfica 84
17.7 Criterio para continuar 84

18 Cierre de SIL y transición a PIL **86**
18.1 Alcance de la verificación SIL 86
18.2 Qué no demuestra SIL 86
18.3 De SIL a PIL 87
18.4 Hardware para la prueba PIL 87
18.5 Implicaciones del cambio de hardware 88

18.6 Arquitectura para la comparación PIL . . . 88
18.7 Artefactos que deben conservarse . . . 89
18.8 Criterio para iniciar PIL . . . 90

19 Preparación del banco de pruebas para PIL 91
19.1 Conservación de la configuración SIL . . . 91
19.2 Creación del banco de pruebas PIL . . . 91
19.3 Arquitectura Normal–PIL . . . 92
19.4 Callbacks del banco de pruebas . . . 93

20 Configuración del controlador para PIL 94
20.1 Comprobación del Arduino . . . 94
20.2 Hardware objetivo . . . 94
20.3 Configuración de comunicación PIL . . . 95
20.4 Configuración de Model Referencing . . . 95
20.5 Limpieza de artefactos de generación . . . 96
20.6 Comprobación previa . . . 97

21 Ejecución Processor-in-the-Loop 98
21.1 Configuración del SIL/PIL Manager . . . 98
21.2 Ejecución . . . 98
21.3 Comprobación de los resultados registrados . . . 100

22 Comparación *back-to-back* Normal–PIL 102
22.1 Script de comparación . . . 102
22.2 Métricas de comparación . . . 102
22.3 Evidencia gráfica . . . 103
22.4 Registro de resultados . . . 103
22.5 Guardado de los resultados . . . 104
22.6 Diagnóstico ante resultados inesperados . . . 104

23 Cierre de la etapa PIL 106
23.1 Alcance de la verificación . . . 106
23.2 Aspectos que PIL no verifica . . . 106
23.3 Conservación de la evidencia . . . 107

24 Implementación física **108**

24.1 Arquitectura de ejecución con `Monitor & Tune` . . . 108
24.2 Representación numérica para Arduino Leonardo . . . 110
24.3 Preparación de `Controller_implementacion.slx` . . . 110
24.4 Creación de `TCLab_Implementation.slx` . . . 112
24.5 Adquisición de la temperatura `T1` . . . 113
24.6 Filtrado de la temperatura . . . 113
24.7 Integración del controlador . . . 114
24.8 Actuación sobre el calentador . . . 115
24.9 Monitorización durante el experimento . . . 116
24.10 Registro de las señales experimentales . . . 117
24.11 Comprobación previa a la ejecución . . . 117
24.12 Ejecución mediante `Monitor & Tune` . . . 118
24.13 Visualización de los resultados . . . 120
24.14 Cierre de la implementación física y operación autónoma . . . 121

25 Bibliografía **123**

1 Introducción

1.1 Diseño basado en modelos

El diseño basado en modelos (*Model-Based Design*, MBD) utiliza modelos ejecutables para representar tanto el sistema físico como los algoritmos que interactúan con él. Estos modelos permiten estudiar el comportamiento del sistema, diseñar el controlador y verificar progresivamente la solución antes de su implementación final sobre el hardware.

En lugar de pasar directamente desde un diseño matemático hasta el sistema físico, el MBD introduce etapas sucesivas de desarrollo y verificación. Cada etapa permite comprobar aspectos diferentes de la solución y reducir el riesgo de trasladar errores de diseño hacia la implementación.

La Figura 1 muestra algunas de las etapas utilizadas habitualmente dentro de este flujo.

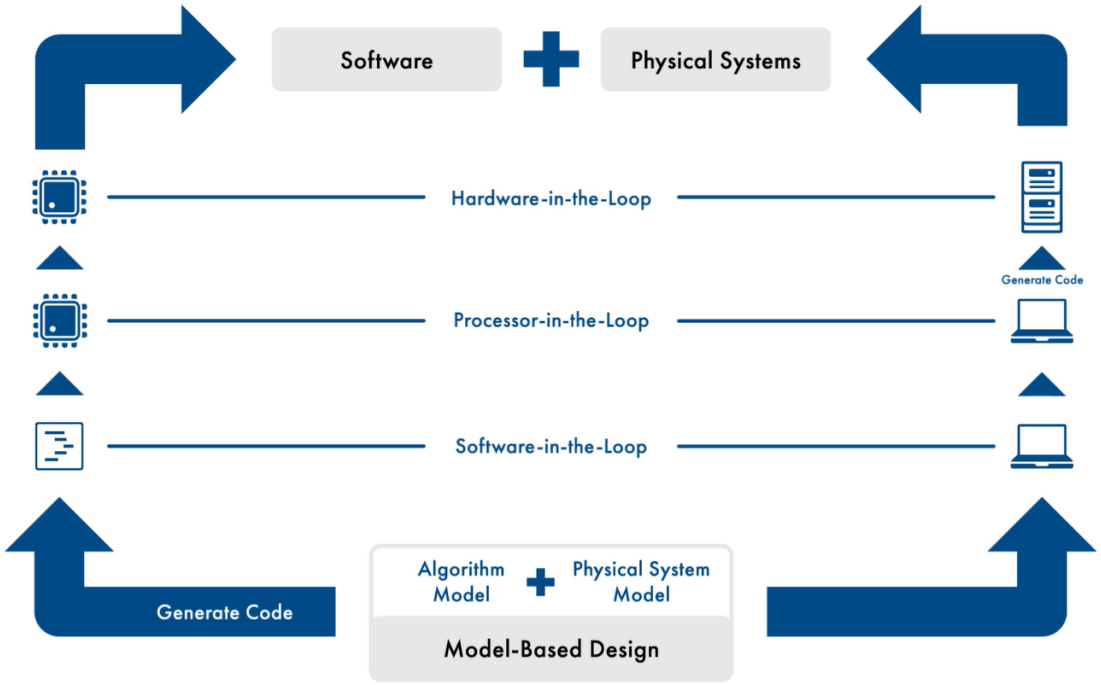


Figura 1: Ubicación de MIL, SIL, PIL y HIL dentro de un flujo de diseño basado en modelos. Adaptada de MathWorks [8].

Las principales etapas representadas en la figura pueden interpretarse de la siguiente manera:

- **Model-in-the-Loop (MIL):** el controlador y la planta se ejecutan como modelos dentro del ambiente de simulación.
- **Software-in-the-Loop (SIL):** el modelo del controlador se sustituye por el código generado y compilado, que se ejecuta en la computadora de desarrollo mientras la planta permanece simulada.
- **Processor-in-the-Loop (PIL):** el código generado del controlador se ejecuta sobre un procesador físico, mientras la planta continúa simulada en la computadora.
- **Hardware-in-the-Loop (HIL):** el controlador implementado interactúa con una representación

de la planta ejecutada sobre una plataforma de simulación en tiempo real.

Estas etapas no son equivalentes ni sustituyen unas a otras. Cada una permite evaluar aspectos adicionales de la implementación. MIL verifica principalmente el comportamiento funcional del diseño; SIL permite comprobar la ejecución del código generado; y PIL incorpora además las características del procesador utilizado para ejecutar dicho código.

En esta práctica se recorrerá una parte amplia de este flujo. El trabajo comenzará con el modelado físico y la identificación experimental de la TCLab, continuará con el diseño y evaluación del controlador mediante MIL, y avanzará posteriormente hacia la generación de código, las verificaciones SIL y PIL, y la implementación final del controlador sobre la planta física.

La etapa HIL se muestra en la Figura 1 como referencia dentro del flujo general de MBD, pero no forma parte de esta práctica.

Alcance de las pruebas experimentales

Los experimentos realizados inicialmente sobre la TCLab física permiten caracterizar e identificar la planta, pero no constituyen por sí mismos una prueba MIL.
De manera análoga, la prueba PIL desarrollada posteriormente utiliza un procesador físico para ejecutar el código del controlador, pero mantiene la planta simulada. La interacción directa entre el controlador y la TCLab física ocurre únicamente durante la etapa final de implementación.

1.2 Alcance de la práctica

La práctica desarrolla el sistema de control de temperatura de la TCLab siguiendo el flujo general

Modelo físico → Linealización → Experimentos → Identificación → Validación
→ Diseño PI → MIL → Generación de código → SIL → PIL → Implementación física.

El modelo físico se construirá inicialmente a partir de principios de conservación de la energía. Posteriormente se obtendrán representaciones lineales de la planta mediante dos enfoques distintos:

1. la linealización analítica del modelo físico alrededor de un punto de operación; y
2. la identificación de modelos FOPDT a partir de datos experimentales obtenidos directamente de la TCLab.

Esta distinción es importante. El modelo derivado de principios físicos permite comprender los mecanismos térmicos dominantes y estudiar el efecto de sus parámetros, mientras que los modelos identificados buscan representar la dinámica observada en la unidad TCLab utilizada durante el experimento.

A partir del modelo identificado y validado se diseñará un controlador PI, cuya implementación se evaluará inicialmente mediante MIL. Posteriormente, el controlador seleccionado se preparará para generación de código mediante Embedded Coder y se verificará progresivamente mediante SIL y PIL.

Debido a las capacidades disponibles en MATLAB/Simulink R2026a, la prueba PIL se realizará utilizando un Arduino Uno R4 Minima como procesador de verificación. Esta etapa no constituye

una implementación de la TCLab sobre dicha tarjeta: la planta continuará simulada y únicamente el algoritmo del controlador se ejecutará sobre el procesador.

Finalmente, el controlador se implementará sobre el Arduino Leonardo integrado en la TCLab y se conectará directamente con los sensores y el calentador de la planta. La ejecución mediante `Monitor & Tune` permitirá observar y registrar el comportamiento del sistema mientras las operaciones del lazo de control se ejecutan sobre el hardware embebido.

2 Objetivos

2.1 Objetivo general

Desarrollar, verificar e implementar un sistema de control de temperatura para la TCLab siguiendo un flujo de diseño basado en modelos (*Model-Based Design*, MBD), desde el modelado y la identificación de la planta hasta la generación de código, su verificación y la implementación experimental del controlador.

2.2 Objetivos específicos

Al finalizar la práctica, el estudiante será capaz de:

- formular un modelo térmico no lineal de la TCLab mediante un balance de energía;
- linealizar el modelo físico alrededor de un punto de operación y analizar el alcance de esta aproximación;
- ejecutar experimentos sobre la planta física para obtener datos de identificación;
- identificar, validar y seleccionar un modelo FOPDT representativo de la dinámica experimental de la TCLab;
- diseñar controladores PI mediante el método SIMC y mediante *Control System Designer*;
- evaluar mediante *Model-in-the-Loop* (MIL) el desempeño del sistema de control y el efecto de la saturación y del mecanismo de anti-windup;
- preparar el controlador seleccionado para generación de código mediante Embedded Coder e inspeccionar la implementación C generada;
- verificar mediante *Software-in-the-Loop* (SIL) la equivalencia entre la ejecución del modelo del controlador y la ejecución de su código generado;
- evaluar mediante *Processor-in-the-Loop* (PIL) la ejecución del código del controlador sobre un procesador físico; y
- implementar y validar experimentalmente el controlador sobre la TCLab física mediante `Monitor & Tune`.

3 Recursos y requisitos

3.1 Equipo y software

Para desarrollar la práctica se requiere:

- planta térmica *Temperature Control Lab* (TCLab), equipada con Arduino Leonardo;
- cable USB para alimentación y comunicación con la TCLab;
- Arduino Uno R4 Minima para la etapa *Processor-in-the-Loop* (PIL);
- cable USB-C con capacidad de transferencia de datos para la conexión del Arduino Uno R4 Minima;
- computadora de desarrollo con MATLAB y Simulink R2026a;
- Embedded Coder;
- Control System Toolbox;
- System Identification Toolbox;
- MATLAB Support Package for Arduino Hardware; y
- Simulink Support Package for Arduino Hardware.

La TCLab y el Arduino Uno R4 Minima cumplen funciones diferentes dentro de la práctica. El Uno R4 Minima se utilizará exclusivamente como procesador durante la verificación PIL, mientras que la implementación final del controlador se realizará sobre el Arduino Leonardo integrado en la TCLab.

La utilización del Arduino Uno R4 Minima durante PIL responde a la configuración disponible en MATLAB/Simulink R2026a. En esta versión, el Arduino Leonardo no proporciona el mecanismo de conectividad requerido para la prueba PIL utilizada en esta guía. Esta limitación no impide generar y ejecutar código sobre el Leonardo durante la etapa posterior de implementación física.

3.2 Archivos suministrados

El profesor suministrará los archivos necesarios para realizar las etapas de modelado, identificación, diseño y comparación automática desarrolladas en la práctica.

Antes de comenzar, verifique que dispone al menos de los archivos indicados en la Tabla 1.

Tabla 1: Archivos principales suministrados para la práctica.

Archivo	Propósito
`A1_simular_modelo_no_lineal_TCLab.m`	Simular el modelo no lineal de la TCLab.
`A2_linealizar_modelo_TCLab.m`	Linealizar el modelo físico alrededor del punto de operación.
`A3_prueba_LED_TCLab.m`	Comprobar la conexión con la TCLab y el funcionamiento del LED.
`A3_prueba_sensores_TCLab.m`	Comprobar las mediciones de los sensores T_1 y T_2.
`A3_prueba_Q1_TCLab.m`	Comprobar la actuación del calentador Q_1.
`A4_experimento_escalon_Q1_TCLab.m`	Ejecutar el experimento de identificación con entrada escalón.
`A4_experimento_multinivel_Q1_TCLab.m`	Ejecutar el experimento de identificación con entrada multinivel.
`A5_identificar_FOPDT_TCLab.m`	Identificar un modelo FOPDT a partir de datos experimentales.
`A6_validar_FOPDT_TCLab.m`	Realizar la validación cruzada de los modelos FOPDT.
`A7_SIMC_controller_TCLab.m`	Calcular los parámetros del controlador PI mediante SIMC.
`A8_init_TCLab_SIMC.m`	Inicializar los parámetros de la planta, del controlador y de las pruebas de simulación.
`A8_TCLab_MIL_SIMC.slx`	Ejecutar la evaluación Model-in-the-Loop del sistema de control.
`A8_validar_TCLab_SIMC.m`	Calcular y comparar las métricas de desempeño durante MIL.
`A10_comparar_Normal_SIL.m`	Comparar muestra a muestra las ejecuciones Normal y SIL.
`A11_comparar_Normal_PIL.m`	Comparar muestra a muestra las ejecuciones Normal y PIL.

3.3 Archivos desarrollados durante la práctica

Además de los archivos suministrados, a lo largo del procedimiento se crearán modelos, resultados experimentales, código generado y bancos de pruebas.

Entre los principales archivos que deberán construirse y conservarse se encuentran:

- `A9_Code_preparation.slx`;
- `Controller.slx`;
- `TCLab_Test_SIL.slx`;
- `TCLab_Test_PIL.slx`;
- `Controller_implementacion.slx`; y
- `TCLab_Implementation.slx`.

También deberán conservarse los archivos `.mat` generados durante los experimentos, la identificación, la validación y las comparaciones, así como la evidencia correspondiente al código generado y a las pruebas SIL, PIL e implementación física.

Estos archivos no se suministran previamente, ya que su creación forma parte del procedimiento de desarrollo y verificación que se realizará durante la práctica.

Actividad obligatoria fuera de clase

El diseño mediante *Control System Designer* forma parte obligatoria de la práctica y deberá completarse fuera de las horas de clase.
Para esta actividad no se suministrará un script ni un procedimiento paso a paso. El estudiante deberá emplear la herramienta para diseñar un controlador PI adicional, justificar sus decisiones de diseño y compararlo con el controlador obtenido mediante SIMC.

4 Modelo físico de la TCLab

4.1 Descripción de la planta

La TCLab contiene dos calentadores resistivos, Q_1 y Q_2, y dos sensores de temperatura, T_1 y T_2. En esta práctica se estudiará principalmente la dinámica entre la potencia porcentual aplicada al calentador Q_1 y la temperatura medida por T_1, manteniendo $Q_2 = 0\,\%$.

Al energizar Q_1, parte de la potencia eléctrica se transforma en calor y eleva la temperatura del conjunto calentador–sensor. La conducción transfiere energía a través del contacto sólido entre el transistor, el disipador y el sensor. La convección transporta energía desde la superficie hacia el aire circundante, mientras que la radiación térmica intercambia energía mediante ondas electromagnéticas con las superficies del entorno. En el modelo concentrado se supone que la conducción interna es suficientemente rápida para representar el conjunto mediante una sola temperatura; las pérdidas externas se modelan mediante convección y radiación [1].

4.2 Suposiciones

Para obtener un modelo concentrado de primer orden se adoptan las siguientes suposiciones:

- el conjunto calentador–sensor se representa mediante una sola temperatura $T(t)$;
- la masa, el calor específico, el área efectiva y los coeficientes de transferencia permanecen constantes;
- la temperatura del aire T_a y la temperatura radiativa del entorno T_∞ permanecen aproximadamente constantes durante cada ensayo;
- las pérdidas de calor consideradas son la convección y la radiación;
- Q_2 permanece apagado y se desprecia su interacción térmica; y
- la entrada $Q(t)$ se expresa en porcentaje, entre $0\,\%$ y $100\,\%$.

Estas suposiciones simplifican el fenómeno real. En particular, no representan explícitamente la dinámica interna del sensor, los gradientes de temperatura, los retardos de comunicación ni las variaciones entre distintas unidades de la TCLab.

4.3 Balance de energía

Para construir el modelo se toma como *volumen de control* el conjunto formado por el transistor calefactor, el disipador y el sensor de temperatura. El modelo concentrado asigna a todo este conjunto una única temperatura $T(t)$. La conducción entre sus componentes ocurre dentro del volumen de control y permite que sus temperaturas tiendan a igualarse; por ello, no se incluye como una pérdida independiente hacia el ambiente. Su efecto queda incorporado, de manera aproximada, en los parámetros efectivos del modelo.

El primer principio de la termodinámica establece que la variación de la energía almacenada en el volumen de control debe ser igual a la tasa neta con la que se le suministra energía. En términos de potencia, la forma general del balance es

$$\underbrace{\frac{dE_{\text{alm}}}{dt}}_{\text{acumulación}} = \underbrace{\sum \dot{q}_{\text{in}}}_{\text{potencia que entra}} - \underbrace{\sum \dot{q}_{\text{out}}}_{\text{potencia que sale}} + \underbrace{\dot{q}_{\text{gen}}}_{\text{potencia generada}} . \tag{1}$$

Todos los términos de la Ecuación (1) tienen unidades de potencia, es decir, joules por segundo o watts. Para la TCLab se consideran tres contribuciones: almacenamiento de energía sensible, potencia térmica suministrada por el calentador y pérdidas hacia el entorno mediante convección y radiación.

4.3.1 Energía almacenada

Si la masa térmica efectiva m y el calor específico c_p permanecen constantes, la energía sensible almacenada, medida con respecto a una temperatura de referencia, puede expresarse como

$$E_{\text{alm}} = mc_p(T - T_{ref}), \tag{2}$$

Por tanto, la rapidez con que cambia esa energía es

$$\frac{dE_{\text{alm}}}{dt} = mc_p \frac{dT}{dt}. \tag{3}$$

El producto mc_p es la *capacidad térmica efectiva* del conjunto y se mide en $\mathrm{J\,K^{-1}}$. Una capacidad térmica mayor implica que se requiere más energía para producir el mismo cambio de temperatura; por ello, el sistema responde más lentamente ante una potencia determinada.

4.3.2 Potencia del calentador

La entrada $Q(t)$ enviada a la TCLab no está expresada directamente en watts, sino como un porcentaje entre 0 % y 100 %. El parámetro α convierte ese porcentaje en una potencia térmica efectiva:

$$\dot{q}_{\text{cal}} = \alpha Q. \tag{4}$$

Con el valor nominal $\alpha = 0.01\,\mathrm{W\,\%^{-1}}$, una entrada $Q = 50\,\%$ representa una potencia térmica de $\dot{q}_{\text{cal}} = 0.50\,\mathrm{W}$, mientras que $Q = 100\,\%$ representa aproximadamente 1 W. El parámetro α es efectivo: agrupa la conversión electro–térmica y las diferencias entre la potencia nominal y la que realmente calienta el conjunto modelado.

4.3.3 Pérdidas por convección

La convección describe el intercambio de calor entre la superficie del conjunto y el aire circundante. Para diferencias moderadas de temperatura se emplea la ley de enfriamiento de Newton. Si se define como positiva la potencia que sale del sistema, la pérdida convectiva es

$$\dot{q}_{\text{conv}} = UA(T - T_a), \tag{5}$$

donde U es el coeficiente global de transferencia de calor, A es el área efectiva expuesta y T_a es la temperatura del aire. Cuando $T > T_a$, el término es positivo y el sistema pierde calor. Si $T = T_a$, no existe transferencia convectiva neta. Si $T < T_a$, el término se hace negativo, lo que indica que el aire entrega calor al sistema.

La convección puede ser natural, producida por el movimiento del aire debido a diferencias de densidad, o forzada, cuando existe una corriente de aire. En la práctica se supone convección predominantemente natural; por eso debe evitarse colocar la TCLab cerca de ventiladores o corrientes de aire, ya que modificarían el valor efectivo de U.

4.3.4 Pérdidas por radiación

Todo cuerpo a una temperatura superior al cero absoluto emite radiación térmica. El intercambio radiativo neto entre el conjunto a temperatura T y las superficies que lo rodean, representadas mediante una temperatura radiativa T_∞, se aproxima con la ley de Stefan–Boltzmann:

$$\dot{q}_{\text{rad}} = \epsilon\sigma A \left(T^4 - T_\infty^4\right). \tag{6}$$

La emisividad ϵ representa qué tan eficazmente la superficie emite radiación en comparación con un cuerpo negro, y σ es la constante de Stefan–Boltzmann. Cuando $T > T_\infty$, la potencia radiativa neta sale del sistema. La dependencia con T^4 hace que esta pérdida aumente de forma no lineal conforme se eleva la temperatura.

Temperaturas del entorno

T_a representa la temperatura del aire que interviene en la convección, mientras que T_∞ representa la temperatura radiativa equivalente de las superficies que rodean la TCLab. En esta práctica puede adoptarse $T_a \approx T_\infty$ si el equipo y el recinto se encuentran inicialmente en equilibrio térmico, aunque físicamente no son exactamente la misma magnitud.

4.3.5 Ecuación dinámica

Al sustituir las Ecuaciones (3), (4), (5) y (6) en el balance general, se obtiene

$$mc_p\frac{dT}{dt} = \alpha Q - UA\,(T - T_a) - \epsilon\sigma A\left(T^4 - T_\infty^4\right). \tag{7}$$

De manera equivalente, agrupando los términos como flujos de calor hacia el volumen de control,

$$mc_p \frac{dT}{dt} = UA\,(T_a - T) + \epsilon\sigma A\left(T_\infty^4 - T^4\right) + \alpha Q. \tag{8}$$

Las Ecuaciones (7) y (8) son idénticas; solamente utilizan convenios de signo distintos. La primera muestra explícitamente la potencia del calentador menos las pérdidas, mientras que la segunda permite que los términos de convección y radiación indiquen mediante su signo si el calor entra o sale. Esta formulación sigue el modelo energético propuesto para la TCLab en APMonitor [1].

La ecuación permite interpretar tres situaciones importantes:

- Si la potencia suministrada es mayor que las pérdidas, $dT/dt > 0$ y la temperatura aumenta.
- Si la potencia suministrada es igual a las pérdidas, $dT/dt = 0$ y el sistema se encuentra en equilibrio térmico.
- Si la potencia suministrada es menor que las pérdidas, $dT/dt < 0$ y la temperatura disminuye.

Uso de la temperatura absoluta

Las temperaturas del término de radiación deben expresarse en kelvin. Aunque una diferencia de temperatura tiene el mismo valor numérico en kelvin y en grados Celsius, no es válido elevar una temperatura en grados Celsius a la cuarta potencia en la ley de Stefan–Boltzmann.

La Tabla 2 resume el significado de los parámetros y los valores nominales empleados en la actividad de linealización de APMonitor [2]. Estos valores no deben interpretarse como parámetros calibrados para todas las unidades de TCLab.

Tabla 2: Parámetros nominales del modelo energético [2].

Símbolo	Descripción	Valor nominal	Unidad
m	Masa térmica efectiva	0,004	kg
c_p	Calor específico	500	$\mathrm{J\,kg^{-1}\,K^{-1}}$
A	Área efectiva	$1{,}2 \times 10^{-3}$	$\mathrm{m^2}$
U	Coeficiente global de transferencia	5	$\mathrm{W\,m^{-2}\,K^{-1}}$
ϵ	Emisividad	0,9	–
σ	Constante de Stefan–Boltzmann	$5{,}67 \times 10^{-8}$	$\mathrm{W\,m^{-2}\,K^{-4}}$
α	Conversión de entrada a potencia	0,01	$\mathrm{W\,\%^{-1}}$
T_a	Temperatura del aire	medida	K
T_∞	Temperatura radiativa del entorno	medida	K
Q	Entrada del calentador	0–100	%

4.4 Modelo no lineal

Al despejar la derivada de la temperatura en la Ecuación (8), se obtiene

$$\frac{dT}{dt} = f(T, Q) = \frac{UA(T_a - T) + \epsilon\sigma A(T_\infty^4 - T^4) + \alpha Q}{mc_p}. \tag{9}$$

El modelo es no lineal debido al término T^4 asociado con la radiación. Para una entrada constante, la temperatura de equilibrio se alcanza cuando la potencia suministrada por el calentador es igual a las pérdidas térmicas hacia el ambiente.

Actividad 1: simulación del modelo no lineal

1. Revise el script `A1_simular_modelo_no_lineal_TCLab.m` e identifique cómo se implementa la Ecuación (9).
2. Sustituya en el script la temperatura ambiente por el valor medido al inicio de la práctica. Este valor se utilizará también como condición inicial del modelo.
3. Ejecute la simulación para $Q = 50\,\%$ y verifique que el intervalo simulado sea suficiente para aproximarse al estado estacionario.
4. Presente las gráficas de $Q(t)$ y $T(t)$ e indique la temperatura de equilibrio predicha por el modelo.
5. Presente por separado las tasas de transferencia de calor por convección y radiación, y explique cómo cambia la contribución de cada mecanismo durante la simulación.
6. Analice cuáles parámetros del modelo presentan mayor incertidumbre y cómo podrían afectar la temperatura de equilibrio y la rapidez de la respuesta.

5 Linealización en $Q = 50\,\%$

5.1 Punto de operación

Se define el punto de operación mediante la entrada constante

$$\overline{Q} = 50\,\%. \tag{10}$$

La temperatura de equilibrio $\overline{T}$ no debe imponerse arbitrariamente. Se obtiene al hacer $dT/dt = 0$ en el modelo no lineal:

$$0 = UA(T_a - \overline{T}) + \epsilon\sigma A(T_\infty^4 - \overline{T}^4) + \alpha\overline{Q}. \tag{11}$$

La Ecuación (11) es algebraica y no lineal, por lo que $\overline{T}$ puede calcularse mediante un método numérico. Por ejemplo, en MATLAB se puede definir el lado derecho como una función anónima y encontrar su raíz con `fzero`:

```
balance = @(T) U*A*(Ta - T) + eps*sigma*A*(Tinf^4 - T^4) + alpha*Qbar;
Tbar = fzero(balance,[Tinf, Tinf + 100]);
```

5.2 Variables de desviación

Las variables de desviación expresan la separación respecto al punto de operación:

$$\widehat{T} = T - \overline{T}, \qquad \widehat{Q} = Q - \overline{Q}. \tag{12}$$

Por tanto, $\widehat{T} = 0$ y $\widehat{Q} = 0$ representan exactamente el equilibrio seleccionado. El modelo lineal describirá cambios pequeños alrededor de este punto, no las variables absolutas desde el origen.

5.3 Expansión de Taylor

La aproximación de Taylor de primer orden de $f(T, Q)$ alrededor de $(\overline{T}, \overline{Q})$ conduce a

$$\frac{d\widehat{T}}{dt} = \frac{\partial f}{\partial T}\Big|_{\overline{T},\overline{Q}} \widehat{T} + \frac{\partial f}{\partial Q}\Big|_{\overline{T},\overline{Q}} \widehat{Q}. \tag{13}$$

Se definen las constantes

$$\gamma = \frac{\partial f}{\partial T}\Big|_{\overline{T},\overline{Q}} = -\frac{UA + 4\epsilon\sigma A\overline{T}^3}{mc_p}, \tag{14}$$

$$\beta = \frac{\partial f}{\partial Q}\Big|_{\overline{T},\overline{Q}} = \frac{\alpha}{mc_p}. \tag{15}$$

El modelo lineal en variables de desviación queda entonces

$$\frac{d\widehat{T}}{dt} = \gamma\widehat{T} + \beta\widehat{Q}. \tag{16}$$

Como todos los parámetros físicos son positivos, $\gamma < 0$. En consecuencia, el modelo lineal nominal tiene un polo real negativo y es estable.

5.4 Función de transferencia

Al aplicar la transformada de Laplace a la Ecuación (16), con condiciones iniciales nulas en variables de desviación, se obtiene

$$G(s) = \frac{\widehat{T}(s)}{\widehat{Q}(s)} = \frac{\beta}{s - \gamma}. \tag{17}$$

Esta expresión puede escribirse en la forma estándar de primer orden

$$G(s) = \frac{K}{\tau s + 1}, \qquad \tau = -\frac{1}{\gamma}, \qquad K = -\frac{\beta}{\gamma}. \tag{18}$$

El modelo físico concentrado no contiene un tiempo muerto explícito. Más adelante, la identificación experimental permitirá determinar si es necesario incorporar un retardo θ para representar la respuesta medida mediante un modelo FOPDT.

Actividad 2: linealización alrededor de $Q = 50\,\%$

1. Revise el script `A2_linealizar_modelo_TCLab.m` e identifique las secciones correspondientes al cálculo del punto de operación y a la linealización del modelo.
2. Utilice la misma temperatura ambiente empleada en la Actividad 1 y ejecute el script.
3. Registre los valores de $\overline{T}$, γ, β, K y τ, incluyendo sus unidades, y compruebe que coincidan con las Ecuaciones (11), (14) y (15).
4. Escriba la función de transferencia $G(s) = \widehat{T}(s)/\widehat{Q}(s)$ con valores numéricos y unidades.
5. Compare las respuestas de los modelos no lineal y lineal ante la perturbación pequeña definida en el script. Reporte el error estacionario, el error máximo y el RMSE.
6. Analice la comparación correspondiente a la perturbación de mayor amplitud y explique por qué disminuye la precisión del modelo lineal al alejarse del punto de operación.

Interpretación del modelo lineal

La salida de $G(s)$ es $\widehat{T}$, no la temperatura absoluta. Para recuperar la temperatura física debe utilizarse $T = \overline{T} + \widehat{T}$. De manera análoga, la entrada del modelo es $\widehat{Q} = Q - \overline{Q}$.

6 Verificación previa de la planta

Antes de ejecutar los experimentos de identificación debe comprobarse que la comunicación con la TCLab, los sensores de temperatura y el calentador Q_1 funcionan correctamente. Esta verificación evita atribuir a la dinámica de la planta errores causados por una conexión inestable, una lectura defectuosa o un actuador que no responde.

Seguridad y protección del equipo

La superficie de los calentadores puede alcanzar temperaturas elevadas. No toque los componentes durante las pruebas ni durante el enfriamiento. Mantenga la TCLab sobre una superficie estable y despejada. Si observa un aumento de temperatura anormal, pérdida reiterada de comunicación o un comportamiento que no corresponde con la señal aplicada, detenga el script y desconecte la alimentación del calentador.

6.1 Condiciones iniciales

Antes de conectar la planta:

1. inspeccione visualmente la TCLab y el cable USB;
2. coloque la planta en una superficie firme, alejada de corrientes de aire y fuentes externas de calor;
3. conecte la TCLab directamente a un puerto USB de la computadora;
4. confirme que MATLAB reconoce el puerto de comunicación;
5. mantenga ambos calentadores apagados antes de comenzar; y
6. cierre cualquier programa que pueda estar utilizando el mismo puerto.

Registre las condiciones de la prueba en la Tabla 3. La misma unidad física deberá utilizarse posteriormente en los experimentos de escalón y multinivel.

Tabla 3: Registro inicial de la planta y del entorno.

Dato	Valor registrado
Código o número de la TCLab	
Fecha y hora	
Integrantes	
Puerto de comunicación	
Temperatura ambiente inicial	
Observaciones del montaje	

6.2 Prueba de conexión y del LED

Ejecute el siguiente script:

```
A3_prueba_LED_TCLab
```

Compruebe que MATLAB establece la conexión sin errores y que el LED responde a la secuencia programada. Una respuesta correcta confirma la comunicación básica con la tarjeta, pero todavía no demuestra que los sensores y el calentador funcionen adecuadamente.

6.3 Prueba de los sensores

Sin cambiar la conexión, ejecute:

```
A3_prueba_sensores_TCLab
```

Observe las mediciones de T_1 y T_2. Al inicio, ambas deben ser físicamente razonables y próximas a la temperatura ambiente. No es necesario que sean idénticas: cada sensor posee incertidumbre y puede existir una pequeña diferencia térmica entre sus ubicaciones. No obstante, las señales no deben presentar valores imposibles, saltos abruptos persistentes ni interrupciones de comunicación.

Registre el valor medio observado de cada sensor y la diferencia

$$\Delta T_{12} = T_1 - T_2. \tag{19}$$

Esta diferencia constituye una comprobación inicial, no una calibración de los sensores.

6.4 Prueba del calentador Q_1

Ejecute:

```
A3_prueba_Q1_TCLab
```

Durante la prueba verifique que la señal Q_1 sigue la secuencia definida por el script y que T_1 aumenta de manera gradual. Al terminar, confirme que el script devuelve Q_1 a $0\,\%$. La respuesta térmica no es instantánea: debido a la capacidad térmica del conjunto, la temperatura puede continuar aumentando brevemente aun después de reducir la potencia.

Criterio para continuar

No inicie los experimentos de identificación si alguna de las tres pruebas falla. Primero corrija la conexión, el puerto o el problema del equipo y repita la secuencia completa. Haber ejecutado el script sin que MATLAB muestre un error no es suficiente; también debe comprobarse la respuesta física esperada.

6.5 Lista de comprobación

Complete la Tabla 4 e incluya una breve descripción de cualquier anomalía observada.

Tabla 4: Resultados de la verificación previa.

Verificación	Satisfactoria	No satisfactoria	Observaciones
Conexión con la TCLab	☐	☐	
Respuesta del LED	☐	☐	
Lectura de T_1	☐	☐	
Lectura de T_2	☐	☐	
Respuesta térmica ante Q_1	☐	☐	
$Q_1 = 0\,\%$ al finalizar	☐	☐	

Actividad 3: verificación funcional de la TCLab

1. Registre la identificación de la planta y las condiciones del entorno.
2. Ejecute, en orden, los tres scripts de verificación.
3. Complete la lista de comprobación de la Tabla 4.
4. Reporte los valores iniciales de T_1, T_2 y ΔT_{12}.
5. Explique por qué una respuesta correcta del LED no garantiza por sí sola que la planta esté preparada para los experimentos.

7 Experimentos de identificación

El propósito de esta etapa es obtener dos conjuntos de datos experimentales que describan la relación dinámica entre la potencia aplicada al calentador Q_1 y la temperatura medida T_1. El primer ensayo utiliza una entrada escalón y el segundo una secuencia multinivel. Ambos conjuntos se emplearán en la Sección 8 para identificar modelos FOPDT y, posteriormente, en la Sección 9 para efectuar la validación cruzada.

Independencia de los datos

Los parámetros de un modelo deben estimarse con un conjunto de datos y evaluarse con datos que no hayan intervenido en esa estimación. Por esta razón, los dos experimentos no son repeticiones equivalentes: proporcionan perfiles de excitación distintos y permiten comprobar después si el modelo conserva su capacidad predictiva ante otra entrada.

7.1 Condiciones experimentales

Para que las diferencias observadas entre los ensayos se deban principalmente a la dinámica de la planta, mantenga constantes las condiciones que sea posible controlar:

- utilice la misma unidad TCLab en ambos experimentos;
- conserve la posición y orientación de la planta;
- mantenga $Q_2 = 0\,\%$ durante toda la adquisición;
- evite corrientes de aire, radiación solar directa y fuentes externas de calor;
- no toque ni desplace la planta mientras se registran los datos;
- registre la temperatura ambiente al inicio de cada ensayo; y
- no modifique la secuencia de Q_1 definida en las funciones suministradas.

Antes de cada experimento, compruebe que la planta se encuentra cerca del equilibrio térmico con el ambiente. Como criterio de referencia puede utilizarse

$$|T_1 - T_{\text{amb}}| \leq 1\,^{\circ}\text{C} \qquad \text{y} \qquad \left|\frac{dT_1}{dt}\right| \leq 0.05\,^{\circ}\text{C/min}, \tag{20}$$

donde la pendiente debe estimarse utilizando, al menos, los últimos dos minutos de medición. El tiempo transcurrido por sí solo no constituye un criterio suficiente: una planta que todavía se enfría puede producir una respuesta diferente aunque se espere el mismo número de minutos.

7.2 Ensayo escalón

Con la TCLab en las condiciones iniciales indicadas, identifique el puerto de comunicación y ejecute la función utilizando nombres representativos para las variables de salida. Por ejemplo:

```
datosEscalon = A4_experimento_escalon_Q1_TCLab("COM8",50);
```

Sustituya `"COM8"` por el puerto asignado a la TCLab. El segundo argumento corresponde al valor de la entrada escalón aplicado a Q_1, en porcentaje.

La función aplica la secuencia escalón definida para Q_1 y registra el tiempo, la señal aplicada y la temperatura T_1. Durante la ejecución:

1. supervise que la comunicación permanezca activa;
2. verifique que T_1 evolucione de forma continua y físicamente razonable;
3. no ejecute otras funciones que utilicen el mismo puerto; y
4. no interrumpa el ensayo salvo que aparezca una condición anormal.

Al finalizar, confirme que Q_1 regresa a 0 %. Verifique que se haya generado el archivo `.mat` correspondiente y registre su nombre exacto. El nombre debe permitir reconocer el tipo de experimento y diferenciar esta ejecución de cualquier otra. Por ejemplo:

`escalon_Q1_TCLab01_aaaammdd_HHMM.mat`

No sobrescriba ni edite manualmente los datos experimentales originales.

Conservación de los resultados

Utilice nombres diferentes para las variables de salida de cada experimento. No ejecute ambos ensayos utilizando nuevamente la misma variable `datos`, porque los resultados del primer ensayo serían sustituidos en el espacio de trabajo de MATLAB.
Utilice nombres como `datosEscalon` y `datosMultinivel`. Además, cada ejecución debe generar un archivo `.mat` independiente y con un nombre representativo. Antes de iniciar otro experimento, compruebe que el archivo de la ejecución anterior se encuentre correctamente guardado.

7.3 Enfriamiento entre ensayos

Después del ensayo escalón, mantenga ambos calentadores apagados y permita que la TCLab se enfríe naturalmente. No inicie el segundo experimento hasta satisfacer nuevamente el criterio de la Ecuación (20).

Mismo estado inicial

Una temperatura inicial próxima al ambiente no basta si T_1 todavía presenta una pendiente negativa apreciable. En ese caso, el enfriamiento residual se superpondría a la respuesta provocada por Q_1 y sesgaría la identificación.

7.4 Ensayo multinivel

Una vez restablecidas las condiciones iniciales, ejecute la función utilizando nombres de salida diferentes de los empleados en el ensayo escalón. Por ejemplo:

```
datosMultinivel = A4_experimento_multinivel_Q1_TCLab("COM8");
```

Sustituya nuevamente `"COM8"` por el puerto correspondiente a la TCLab utilizada.

Este ensayo aplica varios niveles de potencia para observar la respuesta de la planta en diferentes regiones de operación. Mantenga las mismas precauciones utilizadas durante el ensayo escalón.

Al finalizar, confirme que $Q_1 = 0\,\%$. Verifique que se haya generado un archivo `.mat` diferente del correspondiente al ensayo escalón y registre su nombre exacto. Por ejemplo:

```
multinivel_Q1_TCLab01_aaaammdd_HHMM.mat
```

7.5 Comprobación de los datos

Antes de desconectar la planta, revise los archivos y las gráficas generadas en ambos experimentos. Como mínimo, compruebe que:

- se generó un archivo `.mat` independiente para cada experimento;
- los nombres de los archivos permiten distinguir inequívocamente el ensayo escalón del ensayo multinivel;
- el vector de tiempo es estrictamente creciente;
- las señales no contienen valores `NaN` ni muestras ausentes;
- la señal registrada de Q_1 coincide con la secuencia programada;
- T_1 no presenta saltos no físicos ni discontinuidades;
- la duración del registro comprende toda la secuencia de entrada; y
- las condiciones iniciales y el código de la unidad TCLab están documentados.

Si se detecta una interrupción de comunicación, un archivo incompleto o una condición inicial inadecuada, repita el experimento después de que la planta vuelva al equilibrio. Conserve el archivo defectuoso por separado y documente la causa por la cual no se utilizará.

Tabla 5: Registro de los conjuntos de datos experimentales.

Dato	Ensayo escalón	Ensayo multinivel
Código de la TCLab		
Fecha y hora		
T_{amb} **inicial**		
T_1 **inicial**		
Duración del registro		
Nombre del archivo `.mat`		
Observaciones		

Actividad 4: adquisición de datos para identificación

1. Verifique y documente las condiciones iniciales antes de cada ensayo.
2. Ejecute el experimento escalón utilizando nombres representativos para las variables de salida.
3. Verifique y registre el nombre exacto del archivo `.mat` generado durante el ensayo escalón.
4. Permita que la planta se enfríe hasta cumplir el criterio de la Ecuación (20).
5. Ejecute el experimento multinivel utilizando variables de salida diferentes de las empleadas en el ensayo escalón.
6. Verifique y registre el nombre exacto del archivo `.mat` generado durante el ensayo multinivel.
7. Complete la Tabla 5.
8. Presente, para cada ensayo, una gráfica con $Q_1(t)$ y $T_1(t)$ en ejes claramente identificados.
9. Explique por qué un conjunto de datos diferente del empleado para identificar el modelo resulta necesario para evaluar su capacidad predictiva.

8 Identificación de modelos FOPDT

En esta etapa se estimarán dos modelos dinámicos a partir de los conjuntos de datos obtenidos en la Sección 7: uno con el ensayo escalón y otro con el ensayo multinivel. El propósito de la identificación no es reproducir todos los fenómenos térmicos de la TCLab, sino obtener una representación matemática sencilla que relacione la variación de la entrada Q_1 con la variación de la temperatura T_1.

8.1 Estructura del modelo

Se utilizará un modelo de primer orden con tiempo muerto (*First-Order Plus Dead Time*, FOPDT):

$$G_p(s) = \frac{\Delta T_1(s)}{\Delta Q_1(s)} = \frac{Ke^{-\theta s}}{\tau s + 1}. \tag{21}$$

Esta estructura se utiliza habitualmente para representar la dinámica térmica dominante de la TCLab [3].

La Ecuación (21) está expresada en variables de desviación. Por tanto,

$$\Delta T_1 = T_1 - T_{1,0}, \qquad \Delta Q_1 = Q_1 - Q_{1,0}, \tag{22}$$

donde $T_{1,0}$ y $Q_{1,0}$ representan las condiciones alrededor de las cuales se describe la respuesta. El modelo predice cambios de temperatura; la temperatura absoluta se recupera mediante

$$T_1 = T_{1,0} + \Delta T_1. \tag{23}$$

Los parámetros del modelo tienen la siguiente interpretación:

- K es la **ganancia estática**. Indica el cambio de temperatura en régimen permanente producido por un cambio unitario de la entrada:

$$K = \frac{\Delta T_{1,\text{ss}}}{\Delta Q_1}, \qquad [K] =^\circ \text{C}/\%. \tag{24}$$

- τ es la **constante de tiempo**. Una vez transcurrido el tiempo muerto, determina la rapidez con que la respuesta se aproxima al nuevo equilibrio. Para un escalón ideal, al transcurrir un tiempo τ la respuesta alcanza aproximadamente el 63.2 % de su cambio total.
- θ es el **tiempo muerto**. Representa el intervalo entre la aplicación de un cambio en Q_1 y el inicio de una variación apreciable de T_1. Puede agrupar el transporte térmico, la dinámica no representada y los retardos de medición o adquisición.

Para un escalón ΔQ_1 aplicado en $t = t_0$, la respuesta ideal del modelo es

$$\Delta T_1(t) = \begin{cases} 0, & t < t_0 + \theta, \\ K\Delta Q_1 \left[1 - e^{-(t - t_0 - \theta)/\tau}\right], & t \geq t_0 + \theta. \end{cases} \tag{25}$$

Esta expresión muestra que K fija el cambio final, θ desplaza la respuesta en el tiempo y τ determina la velocidad de aproximación al equilibrio.

Modelo empírico

El modelo FOPDT no reemplaza el balance de energía desarrollado en la Sección 4.3. El balance describe los mecanismos físicos mediante parámetros nominales; el modelo FOPDT resume la dinámica observada en una unidad TCLab y en las condiciones concretas del experimento. Por ello, sus parámetros pueden variar entre plantas y regiones de operación.

Para una planta térmica estable y con calentamiento positivo se espera, en condiciones normales,

$$K > 0, \qquad \tau > 0, \qquad \theta \geq 0. \tag{26}$$

Un resultado que incumpla estas relaciones o que produzca valores físicamente inverosímiles debe motivar una revisión de los datos, las unidades, las condiciones iniciales y la ejecución del experimento.

8.2 Identificación con el ensayo escalón

Asigne a una variable el nombre exacto del archivo generado durante el ensayo escalón. Después, ejecute la función de identificación:

```
modeloEscalon = A5_identificar_FOPDT_TCLab(archivoExperimentoEscalon);
```

Sustituya el nombre mostrado, `archivoExperimentoEscalon`, por el nombre exacto del archivo registrado en la Sección 7. Recuerde agregar la ruta completa para acceder al archivo. No utilice un archivo correspondiente a otra unidad TCLab ni cambie los datos experimentales antes de ejecutar la identificación.

Registre los valores estimados de K_{esc}, τ_{esc} y θ_{esc}, así como el indicador de ajuste reportado por la función. Conserve también la gráfica que compara la temperatura experimental con la respuesta estimada durante la identificación.

El ajuste obtenido indica el grado en que el modelo reproduce el ensayo utilizado para estimarlo. Sin embargo, no demuestra todavía que el modelo pueda predecir la respuesta ante una entrada diferente; esa comprobación se realizará en la Sección 9.

8.3 Identificación con el ensayo multinivel

Asigne a otra variable el nombre exacto del archivo correspondiente al ensayo multinivel y repita el procedimiento:

```
modeloMultinivel = A5_identificar_FOPDT_TCLab(archivoExperimentoMultinivel);
```

Registre K_{multi}, τ_{multi}, θ_{multi} y el indicador de ajuste. Utilice nombres distintos para los archivos de entrada y para los modelos identificados.

Una entrada multinivel excita la planta en varias condiciones de operación y aporta información diferente de la respuesta a un único escalón. Sin embargo, si los niveles recorren un intervalo amplio, el modelo estimado representa un compromiso entre regiones en las que la dinámica real puede variar debido a la naturaleza no lineal de la planta.

Conservación de los modelos identificados

No utilice una misma variable genérica, como `modelo`, para ejecutar ambas identificaciones, porque el segundo resultado sustituiría al primero en el espacio de trabajo de MATLAB. Conserve los resultados en las variables `modeloEscalon` y `modeloMultinivel`. Después de completar ambas identificaciones, guárdelos en un archivo `.mat` con un nombre representativo:

```
save("modelos_FOPDT_TCLab01_fecha_hora.mat", "modeloEscalon","modeloMultinivel");
```

Sustituya `TCLab01` por el código de la unidad utilizada y `fecha_hora` por la fecha y hora correspondientes. Este archivo se utilizará posteriormente durante la validación cruzada.

8.4 Comparación de los parámetros

Complete la Tabla 6 con los resultados obtenidos mediante la función suministrada.

Tabla 6: Parámetros de los modelos FOPDT identificados.

Datos utilizados	K (°C/%)	τ (s)	θ (s)	Ajuste (%)
Ensayo escalón				
Ensayo multinivel				

No se espera necesariamente que ambos modelos produzcan parámetros idénticos. Las diferencias pueden estar asociadas con el ruido de medición, el contenido de información de cada entrada, pequeñas variaciones ambientales, diferencias en el estado térmico inicial y la no linealidad de la planta.

En esta sección deben describirse y analizarse esas diferencias, pero todavía no debe seleccionarse el modelo que se utilizará para diseñar el controlador.

Criterio de selección

No seleccione un modelo únicamente porque presente el mayor ajuste con sus propios datos de identificación. El criterio principal será su capacidad para predecir el conjunto de datos que no se utilizó al estimar sus parámetros. La selección se realizará después de la validación cruzada de la Sección 9.

8.5 Identificación con System Identification Toolbox

Como segunda estrategia de identificación, utilice *System Identification Toolbox* para estimar modelos de primer orden con tiempo muerto a partir de los mismos dos conjuntos de datos.

Utilice nombres inequívocos para distinguir estos resultados de los obtenidos mediante la función suministrada. Por ejemplo:

- `modeloEscalonSIT` para el modelo obtenido con los datos del ensayo escalón; y
- `modeloMultinivelSIT` para el modelo obtenido con los datos del ensayo multinivel.

Mantenga correctamente las unidades de tiempo, entrada y salida. Para cada modelo, utilice únicamente el conjunto de datos correspondiente y conserve el otro conjunto para la validación.

Compare los parámetros obtenidos con los estimados mediante `A5_identificar_FOPDT_TCLab`. Las diferencias deben analizarse considerando el método de estimación, el tratamiento de las condiciones iniciales y el intervalo de datos utilizado, no únicamente mediante una comparación numérica.

Los indicadores de ajuste producidos por métodos diferentes solo deben compararse directamente si emplean la misma definición, el mismo intervalo temporal y los mismos datos experimentales.

Salvo indicación contraria, la validación cruzada de la Sección 9 se realizará con `modeloEscalon` y `modeloMultinivel`. Los modelos obtenidos mediante *System Identification Toolbox* se utilizarán para comparar los métodos de estimación.

Actividad 5: identificación de modelos FOPDT

1. Ejecute `A5_identificar_FOPDT_TCLab` utilizando el archivo correspondiente al ensayo escalón.
2. Repita la identificación utilizando el archivo correspondiente al ensayo multinivel.
3. Registre los parámetros, las unidades y el ajuste de cada modelo en la Tabla 6.
4. Presente las gráficas generadas durante ambas identificaciones.
5. Compruebe que los parámetros obtenidos sean coherentes con una planta térmica estable y explique cualquier resultado atípico.
6. Compare ambos conjuntos de parámetros y analice las posibles causas de sus diferencias.
7. Repita ambas identificaciones mediante *System Identification Toolbox* y compare los resultados con los obtenidos mediante la función suministrada.
8. Guarde `modeloEscalon` y `modeloMultinivel` en un archivo `.mat` con un nombre representativo para utilizarlos posteriormente en la validación cruzada.

9 Validación cruzada

La identificación determina los parámetros que mejor representan un conjunto de datos, mientras que la validación evalúa si el modelo conserva su capacidad predictiva ante datos que no participaron en la estimación. Por esta razón, los dos modelos obtenidos en la Sección 8 se evaluarán mediante los cruces presentados en la Tabla 7.

Tabla 7: Esquema de validación cruzada de los modelos FOPDT.

Modelo	Datos usados para identificar	Datos usados para validar
`modeloEscalon`	Ensayo escalón	Ensayo multinivel
`modeloMultinivel`	Ensayo multinivel	Ensayo escalón

Para cada validación se utiliza la entrada experimental $Q_1(t)$ del otro ensayo, se simula el modelo sin modificar K, τ ni θ, y se compara la temperatura predicha con la temperatura experimental medida.

Modelo fijo durante la validación

La función `A6_validar_FOPDT_TCLab` no modifica ni vuelve a estimar K, τ o θ. Si los parámetros se optimizaran utilizando el archivo de validación, los datos dejarían de ser independientes y se estaría realizando una nueva identificación, no una validación.

9.1 Modelo escalón frente a datos multinivel

Utilice el modelo identificado con el ensayo escalón y el archivo experimental correspondiente al ensayo multinivel:

```
validacionEscalonEnMulti = A6_validar_FOPDT_TCLab(modeloEscalon,
    archivoExperimentoMultinivel);
```

Sustituya el nombre mostrado por el nombre exacto del archivo generado en la Sección 7. La función conserva los parámetros de `modeloEscalon`, utiliza la entrada multinivel experimental y simula la respuesta a partir de la condición inicial medida antes del primer cambio de Q_1.

Este cruce permite comprobar si un modelo identificado mediante un único escalón puede predecir el comportamiento de la planta ante diferentes niveles de potencia.

9.2 Modelo multinivel frente a datos escalón

Realice el cruce recíproco utilizando el modelo identificado con el ensayo multinivel y los datos experimentales del ensayo escalón:

```
validacionMultiEnEscalon = A6_validar_FOPDT_TCLab(modeloMultinivel,
    archivoExperimentoEscalon);
```

Este cruce permite comprobar si un modelo identificado con diferentes niveles de potencia puede representar adecuadamente la respuesta de la planta ante una entrada escalón.

Antes de interpretar los resultados, confirme en la salida de MATLAB que el archivo empleado para validar cada modelo es distinto del utilizado para identificarlo. Los dos modelos y los dos conjuntos de datos deben provenir de la misma unidad física TCLab.

Conservación de los resultados

Utilice nombres diferentes para los resultados de cada validación: `validacionEscalonEnMulti` y `validacionMultiEnEscalon`. No reutilice una variable genérica como `validacion`, porque el segundo resultado sustituiría al primero en el espacio de trabajo de MATLAB.
Después de completar ambos cruces, guarde los resultados en un archivo `.mat` con un nombre representativo:

```
save("validaciones_FOPDT_TCLab01_fecha_hora.mat");
```

9.3 Gráficas y residuos

Para cada cruce, la función genera tres gráficas:

- temperatura experimental y temperatura predicha por el modelo FOPDT;
- entrada experimental $Q_1(t)$ utilizada en la simulación; y
- residuo de predicción,

$$e(k) = T_{1,\text{exp}}(k) - T_{1,\text{mod}}(k). \tag{27}$$

Además de comprobar la cercanía entre las curvas, examine la estructura temporal del residuo. Un residuo pequeño y distribuido alrededor de cero es compatible con una buena capacidad predictiva.

En cambio, los siguientes comportamientos pueden revelar deficiencias del modelo:

- un valor medio diferente de cero indica una tendencia sistemática a sobreestimar o subestimar la temperatura;
- una tendencia creciente o decreciente puede indicar que la ganancia o la constante de tiempo no representan adecuadamente la planta;
- un error concentrado al inicio de cada transición puede indicar una representación inadecuada del tiempo muerto; y
- un patrón que cambia con el nivel de Q_1 puede indicar dependencia con la región de operación o dinámica no lineal no representada.

9.4 Indicadores de validación

Durante la validación se calculan los indicadores de la Tabla 8. Para N muestras se define

$$\begin{aligned} y(k) &= T_{1,\text{exp}}(k), \qquad \widehat{y}(k) = T_{1,\text{mod}}(k), \\ \overline{y} &= \frac{1}{N}\sum_{k=1}^{N} y(k), \quad e(k) = y(k) - \widehat{y}(k), \end{aligned} \tag{28}$$

donde $y(k)$ corresponde a la temperatura medida experimentalmente, $\widehat{y}(k)$ a la temperatura predicha por el modelo, $e(k)$ al error de predicción y $\overline{y}$ al promedio de las mediciones experimentales.

Tabla 8: Indicadores calculados durante la validación.

Indicador	Interpretación
RMSE	Mide la magnitud del error y penaliza con mayor peso los errores de gran magnitud. Conserva las unidades de la temperatura.
MAE	Promedio de la magnitud absoluta del error. Expresa directamente el error medio en °C.
Sesgo	Promedio de $e(k)$. Permite detectar una tendencia sistemática a sobreestimar o subestimar la temperatura.
SSE	Suma de los errores al cuadrado. Representa el error cuadrático acumulado y depende del número de muestras.
R^2	Indica qué tan bien la predicción reproduce la variabilidad observada en los datos experimentales. Un valor próximo a uno indica mayor concordancia.
Ajuste	Compara porcentualmente el error del modelo con la variabilidad de los datos experimentales respecto de su media. Un valor de 100 % corresponde a una predicción perfecta.

Las expresiones utilizadas son

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e^2(k)}, \qquad \text{MAE} = \frac{1}{N}\sum_{k=1}^{N} |e(k)|, \tag{29}$$

$$\text{Sesgo} = \frac{1}{N}\sum_{k=1}^{N} e(k), \qquad \text{SSE} = \sum_{k=1}^{N} e^2(k), \tag{30}$$

$$R^2 = 1 - \frac{\sum_{k=1}^{N} e^2(k)}{\sum_{k=1}^{N}[y(k) - \overline{y}]^2}, \qquad \text{Ajuste} = 100\left(1 - \frac{\|y - \widehat{y}\|_2}{\|y - \overline{y}\|_2}\right). \tag{31}$$

Aunque normalmente se buscan valores próximos a uno para R^2 y próximos al 100 % para el ajuste, ambos indicadores pueden tomar valores negativos. Esto significa que el modelo predice peor que utilizar la media de los datos experimentales, $\overline{y}$, como estimación constante.

El R^2 y el ajuste contienen información relacionada, ya que ambos comparan el error de predicción con la variabilidad de los datos experimentales respecto de su media. Por esta razón, no deben interpretarse como indicadores completamente independientes.

El signo del sesgo depende de la definición $e(k) = y(k) - \widehat{y}(k)$. Por tanto, un sesgo positivo indica que, en promedio, el modelo subestima la temperatura experimental, mientras que un sesgo negativo indica una sobreestimación.

Los indicadores se reportan para dos intervalos:

- **Datos completos:** incluye el periodo inicial anterior al primer cambio de Q_1.

- **Desde el primer cambio:** evalúa específicamente la respuesta dinámica producida por la entrada de validación.

Para comparar el desempeño dinámico utilice principalmente el segundo intervalo. El registro completo sirve como comprobación adicional de la condición inicial y de la calidad global de la simulación.

Lectura de las métricas

Dentro de una misma validación, valores menores de RMSE y MAE, un sesgo de menor magnitud y próximo a cero, junto con valores mayores de R^2 y ajuste, indican una mayor concordancia entre el modelo y los datos experimentales.
La SSE depende del número de muestras y no debe utilizarse para comparar directamente experimentos de distinta duración. Además, los dos cruces emplean entradas experimentales diferentes; por ello, la comparación entre los modelos debe considerar también las gráficas, los residuos, la región de operación y la coherencia física de los parámetros.

Registre las métricas calculadas desde el primer cambio en las Tablas 9 y 10. Conserve también las tablas completas generadas por MATLAB.

Tabla 9: Desempeño predictivo en la validación cruzada.

Cruce de validación	RMSE (°C)	MAE (°C)	R^2	Ajuste (%)
Escalón → multinivel				
Multinivel → escalón				

Tabla 10: Error acumulado y sesgo en la validación cruzada.

Cruce de validación	Sesgo (°C)	SSE ((°C)^2)
Escalón → multinivel		
Multinivel → escalón		

9.5 Selección del modelo nominal

Después de analizar ambos cruces, seleccione el modelo que se empleará como planta nominal para diseñar el controlador PI. La selección no debe realizarse comparando una única métrica, porque cada modelo se valida mediante un experimento diferente.

Aplique el siguiente orden de análisis:

1. Compruebe que los parámetros sean físicamente coherentes:

$$K > 0, \qquad \tau > 0, \qquad \theta \geq 0. \tag{32}$$

2. Analice principalmente las métricas calculadas desde el primer cambio de Q_1.

3. Compruebe si el modelo reproduce adecuadamente la amplitud de la respuesta, el tiempo muerto y la rapidez de las transiciones.
4. Examine la magnitud y la estructura temporal de los residuos, prestando especial atención al sesgo y a los patrones relacionados con los cambios de entrada.
5. Determine si la región cubierta por los datos de identificación y validación es representativa de aquella en la que operará el sistema de control.

La decisión debe tomarse de acuerdo con los criterios resumidos en la Tabla 11.

Tabla 11: Criterios para seleccionar el modelo nominal.

Resultado de la validación	**Decisión recomendada**
Un modelo presenta mejor capacidad predictiva, residuos adecuados y parámetros físicamente coherentes.	Seleccionarlo como modelo nominal.
Ambos modelos presentan resultados similares y aceptables.	Seleccionar el modelo más representativo de la región donde operará el controlador y documentar que ambos modelos son plausibles.
Las métricas favorecen modelos diferentes o los residuos conducen a conclusiones contradictorias.	Priorizar la representación del tiempo muerto, la ausencia de sesgo, la región de operación y el análisis gráfico. Indicar que la evidencia no permite establecer un modelo claramente superior.
Ambos modelos presentan errores importantes, sesgo o patrones sistemáticos en los residuos.	No seleccionar ninguno. Revisar los experimentos, repetir la identificación o adquirir un nuevo conjunto de datos.

Si los resultados son contradictorios y se requiere una comparación rigurosa, ambos modelos deberían evaluarse mediante un tercer experimento independiente, reservado exclusivamente para validación y común para los dos modelos.

Para esta práctica, si ambos modelos presentan un desempeño aceptable, puede seleccionarse el que represente mejor la región de operación prevista para el controlador, siempre que la decisión se justifique mediante las métricas, las gráficas y los residuos.

No promediar los parámetros

No construya el modelo nominal promediando los valores de K, τ y θ de los dos modelos. Ese promedio no corresponde a un modelo identificado ni garantiza una mejor representación de la planta.

Exprese la planta seleccionada en la forma

$$G_{p,\mathrm{nom}}(s) = \frac{K_{\mathrm{nom}} e^{-\theta_{\mathrm{nom}} s}}{\tau_{\mathrm{nom}} s + 1}, \tag{33}$$

e indique los valores numéricos y las unidades de K_{nom}, τ_{nom} y θ_{nom}. Este será el modelo utilizado en la siguiente etapa; no debe diseñarse el controlador antes de completar y justificar esta selección.

Actividad 6: validación cruzada y selección del modelo

1. Realice ambos cruces de validación: `modeloEscalon` frente a los datos multinivel y `modeloMultinivel` frente a los datos escalón.
2. Compruebe que en ambos casos se mantuvieron fijos K, τ y θ, y que los datos de validación no fueron utilizados durante la identificación.
3. Presente las gráficas de temperatura, entrada y residuos generadas para cada cruce.
4. Complete las Tablas 9 y 10, e incluya las tablas completas reportadas por MATLAB.
5. Compare las métricas obtenidas para los datos completos y para el intervalo iniciado con el primer cambio de Q_1. Analice también el sesgo y cualquier patrón temporal presente en los residuos.
6. Determine si uno de los modelos presenta una capacidad predictiva claramente superior o si ambos ofrecen resultados similares.
7. Seleccione justificadamente el modelo nominal y escriba $G_{p,\mathrm{nom}}(s)$ con sus parámetros y unidades. Si ninguno resulta adecuado, revise los experimentos antes de continuar.
8. Guarde los resultados de ambas validaciones en un archivo `.mat` con un nombre representativo.

10 Diseño del controlador PI mediante SIMC

En esta etapa se utilizará el modelo FOPDT seleccionado mediante la validación cruzada para diseñar un controlador proporcional–integral (PI). La planta nominal debe estar expresada como

$$G_{p,\mathrm{nom}}(s) = \frac{Ke^{-\theta s}}{\tau s + 1}, \tag{34}$$

donde los valores de K, τ y θ provienen de la Ecuación (33). No se deben sustituir por los parámetros del modelo físico nominal ni por los correspondientes a otra unidad TCLab.

Esta etapa solo debe iniciarse después de comprobar que el modelo nominal presenta parámetros físicamente coherentes y una capacidad predictiva aceptable. Si ninguno de los modelos supera la validación, deben revisarse los experimentos y la identificación antes de diseñar el controlador.

10.1 Fundamento del método

El *Internal Model Control* (IMC) diseña el controlador utilizando explícitamente un modelo de la planta. En términos conceptuales, se compensa la parte invertible del modelo y se incorpora un filtro que determina la rapidez deseada del lazo cerrado.

La constante de tiempo de ese filtro, representada aquí por τ_c, establece el principal compromiso del diseño:

- un τ_c pequeño produce una respuesta más rápida, pero exige una mayor acción de control y aumenta la sensibilidad frente al ruido y los errores del modelo;
- un τ_c grande produce una respuesta más lenta y, generalmente, más robusta frente a la incertidumbre y al tiempo muerto no modelado.

Las reglas SIMC (*Simple IMC*, también conocidas como *Skogestad IMC*) parten de esta idea y la traducen en expresiones analíticas sencillas para controladores PI y PID convencionales [4].

En esta práctica no se implementará una arquitectura IMC explícita. Se calcularán los parámetros obtenidos mediante las reglas SIMC y se utilizará el controlador PI en una estructura convencional de realimentación.

10.2 Estructura del controlador PI

Se empleará la forma paralela continua

$$G_c(s) = K_p + \frac{K_i}{s} = K_p\left(1 + \frac{1}{T_i s}\right), \tag{35}$$

con

$$K_i = \frac{K_p}{T_i}. \tag{36}$$

Para la TCLab, el error $e = T_{\mathrm{ref}} - T_1$ se expresa en grados Celsius y la salida del controlador corresponde a una variación porcentual de la potencia aplicada al calentador. En consecuencia,

$$[K_p] = \frac{\%}{^\circ\mathrm{C}}, \qquad [T_i] = \mathrm{s}, \qquad [K_i] = \frac{\%}{^\circ\mathrm{C\,s}}. \tag{37}$$

La acción integral permite eliminar el error estacionario ante una referencia constante en el modelo nominal, siempre que el lazo sea estable y el actuador disponga de suficiente autoridad. Esta propiedad no evita por sí sola la saturación ni la acumulación excesiva de la acción integral.

Control en variables de desviación

El modelo FOPDT y el controlador se diseñan en variables de desviación. Por tanto, la salida del PI corresponde a

$$\Delta Q_1 = Q_1 - Q_{1,0}. \tag{38}$$

La entrada absoluta aplicada al calentador se recupera mediante

$$Q_1 = Q_{1,0} + \Delta Q_1. \tag{39}$$

De manera análoga,

$$e = \Delta T_{\mathrm{ref}} - \Delta T_1 = T_{\mathrm{ref}} - T_1. \tag{40}$$

Si el punto de operación corresponde al calentador apagado, $Q_{1,0} = 0$ y, por tanto, $\Delta Q_1 = Q_1$.

10.3 Reglas de sintonización SIMC

Para la planta FOPDT de la Ecuación (34), las reglas SIMC para un controlador PI son

$$K_p = \frac{1}{K}\frac{\tau}{\tau_c + \theta}, \tag{41}$$

$$T_i = \min\{\tau,\, 4(\tau_c + \theta)\}\,, \tag{42}$$

$$K_i = \frac{K_p}{T_i}. \tag{43}$$

El parámetro τ_c es la principal decisión de sintonización. Las reglas SIMC suelen utilizar $\tau_c = \theta$ para obtener una respuesta rápida con robustez razonable. En esta práctica se adoptará una configuración más suave:

$$\boxed{\tau_c = 2\theta}. \tag{44}$$

Al utilizar un valor mayor que el tiempo muerto, se reduce la agresividad del controlador y la sensibilidad frente a errores del modelo, a cambio de una respuesta más lenta.

Esta elección no constituye una especificación exacta del tiempo de establecimiento. La respuesta real también depende de la aproximación FOPDT, de la saturación del calentador y de las diferencias entre la planta identificada y el modelo.

Condición sobre el tiempo muerto

La elección $\tau_c = 2\theta$ presupone que $\theta > 0$. Si la identificación produce $\theta = 0$, entonces $\tau_c = 0$ y el cálculo de K_p queda indefinido. En ese caso, revise la identificación o seleccione un valor positivo de τ_c debidamente justificado antes de continuar.

Parámetros del modelo correcto

Calcule el controlador con el modelo nominal seleccionado al final de la Sección 9. Si se modifica K, τ o θ, debe repetirse la sintonización. No mezcle parámetros procedentes de los dos modelos identificados.

10.4 Cálculo en MATLAB

Introduzca los parámetros del modelo nominal validado en el archivo `A7_SIMC_controller_TCLab.m`. Revise los cálculos implementados y ejecute el archivo. El procedimiento mínimo debe ser equivalente a

```
% Parametros del modelo nominal validado
K = ...; % degC/%
tau = ...; % s
theta = ...; % s

if theta <= 0
    error("El metodo requiere theta > 0.");
end

% Sintonia PI mediante SIMC
tau_c = 2*theta;
Kp = tau/(K*(tau_c + theta));
Ti = min(tau, 4*(tau_c + theta));
Ki = Kp/Ti;

% Controlador PI continuo
Gc = tf([Kp Ki],[1 0]);
```

Compruebe que K_p, T_i y K_i sean positivos y que sus unidades coincidan con la Ecuación (37). Registre los resultados en la Tabla 12.

Tabla 12: Modelo nominal y parámetros del controlador PI–SIMC.

Parámetro de planta	Valor	**Parámetro de diseño**	Valor
K (°C/%)		τ_c (s)	
τ (s)		K_p (%/°C)	
θ (s)		T_i (s)	
		K_i (%/(°C s))	

Después de comprobar los resultados, guarde los parámetros del modelo y del controlador en un

archivo `.mat` con un nombre representativo:

```
save("PI_SIMC_TCLab01_fecha_hora.mat", ...
    "K","tau","theta","tau_c", ...
    "Kp","Ti","Ki","Gc");
```

Sustituya `TCLab01` por el código de la unidad utilizada y `fecha_hora` por la fecha y hora correspondientes. Este archivo permite conservar los parámetros del modelo nominal y del controlador obtenidos durante el diseño.

10.5 Implementación discreta

Las reglas SIMC proporcionan los parámetros de un controlador continuo. En la evaluación MIL, el PI se implementará de manera discreta utilizando un periodo de muestreo T_s.

La contribución integral se actualizará mediante

$$I[k+1] = I[k] + K_i T_s e[k], \tag{45}$$

y la salida del controlador, antes de aplicar saturación, será

$$\Delta Q_{1,\mathrm{cmd}}[k] = K_p e[k] + I[k]. \tag{46}$$

Ganancia integral continua

La cantidad $K_i T_s$ representa el incremento integral por muestra. No debe confundirse con la ganancia continua K_i obtenida mediante SIMC.
Los parámetros K_p, T_i y K_i calculados en esta sección no deben modificarse al implementar el controlador discreto.

10.6 Alcance del diseño

La sintonización proporciona un punto de partida basado en el modelo, pero no demuestra por sí sola que el sistema cumpla las especificaciones de desempeño. En la siguiente sección se evaluará el controlador mediante MIL y se comprobará si satisface

$$t_s \leq 600\,\mathrm{s}, \qquad M_p \leq 5\,\%, \tag{47}$$

además del error estacionario, la señal de control y el efecto de la saturación.

Si los requisitos no se cumplen, cualquier cambio de τ_c deberá estar justificado y el controlador deberá evaluarse nuevamente utilizando las mismas condiciones de prueba.

Actividad 7: diseño del controlador PI mediante SIMC

1. Escriba el modelo nominal seleccionado después de la validación cruzada e indique de cuál ensayo procede.
2. Compruebe que $K > 0$, $\tau > 0$ y $\theta > 0$.
3. Establezca $\tau_c = 2\theta$ y calcule K_p, T_i y K_i mediante las Ecuaciones (41)–(43).
4. Revise y ejecute `A7_SIMC_controller_TCLab.m`, y compruebe que sus resultados coincidan con el cálculo anterior.
5. Complete la Tabla 12, incluidas todas las unidades.
6. Construya $G_c(s)$ en la forma de la Ecuación (35) y reporte su expresión numérica.
7. Explique cualitativamente cómo cambiarían la rapidez, la acción de control y la robustez si se redujera o aumentara τ_c.
8. Explique la diferencia entre K_i y el incremento integral $K_i T_s$ utilizado en la implementación discreta.
9. Guarde los parámetros del modelo nominal y del controlador PI en un archivo `.mat` con un nombre representativo.
10. Conserve los valores de K, τ, θ, K_p, T_i, K_i y τ_c ya que deberán incorporarse en `A8_init_TCLab_SIMC.m` para configurar la evaluación MIL.

11 Simulación y evaluación Model-in-the-Loop

En esta etapa se conectará el modelo del controlador PI–SIMC con el modelo FOPDT nominal de la planta dentro de Simulink. Tanto el controlador como la planta son todavía modelos ejecutables; no se genera ni se prueba código C/C++ y tampoco se utiliza el procesador que ejecutará el controlador final. Por tanto, la prueba corresponde a una evaluación *Model-in-the-Loop* (MIL).

El propósito es comprobar el comportamiento del lazo cerrado bajo condiciones reproducibles y estudiar cómo cambian los resultados al incorporar dos características necesarias en una implementación real: los límites del actuador y la protección contra la acumulación del integrador.

11.1 Arquitectura del modelo

Abra el archivo `A8_TCLab_MIL_SIMC.slx`. El modelo contiene tres lazos cerrados equivalentes que reciben la misma referencia y emplean copias idénticas del modelo FOPDT seleccionado en la Sección 9. La única diferencia entre los lazos corresponde al tratamiento de la señal de control.

Tabla 13: Casos de evaluación del controlador PI–SIMC.

Caso	Límites del actuador	Tratamiento del integrador
PI ideal	No se limita Q_{cmd}.	El integrador evoluciona sin restricciones.
Saturación sin anti-windup	$Q_1 = \text{sat}(Q_{\text{cmd}}, 0, 90)\,\%$.	El integrador continúa acumulándose durante la saturación.
Saturación con anti-windup	$Q_1 = \text{sat}(Q_{\text{cmd}}, 0, 90)\,\%$.	Se emplea anti-windup mediante *clamping*.

En los tres casos, la planta conserva los mismos valores identificados de K, τ y θ. El modelo continuo nominal puede expresarse como

$$\Delta T_1(s) = \frac{K e^{-\theta s}}{\tau s + 1} Q_1(s), \qquad T_1(t) = T_{\text{amb}} + \Delta T_1(t), \tag{48}$$

donde ΔT_1 representa el incremento de temperatura respecto al ambiente. Esta expresión es consistente con un experimento que comienza con $Q_1 = 0\,\%$ y la planta en equilibrio térmico con el entorno. Si el modelo identificado utiliza otro punto base, deberán incorporarse explícitamente sus valores de operación para Q_1 y T_1.

Para la simulación discreta, la dinámica de primer orden se discretiza mediante un retenedor de orden cero. El tiempo muerto se implementa mediante un bloque *Integer Delay*. Debido a que el valor identificado de θ no tiene por qué ser un múltiplo entero del periodo de muestreo, se define

$$N_{\text{delay}} = \text{round}\left(\frac{\theta}{T_s}\right), \qquad \theta_d = N_{\text{delay}} T_s, \tag{49}$$

donde θ_d es el tiempo muerto efectivamente representado en el modelo discreto.

La sintonización SIMC conserva el valor identificado de θ. La aproximación θ_d se utiliza únicamente para implementar el retardo en el modelo discreto de la planta.

Comparación controlada

No modifique entre casos la referencia, el modelo de la planta, la condición inicial, el tiempo de simulación ni la configuración numérica. De lo contrario, las diferencias observadas no podrán atribuirse únicamente a la saturación y al anti-windup.

11.2 Implementación discreta del PI

El algoritmo de control se ejecutará con un periodo de muestreo

$$T_s = 1\,\mathrm{s}. \tag{50}$$

La forma discreta del PI paralelo implementada mediante integración *Forward Euler* puede escribirse como

$$Q_{\mathrm{cmd}}[k] = K_p e[k] + I[k], \tag{51}$$
$$I[k+1] = I[k] + K_i T_s e[k], \tag{52}$$
$$e[k] = T_{\mathrm{ref}}[k] - T_1[k]. \tag{53}$$

En el caso ideal, Q_{cmd} se aplica directamente al modelo de la planta. Este caso sirve como referencia académica, pero deja de representar un actuador realizable cuando la señal sale del intervalo permitido.

En los otros dos casos se aplica

$$Q_1[k] = \min\{90, \max\{0, Q_{\mathrm{cmd}}[k]\}\}\,. \tag{54}$$

Si el integrador continúa actualizándose mientras Q_1 está limitado, puede acumular un valor excesivo. Este fenómeno, denominado *integral windup*, puede prolongar la recuperación después de abandonar la saturación y aumentar el sobreimpulso.

En la implementación con *clamping*, la integración se suspende únicamente cuando la salida se encuentra saturada y el error intenta llevarla aún más lejos del intervalo permitido [5]:

$$I[k+1] = \begin{cases} I[k], & Q_{\mathrm{cmd}}[k] \geq 90 \ \wedge \ e[k] > 0, \\ I[k], & Q_{\mathrm{cmd}}[k] \leq 0 \ \wedge \ e[k] < 0, \\ I[k] + K_i T_s e[k], & \text{en otro caso.} \end{cases} \tag{55}$$

Cuando el error favorece el regreso de la señal al intervalo admisible, el integrador debe volver a actualizarse. Por ello, anti-windup no significa desactivar permanentemente la acción integral.

Para identificar el *windup* no basta observar que Q_1 permanece saturada. Debe comprobarse que el controlador continúa solicitando una acción de control que el actuador no puede aplicar mientras el integrador sigue acumulando error.

11.3 Condiciones de simulación

El archivo `A8_TCLab_MIL_SIMC.slx` ejecuta automáticamente `A8_init_TCLab_SIMC.m` mediante la función de inicialización `InitFcn` del modelo. Este archivo carga los parámetros de la planta y del controlador, genera el modelo discreto y establece las condiciones comunes de simulación.

Antes de ejecutar el modelo, revise que los valores de K, τ, θ, K_p, T_i y K_i coincidan con los obtenidos en las Secciones 9 y 10.

Ejecute los tres lazos en una misma simulación con las condiciones de la Tabla 14.

Tabla 14: Condiciones comunes para la prueba MIL.

Condición	Valor o criterio
Temperatura inicial	$T_1(0) = T_{\text{amb}}$.
Referencia principal	Cambio desde T_{amb} hasta $T_{\text{ref}} = 60\,^{\circ}\text{C}$.
Límites de Q_1	$0\,\% \leq Q_1 \leq 90\,\%$.
Periodo de muestreo	$T_s = 1\,\text{s}$.
Tiempo de simulación	$t_{\text{final}} = 1800\,\text{s}$.
Solver	`FixedStepDiscrete`.
Paso fijo	$T_s = 1\,\text{s}$.
Límite térmico de análisis	$T_1 \leq 90\,^{\circ}\text{C}$.

Registre las tres temperaturas y las señales de potencia disponibles. Para el caso ideal, Q_{cmd} coincide con la potencia aplicada a la planta. Para el caso con saturación sin anti-windup se registran tanto Q_{cmd} como Q_1, lo que permite observar directamente la diferencia entre la potencia solicitada por el controlador y la que puede aplicar el actuador.

En el caso con anti-windup, el bloque *Discrete PID Controller* aplica la saturación internamente y entrega directamente Q_1. La señal interna Q_{cmd} anterior a la saturación no se utiliza para el análisis.

11.4 Métricas de desempeño

Después de ejecutar el modelo, utilice `A8_validar_TCLab_SIMC.m`. El script calcula métricas de desempeño dinámico y de utilización del actuador para los tres casos. Los tiempos de subida y establecimiento, así como el sobreimpulso, se calculan respecto al cambio real entre la temperatura inicial y la referencia [6].

Los resultados se presentan en dos grupos. El primero resume el desempeño dinámico:

- tiempo de subida;
- tiempo de establecimiento;
- sobreimpulso; y
- error estacionario.

El error estacionario se estima a partir del promedio de la temperatura durante los últimos 60 s de simulación:

$$e_{\text{ss}} = T_{\text{ref}} - \frac{1}{N_f} \sum_{k \in \mathcal{I}_f} T_1[k], \tag{56}$$

donde $\mathcal{I}_f$ contiene las muestras correspondientes a los últimos 60 s y N_f es el número de muestras incluidas en ese intervalo. Este criterio permite evaluar el error estacionario numéricamente y evita depender únicamente de la inspección visual de la respuesta.

El segundo grupo resume las restricciones físicas y la utilización del actuador:

- temperatura máxima;
- potencia mínima aplicada;
- potencia máxima aplicada; y
- tiempo durante el cual Q_{cmd} permanece fuera del rango realizable, cuando esta señal está disponible.

Aunque el script presenta las métricas en dos grupos para facilitar su lectura en la ventana de comandos, la Tabla 15 las reúne en un único registro comparativo.

Tabla 15: Resultados de la evaluación MIL.

Métrica	PI ideal	Saturación sin AW	Saturación con AW
Tiempo de subida (s)			
Tiempo de establecimiento (s)			
Sobreimpulso (%)			
Error estacionario (°C)			
Temperatura máxima (°C)			
$Q_{1,\text{min}}$ **aplicado (%)**			
$Q_{1,\text{max}}$ **aplicado (%)**			
Tiempo con Q_{cmd} fuera de rango (s)			No disponible

El tiempo con Q_{cmd} fuera de rango no se reporta para el caso con anti-windup, porque el bloque utilizado aplica la saturación internamente y no expone la señal anterior a esta. Esto no constituye un error de simulación.

Para el caso físicamente realizable, con saturación y anti-windup, el script comprueba automáticamente los siguientes requisitos:

$$t_s \leq 600\,\text{s}, \tag{57}$$
$$M_p \leq 5\,\%, \tag{58}$$
$$0\,\% \leq Q_1 \leq 90\,\%, \tag{59}$$
$$T_1 \leq 90\,^{\circ}\text{C}. \tag{60}$$

Interpretación del caso ideal

Que el PI ideal presente mejores métricas de temperatura no demuestra que sea la implementación preferible. Si requiere $Q_{\text{cmd}} > 90\,\%$ o $Q_{\text{cmd}} < 0\,\%$, su respuesta depende de una acción de control que la TCLab no puede producir. El caso ideal se conserva como referencia para medir el efecto de las restricciones físicas.

11.5 Análisis comparativo

La comparación debe responder, al menos, las siguientes preguntas:

- ¿Durante cuánto tiempo solicita el PI una potencia fuera del rango realizable?
- ¿Cómo cambia la respuesta cuando la potencia aplicada se limita a 0 %–90 %?
- ¿Qué evidencia muestra la acumulación del integrador en el caso sin anti-windup?
- ¿Cómo modifica el anti-windup el sobreimpulso y el tiempo de establecimiento?
- ¿Cuál caso constituye una implementación físicamente realizable y cumple simultáneamente las especificaciones de desempeño?

No se debe concluir de antemano que el anti-windup mejora todas las métricas. Su efecto depende de la duración de la saturación, de la sintonización y del mecanismo utilizado. La conclusión debe apoyarse en las señales y los valores obtenidos en la simulación.

Actividad 8: simulación y evaluación MIL

1. Revise `A8_init_TCLab_SIMC.m` y compruebe que el modelo nominal y los parámetros del PI coincidan con los obtenidos en las Secciones 9 y 10.
2. Compruebe los valores de N_{delay} y θ_d utilizados para representar discretamente el tiempo muerto.
3. Abra `A8_TCLab_MIL_SIMC.slx` e identifique los tres subsistemas: PI ideal, saturación sin anti-windup y saturación con anti-windup.
4. Verifique que los tres lazos utilicen la misma referencia, planta, condición inicial, tiempo de simulación y configuración numérica.
5. Ejecute la prueba principal desde T_{amb} hasta $T_{\text{ref}} = 60\,^{\circ}\text{C}$.
6. Presente en una misma figura T_{ref} y las tres temperaturas. Compare también las potencias aplicadas en los tres casos y, para el caso sin anti-windup, presente conjuntamente Q_{cmd} y Q_1.
7. Ejecute `A8_validar_TCLab_SIMC.m`, incluya los resultados generados por MATLAB y complete la Tabla 15.
8. Compruebe el tiempo de establecimiento, el sobreimpulso, el error estacionario, los límites de potencia y el límite térmico.
9. Explique las diferencias entre los tres casos a partir de la saturación y del comportamiento del integrador, no solamente de la forma de las curvas.
10. Seleccione justificadamente la configuración PI–SIMC que podría continuar hacia una implementación y señale cualquier requisito que todavía no se cumpla.

12 Diseño alternativo con Control System Designer

Como cierre de la práctica, diseñe un segundo controlador PI mediante *Control System Designer*. El propósito es obtener una sintonización alternativa e independiente del diseño SIMC desarrollado en la Sección 10, y comparar posteriormente ambas soluciones bajo las mismas condiciones de evaluación MIL.

Para esta actividad no se proporciona un procedimiento paso a paso. El estudiante deberá seleccionar y justificar las herramientas de diseño y análisis que considere apropiadas dentro de *Control System Designer*.

12.1 Problema de diseño

Utilice el modelo FOPDT nominal seleccionado en la Sección 9:

$$G_{p,\mathrm{nom}}(s) = \frac{K e^{-\theta s}}{\tau s + 1}. \tag{61}$$

El modelo utilizado durante el diseño debe conservar el tiempo muerto identificado. El controlador deberá tener estructura PI:

$$G_{c,\mathrm{CSD}}(s) = K_{p,\mathrm{CSD}} + \frac{K_{i,\mathrm{CSD}}}{s}. \tag{62}$$

El diseño deberá buscar el cumplimiento de los mismos requisitos utilizados para evaluar el controlador PI–SIMC:

$$t_s \leq 600\,\mathrm{s}, \qquad M_p \leq 5\,\%. \tag{63}$$

La estrategia utilizada para obtener el controlador deberá ser diferente de la sintonización SIMC empleada en la Sección 10. No se considerará un diseño alternativo aquel que reproduzca las mismas reglas de sintonización.

12.2 Resultado del diseño

Exporte el controlador obtenido al espacio de trabajo de MATLAB y reporte su función de transferencia. Determine además

$$T_{i,\mathrm{CSD}} = \frac{K_{p,\mathrm{CSD}}}{K_{i,\mathrm{CSD}}}. \tag{64}$$

Complete la Tabla 16 con los resultados obtenidos para ambos diseños.

Tabla 16: Comparación de los diseños PI.

Parámetro o característica	PI–SIMC	PI–CSD
K_p (%/°C)		
K_i (%/(°C s))		
T_i (s)		
Tiempo de establecimiento (s)		
Sobreimpulso (%)		

Los resultados obtenidos mediante *Control System Designer* corresponden al análisis del modelo lineal. La aceptación del controlador requiere además evaluarlo bajo las restricciones físicas consideradas en la etapa MIL.

12.3 Comparación final en MIL

Incorpore el controlador PI–CSD al modelo MIL desarrollado en la Sección 11. Para que la comparación sea válida, ambos controladores deberán evaluarse bajo las mismas condiciones de simulación, incluyendo el modelo de la planta, el periodo de muestreo, los límites del actuador, el mecanismo de anti-windup, la condición inicial, la referencia y el tiempo de simulación.

Complete la Tabla 17.

Tabla 17: Comparación MIL de los controladores realizables.

Métrica	PI–SIMC con AW	PI–CSD con AW
Tiempo de subida (s)		
Tiempo de establecimiento (s)		
Sobreimpulso (%)		
Error estacionario (°C)		
Temperatura máxima (°C)		
$Q_{1,\min}$ **aplicado (%)**		
$Q_{1,\max}$ **aplicado (%)**		

La selección final deberá considerar el cumplimiento de las especificaciones, las restricciones físicas, la robustez y el esfuerzo de control. No se presupone que alguno de los dos métodos deba producir necesariamente el mejor resultado.

Actividad 9: diseño alternativo y comparación final

Diseñe de manera autónoma un controlador PI mediante *Control System Designer* utilizando el modelo nominal seleccionado en la Sección 9.
Documente y justifique el procedimiento seguido, los criterios de diseño utilizados y cualquier aproximación introducida durante el proceso.
Reporte el controlador obtenido y complete las Tablas 16 y 17.
Finalmente, compare ambos controladores mediante MIL y seleccione justificadamente cuál podría continuar hacia una etapa posterior de implementación.

Cierre de la práctica

Con esta comparación concluye la fase MIL de la guía. El controlador y la planta continúan siendo modelos ejecutados en Simulink. La generación y prueba de código, así como las etapas SIL, PIL o HIL mostradas en la Figura 1, requieren actividades posteriores y no deben darse por completadas a partir de estos resultados.

13 Transición de MIL a SIL

La evaluación MIL de la Sección 11 permitió estudiar distintas implementaciones del controlador PI–SIMC y seleccionar una configuración realizable antes de generar código. Durante esa etapa, tanto el controlador como la planta se ejecutaron como modelos de Simulink.

La siguiente etapa tiene un propósito diferente: verificar que una implementación en código C del controlador conserve el comportamiento del modelo previamente validado. Para ello se utilizará una prueba *Software-in-the-Loop* (SIL).

En SIL, la planta continuará ejecutándose como un modelo de Simulink. El controlador, en cambio, se ejecutará mediante el código generado y compilado para la computadora de desarrollo.

Por tanto, deben distinguirse dos operaciones:

- **generación de código:** transforma el algoritmo definido en Simulink en una implementación C; y
- **verificación SIL:** ejecuta ese código dentro del banco de pruebas y compara su respuesta con la obtenida mediante el modelo del controlador.

Generar código correctamente no demuestra por sí solo que el código reproduzca el comportamiento del modelo. Esa equivalencia deberá comprobarse mediante una comparación *back-to-back*.

13.1 Controlador seleccionado

Para continuar hacia SIL se utilizará el controlador PI–SIMC con saturación y anti-windup mediante *clamping*, correspondiente al tercer caso evaluado durante MIL.

La implementación deberá conservar:

- controlador PI discreto;
- integración mediante *Forward Euler*;
- periodo de muestreo $T_s = 1\,\mathrm{s}$;
- saturación de la salida en el intervalo 0–90 %; y
- anti-windup mediante *clamping*.

En esta etapa no se rediseñará ni se volverá a sintonizar el controlador. El objetivo es verificar mediante código una implementación que ya fue evaluada funcionalmente durante MIL.

13.2 Definición del sistema bajo prueba

No se generará código para el lazo cerrado completo. Únicamente el controlador constituirá el sistema bajo prueba (*System Under Test*, SUT).

Desde el punto de vista funcional, el controlador recibe la temperatura de referencia T_{ref} y la temperatura realimentada T_1, y genera la potencia aplicada Q_1.

En el modelo de Simulink, estas señales se denominan `T_ref`, `T1` y `Q1`, respectivamente.

La interfaz del SUT se resume en la Tabla 18.

Tabla 18: Interfaz del sistema bajo prueba.

Señal	Dirección	Tipo	Dimensión	Descripción
`T_ref`	Entrada	double	escalar	Temperatura de referencia
`T1`	Entrada	double	escalar	Temperatura realimentada
`Q1`	Salida	double	escalar	Potencia aplicada

Las tres señales son reales y el controlador opera con un único periodo de muestreo de

$$T_s = 1\,\mathrm{s}.$$

Esta interfaz deberá permanecer sin cambios durante la preparación del modelo, la generación del código y las posteriores verificaciones Normal–SIL y Normal–SIL–PIL.

13.3 Frontera del SUT

Antes de modificar el modelo MIL, identifique cuáles elementos pertenecen al algoritmo que posteriormente se convertirá en código.

Dentro del SUT deben permanecer únicamente las operaciones necesarias para implementar el controlador:

1. calcular el error
$$e[k] = T_{\mathrm{ref}}[k] - T_1[k];$$
2. calcular la contribución proporcional;
3. actualizar el estado del integrador;
4. calcular la acción de control antes de la saturación;
5. limitar la salida al intervalo 0–90 %; y
6. aplicar la lógica de anti-windup mediante *clamping*.

La planta y los elementos necesarios únicamente para ejecutar el banco de pruebas deben permanecer fuera del SUT. Esto incluye:

- el tiempo muerto de la planta;
- el modelo FOPDT discreto;
- la fuente de temperatura ambiente `T_amb`;
- la fuente de referencia `T_ref`;
- los bloques de visualización; y
- los bloques utilizados para registrar resultados.

La fuente `T_ref` permanece fuera del SUT, aunque la señal `T_ref` que esta genera constituye una entrada del controlador. De manera análoga, la fuente `T_amb` pertenece al banco de pruebas y no al algoritmo de control.

Esta separación es importante porque el código generado deberá representar exclusivamente el algoritmo del controlador. La planta continuará siendo parte del entorno de simulación durante SIL.

13.4 Estrategia de comparación Normal–SIL

La verificación SIL se realizará posteriormente mediante dos instancias del mismo modelo referenciado del controlador.

Una instancia se ejecutará con

`Simulation mode = Normal`,

mientras que la segunda se ejecutará con

`Simulation mode = Software-in-the-loop (SIL).`

Ambas recibirán la misma referencia T_{ref} y utilizarán copias idénticas del modelo de la planta, con los mismos parámetros, condiciones iniciales y configuración numérica.

La arquitectura permitirá obtener, durante una única simulación, las señales

$$Q_{1,\text{Normal}}, \qquad Q_{1,\text{SIL}},$$

y

$$T_{1,\text{Normal}}, \qquad T_{1,\text{SIL}},$$

para compararlas muestra a muestra.

Es importante notar que cada controlador requiere su propia copia del modelo de la planta, ya que las señales de control $Q_{1,\text{Normal}}$ y $Q_{1,\text{SIL}}$ se calculan de manera independiente. Para que la comparación sea válida, las dos plantas deberán ser equivalentes en estructura, parámetros, condiciones iniciales y configuración numérica.

De esta manera, cualquier diferencia entre ambas ramas podrá atribuirse a la forma en que se ejecuta el controlador y no a diferencias introducidas en la planta o en las condiciones de prueba.

En las siguientes secciones se preparará primero el modelo del SUT y se generará su código. El banco de pruebas Normal–SIL se construirá después de completar esas verificaciones.

Actividad 10: definición del sistema bajo prueba

Abra `A8_TCLab_MIL_SIMC.slx` e identifique el lazo correspondiente al controlador PI–SIMC con saturación y anti-windup.
Antes de realizar modificaciones, documente:

1. las entradas y la salida que tendrá el SUT;
2. los bloques que pertenecen al algoritmo del controlador;
3. los elementos que deberán permanecer fuera del SUT; y
4. la frontera que posteriormente se convertirá en `Controller.slx`.

No modifique todavía la estructura ni los parámetros del controlador.

14 Preparación del controlador para generación de código

Una vez definida la frontera del SUT, debe prepararse el controlador para generar código C y ejecutar posteriormente la verificación SIL.

Esta etapa comprende tres tareas relacionadas:

- separar el controlador del modelo MIL y convertirlo en un modelo referenciado;
- configurar `Controller.slx` para generación de código mediante Embedded Coder; y
- comprobar que su estructura y configuración sean compatibles con la generación de código.

Todas estas operaciones se realizarán sin modificar el algoritmo PI–SIMC validado durante MIL.

14.1 Creación del modelo de trabajo

El archivo utilizado durante MIL debe conservarse sin modificaciones. Para preparar el controlador, cree primero una copia de trabajo.

1. Abra `A8_TCLab_MIL_SIMC.slx`.
2. Ejecute una última simulación y compruebe que los tres casos utilizados durante MIL funcionan correctamente.
3. Verifique que el tercer lazo corresponde al controlador PI–SIMC con saturación y anti-windup seleccionado en la Sección 13.
4. Seleccione `File > Save As` y guarde una copia del modelo con el nombre

   `A9_Code_preparation.slx`.

5. En `A9_Code_preparation.slx`, elimine completamente:
   - el caso PI ideal; y
   - el caso PI con saturación sin anti-windup.
6. Conserve únicamente el tercer lazo, incluyendo:
   - la fuente `T_ref`;
   - el subsistema del controlador;
   - el retardo discreto de la planta;
   - el modelo FOPDT discreto;
   - la fuente `T_amb`;
   - la realimentación de `T1`; y
   - los bloques auxiliares necesarios para comprobar el funcionamiento del lazo.
7. Ejecute nuevamente el modelo y compruebe que la respuesta coincide con la obtenida para el tercer caso durante MIL.

A partir de este punto, todas las modificaciones de preparación para código se realizarán sobre `A9_Code_preparation.slx`. El modelo MIL original debe permanecer sin cambios.

14.2 Comprobación del tiempo de muestreo

El modelo utilizado para preparar el SUT debe operar como un sistema discreto *single-rate* con

$$T_s = 1\,\text{s}.$$

Configure explícitamente el tiempo de muestreo de las fuentes utilizadas por el lazo:

1. Abra el bloque `T_ref`.
2. En la ventana de parámetros del bloque, configure

   `Sample time = Ts`.

3. Repita el procedimiento para el bloque `T_amb`.
4. Actualice el modelo mediante `Ctrl+D` o ejecute nuevamente la simulación.
5. Compruebe que no aparezcan errores asociados con tiempos de muestreo.

Aunque la fuente `T_amb` no forma parte del SUT, se configura explícitamente porque pertenece al modelo discreto utilizado para comprobar el controlador.

14.3 Revisión de la frontera del controlador

Antes de separar el controlador del modelo padre, compruebe nuevamente su frontera.

1. Abra el subsistema correspondiente al controlador PI–SIMC con saturación y anti-windup.
2. Compruebe que sus únicas entradas sean:
   - `T_ref`; y
   - `T1`.
3. Compruebe que su única salida sea:
   - `Q1`.
4. Verifique que dentro del subsistema permanezcan:
   - el cálculo del error;
   - la acción proporcional;
   - el integrador discreto;
   - la saturación; y
   - la lógica de anti-windup.
5. Compruebe que el retardo, la planta FOPDT, la fuente `T_amb` y los bloques de visualización o registro permanezcan fuera del subsistema.

No continúe hasta que la frontera corresponda con la interfaz definida en la Tabla 18.

14.4 Conversión a modelo referenciado

El controlador se convertirá en un modelo referenciado denominado `Controller.slx`.

Primero establezca una frontera de ejecución explícita para el subsistema:

1. Regrese al nivel superior de `A9_Code_preparation.slx`.
2. Seleccione el subsistema del controlador.
3. En la pestaña `Subsystem Block`, seleccione `Make Atomic`.
4. Ejecute nuevamente el modelo y compruebe que este cambio no haya modificado el comportamiento del lazo cerrado.

Posteriormente convierta el subsistema:

1. Seleccione nuevamente el subsistema del controlador.
2. En la pestaña `Subsystem Block`, seleccione

   `Convert > Referenced Model`.

3. En el *Model Reference Conversion Advisor*, utilice

   `Controller`

   como nombre del nuevo modelo.
4. Ejecute la conversión y revise los mensajes presentados por el *Advisor*.
5. Si aparece algún error, corrija su causa antes de continuar.

Al finalizar, compruebe que:

- se creó `Controller.slx`;
- el subsistema original fue sustituido por un bloque *Model*;
- el bloque referencia `Controller.slx`; y
- la planta permanece en `A9_Code_preparation.slx`.

Renombre la instancia del bloque *Model* como

`Controller_SUT`.

14.5 Comprobación de `Controller.slx`

Abra `Controller.slx`.

Compruebe que su nivel superior contenga exclusivamente:

- la entrada `T_ref`;
- la entrada `T1`;
- el algoritmo PI discreto;
- la saturación;
- la lógica de anti-windup; y
- la salida `Q1`.

No deben encontrarse dentro de este modelo:

- el retardo de la planta;
- el modelo FOPDT;
- la fuente `T_amb`; ni
- elementos utilizados únicamente para visualizar o registrar resultados.

La conversión a modelo referenciado no debe modificar el algoritmo validado durante MIL.

14.6 Datos propios del controlador

Los datos necesarios para ejecutar el algoritmo deben mantenerse dentro de `Controller.slx` y separados de los parámetros correspondientes a la planta.

Para configurar estos datos:

1. Con `Controller.slx` abierto, seleccione

   `Modeling > Model Explorer`.

2. En el panel *Model Hierarchy*, seleccione

   `Controller > Model Workspace`.

3. Agregue los siguientes datos utilizando los valores empleados durante MIL:
   - `Kp_SIMC`;
   - `Ki_SIMC`;
   - `Q_min`;
   - `Q_max`; y
   - `Ts`.
4. Defina `Kp_SIMC` y `Ki_SIMC` como objetos `Simulink.Parameter`.
5. Defina `Q_min`, `Q_max` y `Ts` como variables de MATLAB, con

$$Q_{\min} = 0, \qquad Q_{\max} = 90, \qquad T_s = 1\,\text{s}.$$

Las ganancias `Kp_SIMC` y `Ki_SIMC` se conservarán como parámetros calibrables. En cambio, `Q_min`, `Q_max` y `Ts` se consideran datos estructurales de esta implementación.

Compruebe que los bloques internos de `Controller.slx` utilicen estos nombres.

Los parámetros de la planta K, τ, θ y la temperatura ambiente T_{amb} no deben formar parte del *Model Workspace* de `Controller.slx`.

14.7 Separación de la inicialización

El archivo `A8_init_TCLab_SIMC.m` inicializa parámetros de la planta y condiciones necesarias para ejecutar el banco de pruebas. Por esta razón, debe permanecer asociado con el modelo padre y no con el SUT.

Compruebe la configuración de `Controller.slx`:

1. Con `Controller.slx` abierto, seleccione

   `Modeling > Model Properties`.

2. En la ventana *Model Properties*, seleccione la pestaña

   `Callbacks`.

3. Seleccione el callback `InitFcn`.

4. Si aparece una llamada a

   `A8_init_TCLab_SIMC.m`,

   elimínela.

5. Compruebe que `InitFcn` quede vacío.

14.8 Configuración para Embedded Coder

A continuación se configurará `Controller.slx` para generar código C.

14.8.1 Parámetros calibrables

Configure las ganancias del controlador para que permanezcan explícitamente representadas en el código generado:

1. Abra `Modeling > Model Explorer`.

2. Seleccione

   `Controller > Model Workspace`.

3. Localice `Kp_SIMC` y `Ki_SIMC`.
4. Para cada parámetro, configure

`Storage Class = ExportedGlobal`.

Compruebe que ambos continúen siendo objetos `Simulink.Parameter`.

La clase `ExportedGlobal` hace que las ganancias tengan una representación explícita en el código generado y permite conservarlas como parámetros calibrables.

14.8.2 Optimización de parámetros

Abra

`Modeling > Model Settings`

y seleccione

`Code Generation > Optimization`.

Configure

`Default parameter behavior = Inlined`.

Por tanto:

- `Kp_SIMC` y `Ki_SIMC` permanecen como `ExportedGlobal`;
- `Q_min`, `Q_max` y `Ts` permanecen como datos estructurales; y
- los demás parámetros utilizan el comportamiento `Inlined`.

14.8.3 Solver

En la ventana `Model Settings`, seleccione

`Solver`

y configure:

```
Start time       0.0
Stop time        inf
Type             Fixed-step
Solver           discrete (no continuous states)
Fixed-step size  Ts
```

El modelo del controlador es completamente discreto y opera con

$$T_s = 1\,\text{s}.$$

El tiempo final utilizado durante las simulaciones del TCLab pertenece al banco de pruebas y no al algoritmo del controlador. Por esta razón, `Controller.slx` no debe depender de la variable `t_final` definida por el script de inicialización del modelo padre.

La configuración

```
Stop time = inf
```

permite que `Controller.slx` pueda generar su código de manera independiente, sin necesidad de ejecutar previamente el modelo padre ni el script `A8_init_TCLab_SIMC.m`.

El tiempo final de la prueba, por ejemplo $t_{\text{final}} = 1800\,\text{s}$, continuará siendo definido en el modelo padre utilizado para las pruebas MIL, SIL o PIL.

14.8.4 Target y lenguaje

En

```
Model Settings > Code Generation
```

configure

```
System target file = ert.tlc.
```

El archivo `ert.tlc` selecciona el flujo de generación de Embedded Coder utilizado para transformar la semántica del modelo de Simulink en una implementación C/C++ orientada a sistemas embebidos. No representa todavía un hardware específico.

En la misma sección configure:

```
Language           C
Language standard  C99 (ISO)
```

14.8.5 Hardware utilizado durante SIL

Durante SIL, el código generado se ejecutará en la computadora de desarrollo.

Abra

```
Model Settings > Hardware Implementation
```

y configure:

```
Hardware board   None
Device vendor    Intel
Device type      x86-64 (Windows64)
```

Esta configuración corresponde al procesador host utilizado para la prueba SIL. No configure todavía ningún hardware Arduino.

14.8.6 Interfaz del modelo referenciado

El banco de pruebas Normal–SIL utilizará posteriormente más de una instancia de `Controller.slx`. Por esta razón, compruebe primero que el modelo admita múltiples instancias.

Abra

`Model Settings > Model Referencing`

y configure

`Total number of instances allowed per top model = Multiple.`

Posteriormente seleccione

`Code Generation > Interface`

y configure

`Code interface packaging = Reusable function.`

La interfaz funcional del componente continúa siendo

$$T_{\text{ref}},\, T_1 \quad \longrightarrow \quad \texttt{Controller} \quad \longrightarrow \quad Q_1.$$

La Tabla 19 resume la configuración obtenida.

14.9 Verificación para generación de código

Antes de generar el código, utilice las herramientas de análisis de Simulink Coder y Embedded Coder para comprobar la configuración de `Controller.slx`.

14.9.1 Code Generation Advisor

Con `Controller.slx` abierto:

1. Seleccione

Tabla 19: Configuración de `Controller.slx` para generación de código.

Configuración	Valor
System target file	`ert.tlc`
Language	`C`
Language standard	`C99 (ISO)`
Solver type	`Fixed-step`
Solver	`discrete (no continuous states)`
Fixed-step size	`Ts`
Hardware board	`None`
Device vendor	`Intel`
Device type	`x86-64 (Windows64)`
Code interface packaging	`Reusable function`
Default parameter behavior	`Inlined`
`Kp_SIMC`	`ExportedGlobal`
`Ki_SIMC`	`ExportedGlobal`

`Modeling > Model Settings > Code Generation > Code generation objectives`.

2. Seleccione `Set Objectives`.
3. En `Available objectives`, agregue, en este orden:
   (a) `Execution efficiency`;
   (b) `Traceability`.
4. Compruebe que ambos aparezcan en `Prioritized objectives`.
5. Seleccione `OK`.
6. En la sección `Code Generation`, seleccione `Check Model`.
7. Ejecute las comprobaciones del *Code Generation Advisor*.

Revise individualmente los resultados. Ninguna comprobación debe presentar `Fail`. Los resultados `Warning` deben analizarse antes de decidir si requieren modificar el modelo. En esta práctica, el *Code Generation Advisor* puede reportar advertencias aun cuando el modelo sea compatible con generación de código. Durante la preparación de `Controller.slx` se identificaron las advertencias resumidas en la Tabla 20.

Por tanto, la presencia de estas advertencias no impide continuar con la generación de código. No utilice `Modify Parameters`, `Fix All` ni otras opciones de modificación automática únicamente para obtener resultados `Pass`.

El criterio para continuar será que no existan resultados `Fail` y que cada `Warning` restante haya sido revisado y pueda justificarse.

14.9.2 Model Advisor

Realice además las comprobaciones seleccionadas mediante el *Model Advisor*.

Tabla 20: Advertencias observadas en el *Code Generation Advisor*.

Comprobación	Interpretación	Decisión
`Check model configuration settings against code generation objectives`	Algunas opciones de configuración no coinciden con los valores recomendados para maximizar `Execution efficiency`. Las recomendaciones corresponden principalmente a optimizaciones del código generado y no a errores de compatibilidad.	Conservar
`Identify questionable software environment specifications`	Se mantiene soporte para valores no finitos y `Application lifespan = auto`. Embedded Coder recomienda modificar estas opciones para código de producción, pero no son necesarias para la verificación SIL.	Conservar
`Identify blocks that generate expensive fixed-point and saturation code`	Embedded Coder identifica código de protección frente a excepciones en operaciones enteras. Eliminar dicha protección constituye una optimización y requiere demostrar previamente que esas excepciones no pueden producirse.	Conservar

1. Con `Controller.slx` abierto, seleccione

   `Modeling > Model Advisor`.

2. En el panel *Check Selector*, expanda

   `By Product`.

3. Seleccione las comprobaciones indicadas en la Tabla 21.

4. Ejecute las comprobaciones seleccionadas mediante

   `Run`.

Tabla 21: Comprobaciones seleccionadas en *Model Advisor*.

Producto	Comprobación
Simulink Coder	`Check solver for code generation`
Simulink Coder	`Check for blocks not supported by code generation`
Embedded Coder	`Check for blocks not recommended for C/C++ production code deployment`
Embedded Coder	`Check the hardware implementation`
Embedded Coder	`Identify questionable software environment specifications`
Embedded Coder	`Identify questionable code instrumentation (data I/O)`

Revise el resultado de cada comprobación antes de continuar. Un resultado `Warning` no implica necesariamente que el modelo sea incompatible con la generación de código. Algunas comprobaciones corresponden a recomendaciones orientadas principalmente a optimización de código de producción.

En la configuración utilizada para esta práctica, la comprobación

`Identify questionable software environment specifications`

puede producir un resultado `Warning`. En particular, el *Model Advisor* puede indicar que:

- se encuentra habilitado el soporte para valores no finitos (`NaN` e `Inf`); y
- el parámetro `Application lifespan` conserva el valor predeterminado `auto`.

Ambas condiciones corresponden a recomendaciones para optimizar código de producción y no representan, por sí mismas, una incompatibilidad con la verificación SIL. Para esta práctica se conservarán sus valores actuales.

Con la configuración establecida en esta guía, el resultado esperado de las comprobaciones seleccionadas es:

Resultado	Cantidad
`Passed`	5
`Warning`	1
`Fail`	0

Por tanto, no utilice `Fix`, `Justify` ni otras opciones de modificación automática únicamente con el propósito de eliminar la advertencia descrita anteriormente.

Si aparece una advertencia diferente o algún resultado `Fail`, analice su causa antes de continuar. Para cada advertencia adicional:

1. identifique la condición que la produce;
2. revise la recomendación presentada por el *Model Advisor*;
3. determine si corresponde a un problema de compatibilidad o a una recomendación de optimización; y
4. documente la decisión adoptada.

14.10 Comprobación funcional

Después de completar la configuración y revisar los *Advisors*, compruebe que la preparación para código no haya alterado el funcionamiento del controlador.

1. Guarde los cambios realizados en `Controller.slx`.
2. Regrese a `A9_Code_preparation.slx`.
3. Compruebe que `Controller_SUT` permanezca en

   `Simulation mode = Normal`.

4. Ejecute el modelo.

5. Compruebe que:
   - la simulación finaliza sin errores;
   - las entradas del SUT son únicamente `T_ref` y `T1`;
   - la salida es únicamente `Q1`;
   - $0 \leq Q_1 \leq 90\,\%$;
   - la respuesta conserva el comportamiento validado durante MIL; y
   - la planta permanece fuera de `Controller.slx`.

La preparación se considerará satisfactoria cuando los *Advisors* no presenten resultados `Fail`, las advertencias restantes hayan sido revisadas y el modelo continúe reproduciendo correctamente el comportamiento validado durante MIL.

Hasta este punto todavía no se ha generado el código C ni se ha realizado una prueba SIL.

Actividad 11: preparación del controlador para código

Documente la preparación de `Controller.slx` incluyendo:

1. la interfaz final del SUT;
2. los datos almacenados en su *Model Workspace*;
3. la configuración de generación de código resumida en la Tabla 19;
4. los resultados obtenidos mediante *Code Generation Advisor* y *Model Advisor*;
5. Compruebe si aparecen las advertencias de la Tabla 20. Si aparece alguna advertencia adicional, analícela antes de continuar; y
6. la comprobación de que `A9_Code_preparation.slx` continúa reproduciendo el comportamiento validado durante MIL.

No continúe hacia generación de código mientras exista algún resultado `Fail` o mientras la conversión a `Controller.slx` haya modificado el comportamiento del lazo cerrado.

15 Generación e inspección del código C

Una vez completada la preparación de `Controller.slx`, puede generarse el código C correspondiente al SUT.

El propósito de esta etapa no es únicamente comprobar que Embedded Coder pueda completar la construcción. También debe verificarse que el código generado represente exclusivamente el controlador y conserve las características funcionales definidas durante MIL.

La equivalencia numérica entre el modelo y el código todavía no se evaluará en esta sección. Esa comprobación se realizará posteriormente mediante SIL.

15.1 Generación del código

Antes de generar el código, compruebe que:

- `Controller.slx` se encuentre guardado;
- el *Code Generation Advisor* no presente resultados `Fail`;
- el *Model Advisor* no presente resultados `Fail`; y
- cualquier `Warning` restante haya sido revisado según lo indicado en la Sección 14.

Para generar el código:

1. Abra la ventana de comandos de MATLAB.
2. Ejecute

   ```
   slbuild("Controller","ModelReferenceCoderTarget")
   ```

3. Espere a que finalice el proceso de construcción.
4. Compruebe que MATLAB no reporte errores de generación ni de compilación.

El argumento `ModelReferenceCoderTarget` solicita la construcción del target de código correspondiente al modelo referenciado `Controller.slx`.

Una construcción satisfactoria confirma que Embedded Coder pudo transformar el modelo en código C compatible con la configuración establecida. Sin embargo, esto todavía no demuestra que el código produzca los mismos resultados que el modelo ejecutado en modo `Normal`.

15.2 Localización de los archivos generados

Durante la construcción, Embedded Coder almacena los archivos correspondientes al modelo referenciado dentro de la carpeta `slprj` asociada al proyecto.

En la ventana de comandos de MATLAB, ejecute:

```
codeFolder = fullfile(pwd,"slprj","ert","Controller");
winopen(codeFolder)
```

El primer comando construye la ruta de la carpeta donde se generó el código para `Controller.slx`; el segundo abre esa ubicación en el explorador de archivos de Windows.

Compruebe que la carpeta abierta corresponda a

`slprj > ert > Controller`.

La misma ubicación puede identificarse en la salida producida por `slbuild`, mediante el mensaje

`Generating code into build folder: ...\slprj\ert\Controller`.

Dentro de la carpeta

`slprj > ert > Controller`

deben encontrarse, entre otros, los archivos principales asociados directamente con el controlador:

- `Controller.c`;
- `Controller.h`;
- `Controller_private.h`; y
- `Controller_types.h`.

Embedded Coder puede almacenar algunos archivos auxiliares o compartidos, como `rtwtypes.h`, en otras carpetas dentro de la estructura `slprj`. Para localizar este archivo, ejecute:

```
dir(fullfile(pwd,"slprj","**","rtwtypes.h"))
```

La ubicación exacta de estos archivos auxiliares puede depender de la versión de MATLAB y de la configuración utilizada durante la generación de código.

El archivo `Controller.c` contiene la implementación principal del algoritmo generado, mientras que los archivos de cabecera contienen declaraciones, tipos de datos, estructuras e información necesaria para compilar y utilizar el componente.

Embedded Coder genera además otros archivos auxiliares relacionados con la compilación, las dependencias y la construcción del modelo. No es necesario analizarlos exhaustivamente en esta práctica.

Antes de continuar, compruebe que los archivos indicados anteriormente se encuentren disponibles. La inspección de las siguientes subsecciones se concentrará en verificar que el código generado conserve:

- la interfaz definida para el SUT;
- el estado interno asociado con el integrador;
- los parámetros calibrables;
- los límites de saturación;
- la lógica de anti-windup; y
- la separación entre el controlador y el modelo de la planta.

15.3 Inspección de la interfaz

La primera comprobación consiste en verificar que la interfaz del código generado coincida con la frontera definida para el SUT.

Abra el archivo

`Controller.h`.

Localice la declaración de la función principal del modelo referenciado:

```
extern void Controller(const real_T *rtu_T_ref,
                       const real_T *rtu_T1,
                       real_T *rty_Q1,
                       DW_Controller_f_T *localDW);
```

Los prefijos utilizados por Embedded Coder permiten reconocer el papel de las variables dentro de la interfaz. En este caso:

- `rtu_T_ref` corresponde a la entrada `T_ref`;
- `rtu_T1` corresponde a la entrada `T1`; y
- `rty_Q1` corresponde a la salida `Q1`.

Por tanto, la interfaz funcional del código conserva la estructura definida para el controlador:

$$T_{\text{ref}}, T_1 \quad \longrightarrow \quad \texttt{Controller} \quad \longrightarrow \quad Q_1.$$

Las entradas y la salida se declaran utilizando el tipo `real_T`. Para comprobar su representación numérica, abra `rtwtypes.h` y localice la definición

```
typedef double real_T;
```

Por tanto, en esta configuración las señales del SUT se representan mediante datos de doble precisión, de acuerdo con el tipo `double` utilizado en el modelo de Simulink.

Observe además que las entradas y la salida se transfieren mediante punteros. Por ejemplo,

```
const real_T *rtu_T_ref
```

representa un puntero hacia el valor de la entrada `T_ref`. Por esta razón, dentro de `Controller.c` el valor de la señal se obtiene mediante el operador de indirección `*`:

```
rtb_IntegralGain = *rtu_T_ref - *rtu_T1;
```

El modificador `const` indica que la función del controlador utiliza estas entradas para realizar sus cálculos, pero no modifica sus valores.

Compruebe finalmente que la interfaz no incluya señales asociadas con:

- el modelo FOPDT;
- el retardo de la planta;
- la temperatura ambiente T_{amb}; ni
- los elementos utilizados exclusivamente por el banco de pruebas.

Esta comprobación confirma que la frontera definida para el SUT en la Sección 13 se conserva también en la interfaz del código generado.

15.4 Estado interno del controlador

El controlador PI contiene un integrador discreto. Por esta razón, su salida no depende únicamente de las entradas disponibles en la muestra actual, sino también de información almacenada durante las muestras anteriores.

Abra el archivo

`Controller.h`

y localice la estructura utilizada para almacenar los estados internos del modelo:

```
typedef struct {
  real_T Integrator_DSTATE;
} DW_Controller_f_T;
```

En esta implementación, Embedded Coder representa el estado del integrador mediante la variable

`Integrator_DSTATE`.

La estructura `DW_Controller_f_T` agrupa los datos que deben conservarse entre ejecuciones sucesivas del controlador. En este caso contiene un único estado, correspondiente al integrador discreto.

Observe además que la función principal del controlador recibe como argumento

`DW_Controller_f_T *localDW`.

El identificador `localDW` es un puntero hacia la estructura que contiene los estados internos de una instancia del controlador. Por esta razón, dentro de `Controller.c` el estado del integrador se accede mediante

`localDW->Integrator_DSTATE`.

El operador `->` de C permite acceder a un miembro de una estructura a través de un puntero.

A continuación, localice en `Controller.c` la expresión

```
rtb_Sum = Kp_SIMC * rtb_IntegralGain
        + localDW->Integrator_DSTATE;
```

Antes de esta operación, la variable temporal `rtb_IntegralGain` contiene el error de control,

```
rtb_IntegralGain = *rtu_T_ref - *rtu_T1;
```

por lo que la primera expresión implementa la suma de la contribución proporcional y el valor previamente acumulado por el integrador.

De manera conceptual, antes de aplicar la saturación puede escribirse

$$u[k] = K_p e[k] + x_I[k],$$

donde

$$e[k] = T_{\text{ref}}[k] - T_1[k]$$

y $x_I[k]$ representa el estado almacenado del integrador.

Más adelante en `Controller.c`, el error se multiplica por la ganancia integral:

```
rtb_IntegralGain *= Ki_SIMC;
```

Después de aplicar la lógica de anti-windup, que se estudiará en una subsección posterior, el estado se actualiza mediante

```
localDW->Integrator_DSTATE += rtb_IntegralGain;
```

Por tanto, el valor almacenado después de una muestra será utilizado durante la ejecución siguiente. Esto confirma que el controlador posee memoria y que su comportamiento no puede describirse como una operación puramente algebraica entre las entradas actuales y la salida.

Finalmente, localice la función

`Controller_initialize`.

En ella se encuentra la instrucción

```
(void) memset((void *)localDW, 0,
              sizeof(DW_Controller_f_T));
```

Esta operación inicializa en cero la estructura que contiene los estados internos. En particular, para el integrador utilizado en esta práctica,

$$x_I[0] = 0.$$

La inicialización del estado generado es, por tanto, consistente con la condición inicial especificada para el integrador en el modelo de Simulink.

15.5 Parámetros calibrables

Durante la preparación para generación de código se definieron las ganancias

`Kp_SIMC` y `Ki_SIMC`

como parámetros calibrables mediante

`Storage Class = ExportedGlobal`.

La inspección del código generado permite comprobar cómo se representa esta decisión en C.

Abra el archivo

`Controller.h`

y localice la sección identificada como

`Exported Global Parameters`.

En ella deben aparecer declaraciones equivalentes a

```
extern real_T Ki_SIMC;
extern real_T Kp_SIMC;
```

La palabra clave `extern` indica que estos símbolos son variables globales cuya definición y almacenamiento se encuentran en otro archivo fuente. La declaración permite que otros archivos que incluyan `Controller.h` conozcan su nombre y tipo de dato.

Observe además que ambos parámetros utilizan el tipo

`real_T`,

que, para la configuración utilizada en esta práctica, corresponde a `double`.

Abra ahora el archivo

`Controller.c`

y localice las definiciones de los parámetros:

```
real_T Ki_SIMC = 0.015058760578832;
real_T Kp_SIMC = 2.93829548166286;
```

A diferencia de las declaraciones precedidas por `extern`, estas instrucciones reservan el almacenamiento correspondiente y establecen los valores iniciales de las ganancias.

Por tanto, la relación entre ambos archivos puede resumirse como

`Controller.h`	:	declara los símbolos,
`Controller.c`	:	define los símbolos y sus valores iniciales.

A continuación, compruebe que el algoritmo utiliza directamente estos parámetros.

La contribución proporcional aparece en una expresión equivalente a

```
rtb_Sum = Kp_SIMC * rtb_IntegralGain
        + localDW->Integrator_DSTATE;
```

donde `rtb_IntegralGain` contiene, en ese punto del algoritmo, el error de control. Por tanto, `Kp_SIMC` multiplica la contribución proporcional.

La ganancia integral aparece posteriormente en

```
rtb_IntegralGain *= Ki_SIMC;
```

y determina el incremento que puede incorporarse al estado del integrador.

La presencia explícita de los símbolos `Kp_SIMC` y `Ki_SIMC` confirma que Embedded Coder no sustituyó estas ganancias únicamente por constantes numéricas dentro de las operaciones del controlador.

Esto contrasta con un parámetro que hubiera sido incorporado directamente al código, por ejemplo:

```
rtb_Sum = 2.93829548166286 * rtb_IntegralGain
        + localDW->Integrator_DSTATE;
```

En ese caso, el valor estaría integrado directamente en la expresión y ya no existiría un símbolo independiente denominado `Kp_SIMC`.

Por tanto, la configuración `ExportedGlobal` conserva las ganancias como variables explícitas del código generado, lo cual permite modificar sus valores sin cambiar la estructura del algoritmo ni sustituir manualmente constantes dentro del código fuente.

15.6 Saturación de la salida

La señal calculada por el controlador no puede aplicarse directamente al actuador si se encuentra fuera de sus límites físicos. En esta práctica, la salida del controlador se restringió durante MIL al intervalo

$$0 \leq Q_1 \leq 90\,\%.$$

Los límites fueron definidos en el modelo mediante los datos estructurales

$$Q_{\min} = 0, \qquad Q_{\max} = 90.$$

Abra el archivo

`Controller.c`

y localice la lógica asociada con los bloques `Dead Zone` y `Saturation`. El código generado contiene una estructura equivalente a:

```
if (rtb_Sum > 90.0) {
  rtb_DeadZone = rtb_Sum - 90.0;
  *rty_Q1 = 90.0;
} else {
  if (rtb_Sum >= 0.0) {
    rtb_DeadZone = 0.0;
  } else {
    rtb_DeadZone = rtb_Sum;
  }

  if (rtb_Sum < 0.0) {
    *rty_Q1 = 0.0;
  } else {
    *rty_Q1 = rtb_Sum;
  }
}
```

La variable temporal `rtb_Sum` contiene la acción PI calculada antes de aplicar los límites del actuador. Por tanto, puede interpretarse como la señal de control no saturada,

$$u[k] = K_p e[k] + x_I[k].$$

La lógica generada considera tres situaciones.

Si

$$u[k] > 90,$$

la salida se fija en el límite superior:

$$Q_1[k] = 90.$$

Esto se implementa mediante

```
*rty_Q1 = 90.0;
```

Si

$$u[k] < 0,$$

la salida se fija en el límite inferior:

$$Q_1[k] = 0,$$

mediante

```
*rty_Q1 = 0.0;
```

Finalmente, si

$$0 \leq u[k] \leq 90,$$

la señal calculada por el controlador se transmite directamente a la salida:

```
*rty_Q1 = rtb_Sum;
```

Por tanto, el comportamiento implementado puede resumirse como

$$Q_1[k] = \begin{cases} 0, & u[k] < 0, \\ u[k], & 0 \leq u[k] \leq 90, \\ 90, & u[k] > 90. \end{cases}$$

Observe que el código utiliza directamente los valores `0.0` y `90.0`. Esto es consistente con la decisión de mantener `Q_min` y `Q_max` como datos estructurales y no como parámetros calibrables. Por tanto, no se requiere que estos identificadores aparezcan como variables globales en el código generado.

Además de calcular la salida saturada, el código genera la variable temporal

`rtb_DeadZone`.

Su valor indica si la acción PI calculada se encuentra fuera del intervalocpermitido:

$$\texttt{rtb_DeadZone} = \begin{cases} u[k], & u[k] < 0, \\ 0, & 0 \leq u[k] \leq 90, \\ u[k] - 90, & u[k] > 90. \end{cases}$$

Por tanto,

$$\texttt{rtb_DeadZone} = 0$$

cuando la acción de control se encuentra dentro del intervalo realizable, mientras que un valor distinto de cero indica que se ha alcanzado alguno de los límites.

Esta información será utilizada posteriormente por la lógica de anti-windup para determinar si debe permitirse o detenerse temporalmente la actualización del integrador.

15.7 Lógica de anti-windup

La saturación limita la señal entregada al actuador, pero por sí sola no evita que el integrador continúe acumulándose mientras la salida permanece en uno de sus límites. Para evitar este fenómeno, el controlador utiliza una estrategia de anti-windup mediante *clamping*.

En la subsección anterior se identificó la variable temporal

$$\texttt{rtb_DeadZone},$$

cuyo signo permite determinar si la acción PI calculada antes de la saturación se encuentra fuera del intervalo permitido:

$$\texttt{rtb_DeadZone} \begin{cases} < 0, & u[k] < 0, \\ = 0, & 0 \le u[k] \le 90, \\ > 0, & u[k] > 90. \end{cases}$$

Después de calcular la salida saturada, el código determina el incremento que se incorporaría al estado del integrador:

```
rtb_IntegralGain *= Ki_SIMC;
```

Debido a que en esta práctica $T_s = 1\,\mathrm{s}$, esta cantidad corresponde, para la configuración generada, a

$$\Delta x_I[k] = K_i e[k].$$

A continuación, Embedded Coder genera dos variables auxiliares de tipo `int8_T` para representar el signo de la saturación y el signo del incremento integral:

```
if (rtb_DeadZone > 0.0) {
  tmp = 1;
} else {
  tmp = -1;
}

if (rtb_IntegralGain > 0.0) {
  tmp_0 = 1;
} else {
  tmp_0 = -1;
}
```

La variable `tmp` representa el sentido en el que la acción PI excedió los límites, mientras que `tmp_0` representa el sentido en el que el integrador intentaría modificar la acción de control.

La decisión principal del *clamping* aparece en:

```
if ((rtb_DeadZone != 0.0) && (tmp == tmp_0)) {
  rtb_IntegralGain = 0.0;
}
```

Esta condición exige simultáneamente que:

1. la acción PI se encuentre fuera del intervalo permitido; y
2. el incremento integral tenga el mismo sentido que la saturación.

Por ejemplo, si

$$u[k] > 90$$

y el error produce

$$\Delta x_I[k] > 0,$$

el integrador intentaría aumentar todavía más la acción de control. En este caso,

$$\texttt{rtb_DeadZone} > 0, \qquad \texttt{tmp} = \texttt{tmp_0} = 1,$$

por lo que el código fuerza

$$\texttt{rtb_IntegralGain = 0.0}.$$

De manera análoga, si

$$u[k] < 0$$

y el incremento integral es negativo, el integrador intentaría reducir aún más la salida. La actualización también se bloquea.

En cambio, si el controlador se encuentra saturado en el límite superior pero el error genera un incremento integral negativo, la condición `tmp == tmp_0` deja de cumplirse. La integración se permite porque su efecto favorece el regreso de la acción de control hacia el intervalo realizable.

El mismo principio se aplica cuando la salida está saturada en el límite inferior y el incremento integral es positivo.

Por tanto, la lógica puede resumirse como

$$\Delta x_{I,\mathrm{AW}}[k] = \begin{cases} 0, & \text{si existe saturación y el incremento integral} \\ & \text{tiende a aumentar dicha saturación,} \\ K_i e[k], & \text{en cualquier otro caso.} \end{cases}$$

Finalmente, el estado se actualiza mediante

```
localDW->Integrator_DSTATE += rtb_IntegralGain;
```

de modo que

$$x_I[k+1] = x_I[k] + \Delta x_{I,\mathrm{AW}}[k].$$

Así, el integrador se detiene solamente cuando su actualización agravaría la saturación. Cuando el error favorece el retorno de la señal de control al intervalo 0–90 %, la integración se reanuda.

Esta implementación confirma que el código generado conserva la estrategia de anti-windup mediante *clamping* utilizada previamente durante MIL.

15.8 Separación entre controlador y planta

La última comprobación estructural consiste en verificar que el código generado corresponda exclusivamente al SUT definido para esta práctica y no incorpore la dinámica de la planta utilizada en el banco de pruebas.

En las etapas anteriores, el modelo FOPDT de la planta se definió mediante los parámetros

$$K, \qquad \tau, \qquad \theta,$$

junto con la temperatura ambiente T_{amb}. Estos datos pertenecen al modelo de la planta y no forman parte del algoritmo implementado en `Controller.slx`.

La separación puede comprobarse inicialmente mediante la interfaz declarada en

`Controller.h`.

La función principal del controlador recibe únicamente las entradas

`rtu_T_ref` y `rtu_T1`,

y produce la salida

`rty_Q1`.

Por tanto, el código generado no requiere como entradas independientes los parámetros de la planta, el retardo ni la temperatura ambiente.

Revise además la estructura utilizada para almacenar los estados internos del controlador. En la implementación generada aparece únicamente el estado correspondiente al integrador:

```
typedef struct {
  real_T Integrator_DSTATE;
} DW_Controller_f_T;
```

No aparecen estados asociados con la dinámica FOPDT ni con el retardo de la planta. Esto confirma que la memoria interna del código corresponde al controlador y no al proceso térmico utilizado durante la simulación.

Finalmente, examine `Controller.c`. El algoritmo generado contiene las operaciones necesarias para:

- calcular el error $T_{\text{ref}} - T_1$;
- calcular las contribuciones proporcional e integral;
- aplicar los límites de saturación;
- ejecutar la lógica de anti-windup mediante *clamping*; y
- actualizar el estado interno del integrador.

No deben encontrarse en este archivo ecuaciones, estados o retardos destinados a calcular la evolución de T_1 a partir de Q_1. Esa relación corresponde al modelo de la planta y permanece fuera del SUT.

Esta comprobación muestra que la arquitectura utilizada para generación de código conserva la separación

$$\boxed{\text{controlador} \quad \neq \quad \text{planta}}$$

establecida previamente durante la transición de MIL a SIL.

El código contenido en `Controller.c` implementa, por tanto, exclusivamente la transformación

$$T_{\text{ref}}, T_1 \quad \longrightarrow \quad Q_1,$$

mientras que la dinámica que determina la evolución de T_1 a partir de Q_1 continuará siendo responsabilidad del banco de pruebas.

15.9 Criterio para continuar

Antes de construir el banco de pruebas Normal–SIL, verifique que la generación y la inspección del código hayan permitido comprobar los siguientes aspectos:

- `Controller.slx` puede generar su código de manera independiente, sin necesidad de ejecutar previamente el modelo padre ni el script `A8_init_TCLab_SIMC.m`;
- el proceso de construcción finaliza sin errores;
- se generan los archivos principales correspondientes al controlador, incluyendo `Controller.c`, `Controller.h`, `Controller_private.h` y `Controller_types.h`, junto con los archivos compartidos requeridos para la compilación;
- la interfaz del código conserva las entradas `T_ref` y `T1`, y la salida `Q1`;
- las señales del SUT se representan mediante el tipo `real_T`, definido como `double` para la configuración utilizada en esta etapa;
- el estado interno del integrador se conserva mediante `Integrator_DSTATE`;

- `Kp_SIMC` y `Ki_SIMC` permanecen representados explícitamente como parámetros `ExportedGlobal`;
- la acción de control se restringe al intervalo

$$0 \leq Q_1 \leq 90\,\%;$$

- la lógica de anti-windup detiene la actualización del integrador cuando esta agravaría la saturación y permite nuevamente la integración cuando favorece el regreso al intervalo realizable; y
- el código generado contiene exclusivamente la lógica del controlador y no incorpora la dinámica FOPDT, el retardo ni otros elementos propios de la planta o del banco de pruebas.

Estas comprobaciones permiten verificar que Embedded Coder produjo una implementación C coherente con la estructura y las decisiones de diseño definidas para el SUT.

Sin embargo, la inspección del código no constituye todavía una prueba SIL y tampoco demuestra por sí sola que la ejecución del código generado produzca resultados numéricamente equivalentes a los obtenidos mediante el modelo de Simulink.

La siguiente etapa utilizará una comparación *back-to-back* entre una ejecución `Normal` y una ejecución SIL del mismo modelo referenciado para realizar esta verificación.

Actividad 12: generación e inspección del código C

Genere el código correspondiente a `Controller.slx` y presente evidencia de que la construcción finalizó satisfactoriamente.
A partir de los archivos generados, documente mediante fragmentos breves de código:

1. la interfaz de entrada y salida del SUT;
2. el estado interno utilizado para almacenar la memoria del integrador;
3. la declaración, definición y utilización de `Kp_SIMC` y `Ki_SIMC`;
4. la implementación de los límites 0–90 % de la señal de control;
5. la condición utilizada por la lógica de anti-windup para bloquear o permitir la actualización del integrador; y
6. la evidencia de que la dinámica de la planta permanece fuera del código generado para el controlador.

Para cada fragmento, indique el archivo en el que fue localizado y explique brevemente qué comportamiento del modelo permite comprobar.
Concluya indicando si la inspección realizada permite afirmar que el código generado conserva la estructura funcional del controlador diseñado durante MIL.
No interprete esta comprobación como una validación SIL. La equivalencia numérica entre el modelo y el código será evaluada posteriormente mediante una prueba *back-to-back* Normal–SIL.

16 Banco de pruebas Normal–SIL

Una vez generado e inspeccionado el código C del controlador, se construirá el banco de pruebas utilizado para verificar su comportamiento mediante *Software-in-the-Loop* (SIL).

La estrategia empleará simultáneamente dos instancias del mismo modelo referenciado `Controller.slx`:

- una instancia ejecutada en modo `Normal`; y
- una instancia ejecutada en modo `Software-in-the-loop (SIL)`.

Ambas instancias recibirán las mismas condiciones de prueba. Sin embargo, cada controlador se conectará a su propia copia del modelo de la planta, ya que las señales de control calculadas por las dos ejecuciones deben evolucionar de manera independiente.

De esta forma, los resultados Normal y SIL se obtendrán durante una única simulación, lo que facilita posteriormente una comparación *back-to-back* bajo condiciones equivalentes.

16.1 Arquitectura del banco de pruebas

El banco de pruebas estará formado por dos lazos cerrados:

$$T_{\text{ref}} \longrightarrow \texttt{Controller_Normal} \longrightarrow Q_{1,\text{Normal}} \longrightarrow G_{p,d} \longrightarrow T_{1,\text{Normal}},$$

y

$$T_{\text{ref}} \longrightarrow \texttt{Controller_SIL} \longrightarrow Q_{1,\text{SIL}} \longrightarrow G_{p,d} \longrightarrow T_{1,\text{SIL}}.$$

Los bloques `Controller_Normal` y `Controller_SIL` referenciarán exactamente el mismo archivo

`Controller.slx`.

La diferencia entre ellos será únicamente el modo utilizado para ejecutar el controlador.

Cada rama utilizará una copia idéntica del modelo discreto de la planta. Las dos copias deberán conservar:

- los mismos parámetros K, τ y θ;
- el mismo retardo discreto;
- la misma temperatura ambiente T_{amb};
- la misma condición inicial;
- el mismo periodo de muestreo; y
- la misma configuración numérica.

La fuente de referencia `T_ref` y la fuente de temperatura ambiente `T_amb` podrán compartirse entre ambas ramas.

No debe utilizarse una única planta para los dos controladores. Las señales $Q_{1,\text{Normal}}$ y $Q_{1,\text{SIL}}$ se calculan de manera independiente y, por tanto, cada una debe excitar su propia copia de la planta.

16.2 Creación del modelo de prueba

Abra el modelo

`A9_Code_preparation.slx`

y guarde una copia mediante

`File > Save As`.

Utilice el nombre

`TCLab_Test_SIL.slx`.

Este nuevo archivo será el modelo padre utilizado para las pruebas SIL y, posteriormente, para la incorporación de PIL.

Antes de realizar modificaciones, compruebe que el modelo contenga todavía:

- la fuente `T_ref`;
- la instancia del modelo referenciado `Controller.slx`;
- el retardo discreto de la planta;
- el modelo FOPDT discreto;
- la fuente `T_amb`;
- la realimentación de `T1`; y
- los elementos necesarios para visualizar o registrar los resultados.

Ejecute el modelo una vez en modo Normal y compruebe que el lazo cerrado mantenga el comportamiento obtenido previamente.

No modifique los parámetros del controlador ni los parámetros de la planta durante la construcción del banco de pruebas.

16.3 Rama de referencia Normal

La rama existente se utilizará como referencia basada en la ejecución directa del modelo de Simulink.

Renombre la instancia existente del modelo referenciado como

`Controller_Normal`.

Compruebe que continúe referenciando

`Controller.slx`

y configure

`Simulation mode = Normal`.

Mantenga conectados a esta rama los bloques correspondientes al retardo y al modelo discreto de la planta.

La salida de esta rama se identificará posteriormente mediante las señales

$$Q_{1,\text{Normal}} \quad \text{y} \quad T_{1,\text{Normal}}.$$

16.4 Creación de la rama SIL

Duplique el bloque `Controller_Normal` y renombre la nueva instancia como

`Controller_SIL`.

Compruebe que esta segunda instancia también referencia

`Controller.slx`.

Duplique ahora todos los bloques que representan la dinámica de la planta dentro de la rama Normal. La segunda copia debe incluir, según corresponda:

- el retardo discreto;
- el modelo FOPDT discreto;
- la incorporación de T_{amb}; y
- los bloques necesarios para obtener T_1.

Conecte la salida de `Controller_SIL` exclusivamente a esta segunda copia de la planta y cierre la realimentación utilizando su propia señal `T1`.

Conecte a las dos instancias del controlador la misma señal `T_ref`. De igual forma, si se utiliza una única fuente `T_amb`, ramifique su salida hacia las dos copias de la planta.

Antes de modificar el modo de ejecución de la segunda instancia, compruebe visualmente que las dos ramas sean equivalentes en estructura.

A continuación, configure

`Controller_SIL`

con

`Simulation mode = Software-in-the-loop (SIL)`.

Después de aplicar el cambio, compruebe que Simulink identifique la segunda instancia como una ejecución SIL del modelo referenciado.

16.5 Condiciones comunes de simulación

La comparación posterior solamente será válida si ambas ramas se ejecutan bajo las mismas condiciones de prueba.

En `TCLab_Test_SIL.slx`, abra

`Modeling > Model Settings > Solver`

y compruebe:

```
Type             Fixed-step
Solver           discrete (no continuous states)
Fixed-step size  Ts
```

El modelo padre deberá operar con

$$T_s = 1\,\text{s}$$

y conservar el tiempo final utilizado para la prueba:

$$t_{\text{final}} = 1800\,\text{s}.$$

A diferencia de `Controller.slx`, cuyo tiempo final fue configurado como `inf`, el tiempo final de `TCLab_Test_SIL.slx` sí pertenece a la definición del experimento y puede continuar especificándose mediante

`Stop time = t_final`.

Compruebe también que las fuentes `T_ref` y `T_amb` conserven

`Sample time = Ts`.

No realice ajustes independientes de parámetros, condiciones iniciales o configuración numérica entre las dos ramas para intentar hacer coincidir sus resultados.

16.6 Inicialización del modelo padre

El modelo padre debe continuar proporcionando los datos correspondientes a la planta y a las condiciones de simulación.

Con `TCLab_Test_SIL.slx` abierto, seleccione

`Modeling > Model Properties > Callbacks`

y compruebe que el callback `InitFcn` ejecute

`A8_init_TCLab_SIMC.m`.

La separación de responsabilidades debe mantenerse:

- `TCLab_Test_SIL.slx` inicializa la planta y las condiciones del experimento; y
- `Controller.slx` contiene únicamente los datos requeridos para ejecutar y generar el código del controlador.

En particular, `Controller.slx` no debe depender de `t_final` ni requerir que el script `A8_init_TCLab_SIMC.m` se ejecute antes de generar su código.

16.7 Registro de las señales

La comparación Normal–SIL utilizará los resultados obtenidos simultáneamente durante una misma simulación.

Agregue bloques *To Workspace* para registrar:

Señal	Variable en MATLAB
$Q_{1,\text{Normal}}$	`Q1_Normal`
$T_{1,\text{Normal}}$	`T1_Normal`
$Q_{1,\text{SIL}}$	`Q1_SIL`
$T_{1,\text{SIL}}$	`T1_SIL`

Para los cuatro bloques configure

`Save format = Timeseries`.

Con $T_s = 1\,\text{s}$ y una simulación entre 0 y 1800 s, cada señal deberá contener 1801 muestras.

Conserve exactamente los nombres indicados, ya que serán utilizados en la siguiente sección por el procedimiento automático de comparación.

16.8 Comprobación de la arquitectura

Antes de ejecutar la comparación Normal–SIL, compruebe que:

- Compruebe que ambas instancias referencien el mismo modelo. Para verificarlo desde MATLAB, ejecute:

```
get_param('TCLab_Test_SIL/Controller_Normal','ModelName')
get_param('TCLab_Test_SIL/Controller_SIL','ModelName')
```

  En ambos casos deberá obtenerse

  `Controller`.

Esto confirma que `Controller_Normal` y `Controller_SIL` son dos instancias diferentes del mismo modelo referenciado `Controller.slx`.

- `Controller_Normal` utilice `Simulation mode = Normal`;
- `Controller_SIL` utilice `Simulation mode = Software-in-the-loop (SIL)`;
- cada controlador esté conectado a su propia copia de la planta;
- las dos plantas sean idénticas en estructura, parámetros y condiciones iniciales;
- ambas ramas reciban la misma señal `T_ref`;
- ambas utilicen la misma temperatura ambiente;
- el modelo completo opere con $T_s = 1\,\text{s}$;
- el tiempo final sea $1800\,\text{s}$; y
- se registren las cuatro señales requeridas para la comparación.

La arquitectura deberá dejar además espacio para incorporar posteriormente una tercera rama correspondiente a PIL, sin modificar las ramas Normal y SIL que se validarán en esta etapa.

No configure todavía ningún bloque PIL ni el procedimiento automático de comparación.

Actividad 13: construcción del banco de pruebas Normal–SIL

Construya el modelo `TCLab_Test_SIL.slx` y documente:

1. las dos instancias de `Controller.slx` utilizadas en el banco de pruebas;
2. el modo de simulación asignado a cada instancia;
3. las dos copias equivalentes del modelo de la planta;
4. las condiciones comunes utilizadas durante la simulación;
5. las variables utilizadas para registrar $Q_{1,\text{Normal}}$, $T_{1,\text{Normal}}$, $Q_{1,\text{SIL}}$ y $T_{1,\text{SIL}}$; y
6. la arquitectura final del banco de pruebas.

Incluya una captura del modelo completo en la que pueda identificarse claramente la separación entre las ramas Normal y SIL.
No evalúe todavía la equivalencia numérica entre ambas ejecuciones. Esta comparación se realizará en la siguiente etapa.

17 Ejecución y comparación Normal–SIL

Una vez construido el banco de pruebas, se realizará la verificación *back-to-back* entre la ejecución del controlador en modo `Normal` y la ejecución del código generado en modo SIL.

A diferencia de una estrategia basada en simulaciones sucesivas, las dos ejecuciones se encuentran ahora dentro del mismo modelo `TCLab_Test_SIL.slx`. Por tanto, las señales Normal y SIL se obtienen simultáneamente durante una única simulación y bajo las mismas condiciones de prueba.

La comparación utilizará:

$$Q_{1,\text{Normal}}, \qquad Q_{1,\text{SIL}},$$

y

$$T_{1,\text{Normal}}, \qquad T_{1,\text{SIL}}.$$

El objetivo es determinar si la ejecución del código generado reproduce el comportamiento del modelo de referencia.

17.1 Ejecución inicial del banco de pruebas

Abra

`TCLab_Test_SIL.slx`

y compruebe nuevamente que:

- `Controller_Normal` utilice `Simulation mode = Normal`;
- `Controller_SIL` utilice `Simulation mode = Software-in-the-loop (SIL)`;
- ambas ramas reciban la misma referencia y las mismas condiciones de simulación; y
- se encuentren configurados los cuatro bloques *To Workspace*.

Ejecute el modelo mediante

`Run`.

Durante esta simulación, Simulink ejecutará directamente el modelo `Controller.slx` en la rama Normal, mientras que en la rama SIL ejecutará el código generado para ese mismo controlador.

Al finalizar, compruebe en MATLAB que estén disponibles las variables

`Q1_Normal`, `T1_Normal`, `Q1_SIL`, `T1_SIL`.

Las cuatro deberán ser objetos `timeseries`.

17.2 Comprobaciones previas

Antes de calcular diferencias, deberán verificarse las condiciones necesarias para realizar una comparación muestra a muestra.

El procedimiento de comparación comprobará que:

- existan las cuatro variables requeridas;
- todas sean objetos `timeseries`;
- las señales Normal y SIL tengan las mismas dimensiones;
- cada señal contenga 1801 muestras;
- los vectores de tiempo Normal y SIL sean idénticos; y
- el intervalo de simulación corresponda a 0–1800 s.

Si alguna de estas condiciones no se cumple, la comparación no deberá continuar hasta identificar la causa.

17.3 Diferencias muestra a muestra

Para la señal de control se definirá

$$\Delta Q_1[k] = Q_{1,\mathrm{SIL}}[k] - Q_{1,\mathrm{Normal}}[k],$$

mientras que para la temperatura se utilizará

$$\Delta T_1[k] = T_{1,\mathrm{SIL}}[k] - T_{1,\mathrm{Normal}}[k].$$

Para ambas señales se calcularán dos indicadores:

$$e_{\max} = \max_k |\Delta y[k]|$$

y

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} \Delta y[k]^2}.$$

El máximo absoluto permite identificar la mayor discrepancia observada durante la simulación, mientras que el RMSE resume la magnitud global de las diferencias.

Se registrará además el instante en el que ocurre la máxima diferencia.

17.4 Criterio de equivalencia

Para esta práctica, realizada en la computadora host utilizando `double` y el hardware de generación

`Intel > x86-64 (Windows64)`,

se adoptará como criterio de verificación la coincidencia exacta muestra a muestra entre las ejecuciones Normal y SIL.

Por tanto, se requiere

$$Q_{1,\text{Normal}}[k] = Q_{1,\text{SIL}}[k]$$

y

$$T_{1,\text{Normal}}[k] = T_{1,\text{SIL}}[k]$$

para todas las muestras.

El resultado global será

`PASS`

únicamente cuando ambas señales presenten coincidencia exacta.

Si se detecta alguna diferencia, el resultado será

`FAIL`.

Los valores de máximo absoluto y RMSE deberán conservarse aun cuando la comparación sea exacta, ya que permiten cuantificar explícitamente la magnitud de las diferencias obtenidas.

Este criterio corresponde a la configuración utilizada en esta etapa y no debe interpretarse como un requisito general para cualquier plataforma SIL o PIL.

17.5 Script de comparación

La comparación se realizará mediante el archivo

`A10_comparar_Normal_SIL.m`.

Guarde este archivo en la misma carpeta de trabajo utilizada para `TCLab_Test_SIL.slx`, o en una ubicación incluida en el *path* de MATLAB.

El script utiliza las cuatro señales registradas durante la simulación:

`Q1_Normal`, `T1_Normal`, `Q1_SIL`, `T1_SIL`.

Antes de realizar la comparación, comprueba que:

- las cuatro variables estén disponibles;

- todas correspondan a objetos `timeseries`;
- cada señal contenga una muestra por instante de tiempo;
- los cuatro vectores de tiempo sean idénticos;
- se hayan obtenido 1801 muestras; y
- el intervalo registrado corresponda a 0–1800 s.

Posteriormente, el script calcula las diferencias

$$\Delta Q_1[k] = Q_{1,\mathrm{SIL}}[k] - Q_{1,\mathrm{Normal}}[k],$$

y

$$\Delta T_1[k] = T_{1,\mathrm{SIL}}[k] - T_{1,\mathrm{Normal}}[k].$$

Para cada señal determina:

- el máximo absoluto de la diferencia;
- el instante en el que ocurre dicha diferencia;
- el RMSE; y
- la existencia o no de igualdad exacta muestra a muestra.

El resultado global será

`PASS`

únicamente si tanto Q_1 como T_1 presentan igualdad exacta entre las ejecuciones Normal y SIL. En caso contrario, el resultado será

`FAIL`.

El script almacena además un resumen de los resultados en la estructura

`A10_Resultados_Normal_SIL`.

Después de ejecutar una simulación completa de `TCLab_Test_SIL.slx`, ejecute manualmente:

```
A10_comparar_Normal_SIL
```

y revise los resultados mostrados en la ventana de comandos antes de configurar su ejecución automática.

17.6 Comparación gráfica

El script genera una primera figura con las respuestas de ambas ejecuciones superpuestas.

La parte superior compara

$$Q_{1,\text{Normal}} \quad \text{y} \quad Q_{1,\text{SIL}},$$

mientras que la parte inferior compara

$$T_{1,\text{Normal}} \quad \text{y} \quad T_{1,\text{SIL}}.$$

Cuando ambas ejecuciones coinciden, las curvas correspondientes deberán superponerse.

Una segunda figura muestra directamente las diferencias

$$\Delta Q_1 = Q_{1,\text{SIL}} - Q_{1,\text{Normal}},$$

y

$$\Delta T_1 = T_{1,\text{SIL}} - T_{1,\text{Normal}}.$$

Estas gráficas permiten observar tanto la magnitud como la ubicación temporal de cualquier discrepancia entre las dos ejecuciones.

La inspección gráfica complementa los indicadores numéricos. Por tanto, el criterio `PASS`/`FAIL` debe establecerse a partir de la comparación muestra a muestra realizada por el script y no únicamente a partir de la apariencia visual de las curvas.

17.7 Criterio para continuar

La verificación SIL se considerará satisfactoria cuando:

- la simulación Normal–SIL finalice sin errores;
- las cuatro señales requeridas estén disponibles;
- todas contengan 1801 muestras;
- los vectores de tiempo sean idénticos;
- se hayan calculado las diferencias para Q_1 y T_1;
- se hayan revisado el máximo absoluto y el RMSE de ambas diferencias;
- se hayan inspeccionado las figuras de comparación y de error; y
- el procedimiento reporte `PASS`.

Un resultado `PASS` proporciona evidencia de que, bajo las condiciones de prueba utilizadas, la ejecución SIL del código generado reproduce el comportamiento obtenido mediante la ejecución Normal del modelo del controlador.

Esta conclusión corresponde exclusivamente al controlador definido como SUT. Las dos copias de la planta continúan ejecutándose como modelos de Simulink y no forman parte del código sometido a SIL.

La prueba realizada verifica además una condición específica de operación y no constituye una demostración de equivalencia para cualquier posible conjunto de entradas o condiciones de funcionamiento.

Si el procedimiento reporta `FAIL`, no continúe hacia PIL. Revise primero la configuración del SUT, los parámetros utilizados, las condiciones de simulación y la arquitectura del banco de pruebas.

Actividad 14: verificación back-to-back Normal–SIL

Ejecute el banco de pruebas `TCLab_Test_SIL.slx` y utilice `A10_comparar_Normal_SIL.m` para verificar la equivalencia entre las ejecuciones Normal y SIL.
Documente:

1. la correcta generación de `Q1_Normal`, `T1_Normal`,`Q1_SIL` y `T1_SIL`;
2. el número de muestras y la compatibilidad de los vectores de tiempo;
3. el máximo absoluto y el RMSE de ΔQ_1 y ΔT_1;
4. las gráficas con las respuestas Normal y SIL superpuestas;
5. las gráficas de ΔQ_1 y ΔT_1;
6. el resultado global `PASS`/`FAIL`; y
7. una explicación de lo que este resultado permite concluir respecto a la ejecución del código generado para el controlador.

Después de verificar manualmente el funcionamiento del script, configure `A10_comparar_Normal_SIL` en el callback `StopFcn` y compruebe que la comparación se ejecute automáticamente al finalizar una nueva simulación.
No continúe hacia PIL si la verificación SIL produce un resultado `FAIL`.

18 Cierre de SIL y transición a PIL

La comparación *back-to-back* realizada en la sección anterior constituye el cierre de la etapa de verificación *Software-in-the-Loop* (SIL).

En esta etapa, el modelo del controlador y el código generado fueron ejecutados bajo las mismas condiciones de prueba y conectados a copias equivalentes de la planta simulada. El resultado de esta comparación permite evaluar si la implementación generada reproduce el comportamiento del modelo utilizado como referencia.

18.1 Alcance de la verificación SIL

Durante la prueba SIL se compararon dos formas de ejecutar el mismo controlador:

- en modo `Normal`, Simulink ejecutó directamente el modelo contenido en `Controller.slx`; y
- en modo SIL, Simulink ejecutó el código generado para ese mismo controlador.

Por tanto, un resultado

`PASS`

proporciona evidencia de que, para las condiciones utilizadas durante la prueba, la ejecución del código generado conserva el comportamiento numérico del modelo de referencia.

En particular, se verificó la equivalencia de:

- la señal de control Q_1;
- la respuesta de temperatura T_1;
- la dinámica interna del integrador;
- los límites de saturación; y
- la lógica de anti-windup mediante *clamping*.

La prueba SIL no modifica el diseño del controlador. Su propósito es verificar la implementación generada a partir de un diseño que ya fue evaluado previamente durante MIL.

18.2 Qué no demuestra SIL

La prueba SIL realizada no implica que el controlador haya sido ejecutado todavía en el procesador que se utilizará durante una implementación física.

Durante esta etapa, el código se generó y compiló para

`Intel > x86-64 (Windows64)`,

y fue ejecutado en la computadora host.

Por tanto, SIL permite detectar discrepancias introducidas durante la generación o ejecución del código, pero no verifica todavía aspectos dependientes del procesador embebido, como:

- la arquitectura del microcontrolador;
- la representación numérica propia del hardware;
- el compilador y la cadena de herramientas del dispositivo;
- la ejecución del código sobre el procesador físico; y
- el mecanismo de comunicación utilizado para intercambiar datos entre MATLAB/Simulink y el hardware.

Estos aspectos se evaluarán mediante una prueba *Processor-in-the-Loop* (PIL).

18.3 De SIL a PIL

En una prueba PIL, la planta continuará ejecutándose en Simulink, pero el código del controlador dejará de ejecutarse en la computadora host.

En su lugar, el código será compilado para un procesador embebido y ejecutado físicamente en dicho dispositivo. Durante la simulación, Simulink enviará las entradas del controlador al procesador y recibirá de regreso la salida calculada.

Conceptualmente, la ejecución PIL puede representarse como

$$T_{\text{ref}},\, T_1 \longrightarrow \boxed{\begin{array}{c}\text{procesador}\\ \text{código del controlador}\end{array}} \longrightarrow Q_1,$$

mientras que la planta permanece simulada en la computadora.

Por tanto, PIL agrega una nueva capa de verificación:

$$\boxed{\text{modelo} \rightarrow \text{código} \rightarrow \text{procesador}}$$

sin requerir todavía que la planta física forme parte del experimento.

18.4 Hardware para la prueba PIL

El hardware utilizado originalmente por la TCLab en esta práctica es un

`Arduino Leonardo`.

Sin embargo, durante la preparación de la prueba PIL se comprobó que, en MATLAB/Simulink R2026a, la configuración disponible para Arduino Leonardo no proporciona el soporte de conectividad requerido para ejecutar una prueba PIL.

Esta limitación corresponde al soporte disponible para el hardware y no a la estructura del controlador ni al código desarrollado durante las etapas MIL y SIL.

Por esta razón, la verificación PIL se realizará utilizando un

`Arduino Uno R4 Minima`,

para el cual se dispone del mecanismo de conectividad necesario para realizar la ejecución PIL.

El cambio de hardware afecta exclusivamente la etapa de verificación PIL. Las conclusiones obtenidas durante MIL y SIL permanecen válidas para las pruebas realizadas en esas etapas.

18.5 Implicaciones del cambio de hardware

La prueba PIL con Arduino Uno R4 Minima no debe interpretarse como una implementación de la TCLab sobre dicho dispositivo.

La planta térmica continuará siendo simulada en Simulink y el Arduino será utilizado exclusivamente como procesador de prueba para ejecutar el código del controlador.

En consecuencia, durante PIL:

- no se conectarán los calentadores de la TCLab al Arduino Uno R4 Minima;
- no se utilizarán los sensores físicos de temperatura de la TCLab;
- las señales T_{ref} y T_1 serán suministradas por Simulink;
- el Arduino ejecutará únicamente el algoritmo del controlador; y
- la salida Q_1 será devuelta a Simulink para excitar el modelo simulado de la planta.

De esta forma, la prueba evalúa el efecto de ejecutar el algoritmo generado en un procesador real sin incorporar todavía la planta física al lazo de control.

18.6 Arquitectura para la comparación PIL

El banco de pruebas utilizado para la verificación SIL se conservará como referencia en

`TCLab_Test_SIL.slx`

Para la etapa PIL se utilizará un banco de pruebas independiente, denominado

`TCLab_Test_PIL.slx`,

obtenido a partir del modelo anterior.

No se utilizarán simultáneamente una instancia SIL y una instancia PIL que referencien el mismo modelo `Controller.slx`. En esta configuración, Simulink no permite combinar ambas modalidades de ejecución dentro de una misma simulación.

Por esta razón, la comparación PIL se realizará mediante dos ramas:

`Controller_Normal`
`Controller_PIL`

Ambas instancias deberán continuar referenciando el mismo modelo

`Controller.slx`.

La primera se ejecutará en modo `Normal`, mientras que la segunda ejecutará el código generado mediante *Processor-in-the-Loop* (PIL).

Cada controlador se conectará a su propia copia equivalente de la planta. Esta separación es necesaria porque cualquier diferencia entre las acciones de control

$$Q_{1,\text{Normal}} \quad \text{y} \quad Q_{1,\text{PIL}}$$

produciría posteriormente trayectorias distintas de

$$T_{1,\text{Normal}} \quad \text{y} \quad T_{1,\text{PIL}}.$$

Las dos ramas utilizarán:

- la misma señal `T_ref`;
- la misma temperatura ambiente;
- los mismos parámetros de la planta;
- la misma condición inicial;
- el mismo periodo de muestreo; y
- el mismo intervalo de simulación.

De esta manera, la ejecución `Normal` funcionará como referencia directa para evaluar el comportamiento obtenido cuando el código generado del controlador se ejecuta sobre el procesador utilizado para la prueba PIL.

Los resultados obtenidos previamente mediante SIL se conservarán como evidencia de esa etapa de verificación, pero no se volverán a ejecutar como parte de la comparación Normal–PIL.

18.7 Artefactos que deben conservarse

Antes de iniciar la configuración PIL, conserve los artefactos correspondientes a la etapa SIL:

- `Controller.slx`;
- `TCLab_Test_SIL.slx`;
- `A8_init_TCLab_SIMC.m`;
- `A10_comparar_Normal_SIL.m`;
- el código generado durante SIL; y
- la evidencia del resultado `PASS` obtenida durante la comparación Normal–SIL.

Estos elementos constituyen el punto de referencia para la siguiente etapa.

No modifique el controlador para intentar adaptarlo al hardware PIL antes de evaluar si la configuración del nuevo objetivo requiere algún cambio específico.

18.8 Criterio para iniciar PIL

La etapa PIL podrá iniciarse cuando:

- la generación de código SIL finalice satisfactoriamente;
- la comparación Normal–SIL reporte `PASS`;
- `Controller.slx` permanezca separado de la planta;
- el banco de pruebas Normal–SIL se encuentre guardado y funcional;
- las ramas Normal y SIL se conserven sin modificaciones; y
- se disponga del Arduino Uno R4 Minima requerido para la prueba PIL.

La configuración del hardware, la generación de código para el nuevo objetivo y la ejecución PIL se realizarán en la siguiente etapa.

Actividad 15: cierre de SIL y preparación para PIL

Documente el cierre de la etapa SIL indicando:

1. el resultado final de la comparación Normal–SIL;
2. qué aspectos del controlador fueron verificados mediante SIL;
3. qué aspectos permanecen pendientes de verificar sobre un procesador físico;
4. la razón por la cual Arduino Leonardo no será utilizado para la prueba PIL;
5. el hardware seleccionado para realizar la verificación PIL; y
6. la arquitectura propuesta para ampliar el banco de pruebas a una comparación Normal–SIL–PIL.

Conserve los archivos y resultados correspondientes a la etapa SIL antes de modificar la configuración necesaria para PIL.

19 Preparación del banco de pruebas para PIL

Una vez completada satisfactoriamente la verificación SIL, la siguiente etapa consiste en ejecutar el código generado del controlador en un procesador físico mediante una prueba *Processor-in-the-Loop* (PIL).

El controlador continúa siendo el sistema bajo prueba (SUT). La planta FOPDT, el retardo, la temperatura ambiente y la referencia permanecen simulados en Simulink. La diferencia fundamental respecto a SIL es que el código del controlador deja de ejecutarse en la computadora host y pasa a ejecutarse en el procesador utilizado para la prueba PIL.

En esta práctica se empleará un Arduino Uno R4 Minima como procesador para la verificación PIL.

La placa Arduino Leonardo utilizada por la TCLab es compatible con otros flujos de trabajo de MATLAB y Simulink, pero en MATLAB R2026a no proporciona la conectividad requerida para la prueba PIL utilizada en esta guía. Por esta razón, el Uno R4 Minima se utilizará únicamente como plataforma de ejecución del código del controlador.

La prueba PIL realizada en esta sección no constituye todavía una implementación física del controlador sobre la TCLab. No se utilizarán los sensores ni los calentadores de la planta real.

19.1 Conservación de la configuración SIL

Antes de modificar la configuración de hardware de `Controller.slx`, compruebe que la etapa SIL haya sido cerrada y que se hayan conservado sus resultados.

Mantenga como mínimo:

- `TCLab_Test_SIL.slx`;
- la configuración de `Controller.slx` utilizada durante SIL;
- el código generado durante esa etapa;
- los resultados de la comparación Normal–SIL;
- las figuras obtenidas mediante `A10_comparar_Normal_SIL.m`; y
- el punto de control correspondiente a la configuración SIL validada.

Los archivos generados automáticamente dentro de `slprj` no deben considerarse la única copia del código SIL que se desea conservar, ya que esa carpeta se regenerará al cambiar el hardware objetivo.

19.2 Creación del banco de pruebas PIL

Abra

`TCLab_Test_SIL.slx`

y guarde una copia mediante `Save As` con el nombre

`TCLab_Test_PIL.slx`.

Trabaje únicamente sobre esta nueva copia.

La verificación PIL utilizará dos ramas:

- una ejecución `Normal`, utilizada como referencia; y
- una ejecución PIL del código del controlador.

No mantenga simultáneamente una instancia SIL y una instancia PIL que referencien el mismo modelo `Controller.slx`. Esa combinación no está soportada para este tipo de simulación.

Elimine del nuevo banco de pruebas toda la rama utilizada exclusivamente para SIL.

Conserve la rama `Normal` y construya una segunda rama idéntica para PIL.

Las dos ramas deben compartir únicamente las fuentes de

`T_ref` y `T_amb`.

Cada rama debe conservar su propia copia del retardo y de la planta FOPDT.

Esta separación es necesaria porque una eventual diferencia entre las acciones de control Normal y PIL producirá dos trayectorias de temperatura diferentes.

19.3 Arquitectura Normal–PIL

Configure los bloques *Model* de manera que ambos continúen referenciando

`Controller.slx`.

Utilice los nombres

`Controller_Normal` y `Controller_PIL`.

Configure

`Controller_Normal: Simulation mode = Normal`

y

`Controller_PIL: Simulation mode = Processor-in-the-loop (PIL).`

Para `Controller_PIL`, mantenga

`Code interface = Model reference`.

Registre las señales mediante bloques `To Workspace` con

`Save format = Timeseries`.

Utilice los nombres

Rama Normal: `Q1_Normal`, `T1_Normal`,
Rama PIL: `Q1_PIL`, `T1_PIL`.

Mantenga las mismas condiciones utilizadas durante las etapas anteriores:

$$T_s = 1\,\text{s}, \qquad t_{\text{final}} = 1800\,\text{s}.$$

Por tanto, cada señal registrada deberá contener 1801 muestras.

19.4 Callbacks del banco de pruebas

Abra

`Modeling > Model Properties > Callbacks`.

Mantenga en `InitFcn`

`A8_init_TCLab_SIMC`

para inicializar los parámetros de la planta y del controlador.

Deje vacíos

`StartFcn` y `StopFcn`.

En particular, elimine cualquier llamada automática a

`A10_comparar_Normal_SIL.m`.

La comparación Normal–PIL se ejecutará manualmente después de finalizar la prueba PIL.

Actividad 16: banco de pruebas Normal–PIL

Documente la arquitectura final de `TCLab_Test_PIL.slx`.
Incluya:

- una captura del banco de pruebas Normal–PIL;
- la identificación del SUT;
- las señales que cruzan la frontera del SUT;
- las variables utilizadas para registrar Q_1 y T_1 en ambas ramas; y
- una explicación breve de por qué la planta debe permanecer duplicada y separada para las dos ejecuciones.

20 Configuración del controlador para PIL

La configuración SIL utilizó la computadora host como plataforma de ejecución del código. Para PIL, el mismo modelo del controlador deberá generar código para el procesador del Arduino Uno R4 Minima.

No modifique la estructura algorítmica del controlador durante esta transición.

Deben conservarse:

- la ley PI–SIMC;
- la saturación;
- el anti-windup mediante *clamping*;
- los valores de `Kp_SIMC` y `Ki_SIMC`;
- $Q_{\min} = 0\,\%$ y $Q_{\max} = 90\,\%$;
- $T_s = 1\,\mathrm{s}$; y
- el uso del tipo de datos `double`.

20.1 Comprobación del Arduino

Conecte el Arduino Uno R4 Minima mediante USB.

Como comprobación inicial, ejecute en MATLAB

```
a = arduino;
a
```

Compruebe que MATLAB identifique

`Board: UnoR4Minima`.

Registre además el puerto COM asignado al dispositivo.

El número del puerto depende de cada computadora y no debe suponerse que será el mismo utilizado en otra instalación.

Una vez terminada la comprobación, libere la conexión mediante

```
clear a
```

antes de iniciar PIL.

20.2 Hardware objetivo

Abra

`Controller.slx`

y acceda a

`Modeling > Model Settings > Hardware Implementation`.

Configure

`Hardware board = Arduino Uno R4 Minima`.

Compruebe que Simulink identifique automáticamente la arquitectura correspondiente al procesador ARM Cortex utilizado por la placa.

Mantenga el target de generación de código

`ert.tlc`.

La selección de `ert.tlc` continúa indicando que se generará código orientado a sistemas embebidos; la selección del Uno R4 Minima determina ahora la plataforma para la cual ese código será compilado.

20.3 Configuración de comunicación PIL

Dentro de

`Hardware Implementation > Target hardware resources`

revise la configuración de comunicación PIL.

Utilice:

Hardware serial port:	`Serial 0`,
Set host COM port:	`Automatically`,
Baud rate:	`115200 (Default)`.

Si la detección automática no identifica correctamente la placa, seleccione manualmente el puerto COM determinado durante la comprobación anterior.

Mantenga además

`Build action = Build, load and run`.

20.4 Configuración de Model Referencing

En `Controller.slx`, abra

`Model Settings > Model Referencing`.

Mantenga

```
Total number of instances allowed per top model = Multiple.
```

Configure

```
Propagate sizes of variable-size signals = During execution.
```

Esta configuración debe establecerse aun cuando las señales utilizadas en este controlador sean escalares y de tamaño fijo, ya que es requerida por la ejecución SIL/PIL del bloque de modelo bajo esta configuración.

Compruebe también que el modelo del controlador conserve

```
Stop time = inf.
```

El tiempo de ensayo de 1800 s pertenece al banco de pruebas `TCLab_Test_PIL.slx` y no al componente `Controller.slx`.

20.5 Limpieza de artefactos de generación

El cambio desde el hardware utilizado durante SIL hacia el Arduino modifica la configuración de compilación, los tipos y las utilidades compartidas generadas por Embedded Coder.

Para evitar reutilizar artefactos correspondientes a la plataforma anterior, guarde primero todos los modelos y confirme que el código SIL que desea conservar se encuentre respaldado.

A continuación, cierre los modelos y, desde la carpeta principal del proyecto, ejecute

```
if isfolder("slprj")
    rmdir("slprj","s")
end

if isfile("Controller.slxc")
    delete("Controller.slxc")
end

if isfile("TCLab_Test_PIL.slxc")
    delete("TCLab_Test_PIL.slxc")
end
```

Esta operación elimina únicamente artefactos generados y cachés. No elimina los modelos `.slx` ni los scripts de la práctica.

Después de la limpieza, vuelva a abrir

`Controller.slx` y `TCLab_Test_PIL.slx`.

No ejecute `slbuild` manualmente en esta etapa. El flujo PIL generará, compilará y descargará automáticamente el código requerido.

20.6 Comprobación previa

Antes de iniciar la prueba PIL, confirme que:

- el Arduino Uno R4 Minima está conectado;
- ningún objeto `arduino` mantiene ocupado el puerto serie;
- `Controller.slx` tiene seleccionado `Arduino Uno R4 Minima`;
- se mantiene `ert.tlc`;
- `Propagate sizes of variable-size signals` está configurado como `During execution`;
- `Controller_Normal` permanece en modo `Normal`;
- `Controller_PIL` está en modo PIL;
- no existe ningún bloque SIL activo en el banco;
- `StartFcn` y `StopFcn` están vacíos; y
- el `InitFcn` continúa ejecutando `A8_init_TCLab_SIMC.m`.

Actividad 17: configuración del target PIL

Documente la configuración utilizada para ejecutar el controlador en el Arduino Uno R4 Minima. Incluya:

- identificación de la placa y del puerto COM;
- captura de la selección del hardware objetivo;
- configuración de la comunicación PIL;
- configuración de `Propagate sizes of variable-size signals`; y
- una explicación de por qué fue necesario regenerar los artefactos de código después de cambiar de plataforma.

21 Ejecución Processor-in-the-Loop

Una simulación PIL ejecuta sobre el procesador objetivo el código compilado del componente bajo prueba, mientras Simulink continúa ejecutando el banco de pruebas en la computadora host.

En esta práctica, Simulink calcula la referencia y la dinámica de la planta. En cada paso de simulación envía al Arduino las entradas

`T_ref` y `T1`.

El código del controlador se ejecuta en el Uno R4 Minima y devuelve a Simulink

`Q1`.

Esta comunicación se repite para cada muestra del ensayo.

La prueba PIL no se ejecuta en tiempo real. El tiempo de simulación representa el tiempo del modelo y no el tiempo de reloj necesario para completar el intercambio host–target.

21.1 Configuración del SIL/PIL Manager

Abra

`TCLab_Test_PIL.slx`.

Seleccione la aplicación

`SIL/PIL Manager`.

Configure:

System Under Test:	`Model blocks in SIL/PIL mode`,
Top Model Mode:	`Normal`,
Stop Time:	`t_final`.

Compruebe que el único bloque identificado para ejecución SIL/PIL sea

`Controller_PIL`.

21.2 Ejecución

Seleccione

`Run SIL/PIL`.

Durante este proceso, Embedded Coder realiza automáticamente varias etapas:

1. genera o actualiza el código C de `Controller.slx`;
2. compila el código utilizando el toolchain correspondiente al Arduino Uno R4 Minima;
3. genera el código necesario para la comunicación PIL;
4. construye la aplicación destinada al procesador;
5. descarga la aplicación al Arduino;
6. inicia la comunicación entre Simulink y el procesador; y
7. ejecuta la simulación Normal–PIL hasta $t = 1800\,\mathrm{s}$.

No es necesario realizar manualmente una operación adicional de descarga al Arduino. La descarga forma parte de `Run SIL/PIL`.

Durante una ejecución correcta, el *Diagnostic Viewer* deberá mostrar mensajes equivalentes a

```
Using toolchain: Arduino ARM M4
Connectivity configuration for component "Controller"
COM port: ...
Baud rate: 115200
Preparing to start PIL simulation
Deployed code to target successfully
Starting PIL Simulation
Successfully completed PIL Simulation
```

La aparición de

```
Deployed code to target successfully
```

confirma que la aplicación fue descargada al procesador.

La aparición posterior de

```
Successfully completed PIL Simulation
```

confirma que el intercambio host–target y la ejecución PIL finalizaron sin errores.

Si el panel de código indica

```
No code for "TCLab_Test_PIL"
```

no lo interprete como un error. El modelo padre constituye el banco de pruebas y permanece en modo `Normal`. El componente cuyo código se genera y se ejecuta mediante PIL es `Controller_PIL`.

21.3 Comprobación de los resultados registrados

Al finalizar, compruebe en el *Base Workspace* la existencia de

`Q1_Normal`, `T1_Normal`, `Q1_PIL`, `T1_PIL`.

Puede utilizar

```
whos Q1_Normal T1_Normal Q1_PIL T1_PIL
```

Las cuatro variables deben ser objetos

`timeseries`.

Observe que `whos` muestra cada `timeseries` como un objeto 1×1. Este tamaño corresponde al objeto y no al número de muestras contenidas en él.

Compruebe el número real de muestras mediante

```
[numel(Q1_Normal.Time), ...
 numel(T1_Normal.Time), ...
 numel(Q1_PIL.Time), ...
 numel(T1_PIL.Time)]
```

El resultado esperado es

```
1801   1801   1801   1801
```

Compruebe además que los vectores de tiempo sean idénticos:

```
isequal(Q1_Normal.Time,Q1_PIL.Time)
isequal(T1_Normal.Time,T1_PIL.Time)
isequal(Q1_Normal.Time,T1_Normal.Time)
```

Cada expresión deberá devolver

`logical 1`.

Revise finalmente que

$$0 \leq Q_{1,\text{Normal}} \leq 90\,\%$$

y

$$0 \leq Q_{1,\text{PIL}} \leq 90\,\%.$$

Una ejecución PIL sin errores demuestra que el código pudo ser generado, compilado, descargado y ejecutado sobre el procesador, pero todavía no demuestra por sí sola equivalencia numérica respecto al modelo.

Esa comprobación se realizará mediante una comparación *back-to-back*.

Actividad 18: ejecución PIL

Conserve evidencia de una ejecución PIL satisfactoria.
Incluya:

- el hardware y puerto utilizados;
- evidencia de la compilación para el Arduino;
- evidencia del mensaje `Deployed code to target successfully`;
- evidencia de `Successfully completed PIL Simulation`;
- cantidad de muestras registradas para las cuatro señales; y
- confirmación de que los vectores de tiempo son idénticos.

No determine todavía la equivalencia Normal–PIL únicamente mediante inspección visual.

22 Comparación *back-to-back* Normal–PIL

Después de completar la simulación PIL debe comprobarse si la ejecución del código sobre el procesador reproduce el comportamiento obtenido con el modelo del controlador en modo `Normal`.

Debido a que ambas ramas se ejecutan dentro del mismo banco de pruebas, reciben simultáneamente la misma referencia, utilizan los mismos parámetros y parten de las mismas condiciones iniciales.

La comparación se realiza muestra a muestra entre

$$Q_{1,\mathrm{Normal}}[k] \quad \text{y} \quad Q_{1,\mathrm{PIL}}[k]$$

y entre

$$T_{1,\mathrm{Normal}}[k] \quad \text{y} \quad T_{1,\mathrm{PIL}}[k].$$

22.1 Script de comparación

Para esta actividad se suministra

`A11_comparar_Normal_PIL.m`.

Este script debe ejecutarse manualmente después de finalizar `Run SIL/PIL`.

Ejecute

```
A11_comparar_Normal_PIL
```

El script comprueba automáticamente:

- existencia de las cuatro variables requeridas;
- formato `timeseries`;
- compatibilidad de dimensiones;
- ausencia de valores `NaN` o `Inf`;
- igualdad de los cuatro vectores de tiempo;
- existencia de 1801 muestras;
- intervalo temporal entre 0 y 1800 s; y
- diferencias entre las ejecuciones Normal y PIL.

22.2 Métricas de comparación

Para una señal x, se define

$$e_x[k] = x_{\text{PIL}}[k] - x_{\text{Normal}}[k].$$

El script calcula el máximo absoluto

$$e_{\max} = \max_k |e_x[k]|$$

y el error cuadrático medio

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e_x^2[k]}.$$

Estas métricas se calculan independientemente para Q_1 y T_1.

Además, el script comprueba mediante igualdad exacta si los vectores completos de ambas ejecuciones contienen los mismos valores muestra a muestra.

22.3 Evidencia gráfica

El script genera dos grupos de figuras.

El primero superpone:

- $Q_{1,\text{Normal}}$ y $Q_{1,\text{PIL}}$; y
- $T_{1,\text{Normal}}$ y $T_{1,\text{PIL}}$.

El segundo presenta directamente

$$\Delta Q_1 = Q_{1,\text{PIL}} - Q_{1,\text{Normal}}$$

y

$$\Delta T_1 = T_{1,\text{PIL}} - T_{1,\text{Normal}}.$$

Las gráficas de diferencias permiten detectar discrepancias que podrían no observarse cuando las curvas se encuentran superpuestas.

22.4 Registro de resultados

Registre los valores obtenidos en la Tabla 22.

Tabla 22: Comparación *back-to-back* Normal–PIL.

Señal	Máximo absoluto	RMSE	Igualdad exacta	Resultado
Q_1				
T_1				

Para el banco de pruebas utilizado en esta guía se aplicará un criterio deliberadamente estricto:

$$\texttt{PASS} \iff Q_{1,\text{PIL}} = Q_{1,\text{Normal}} \text{ y } T_{1,\text{PIL}} = T_{1,\text{Normal}} \text{ muestra a muestra.}$$

Por tanto, una ejecución `PASS` deberá presentar

$$e_{\max,Q_1} = 0, \qquad \text{RMSE}_{Q_1} = 0$$

y

$$e_{\max,T_1} = 0, \qquad \text{RMSE}_{T_1} = 0.$$

La igualdad exacta utilizada en esta práctica no debe interpretarse como un requisito universal de toda prueba PIL. Diferentes procesadores, representaciones numéricas, compiladores u optimizaciones pueden producir pequeñas diferencias numéricas aun cuando el código sea funcionalmente equivalente. El criterio aplicado aquí corresponde específicamente al SUT y a la configuración utilizada en esta práctica.

22.5 Guardado de los resultados

Después de ejecutar `A11_comparar_Normal_PIL.m`, conserve las señales y el resumen de resultados mediante

```
save("TCLab_Test_PIL_results.mat", ...
     "Q1_Normal","T1_Normal", ...
     "Q1_PIL","T1_PIL", ...
     "A11_Resultados_Normal_PIL");
```

Compruebe que se haya creado

`TCLab_Test_PIL_results.mat`.

Conserve además las figuras generadas por el script.

22.6 Diagnóstico ante resultados inesperados

Si se obtiene igualdad exacta y se desea comprobar de forma adicional que las dos ramas son realmente independientes, puede realizarse una prueba diagnóstica temporal.

Introduzca únicamente en la realimentación de la rama PIL una perturbación de

$$+1\,^{\circ}\text{C}$$

antes de la entrada `T1` de `Controller_PIL`.

No modifique la señal de temperatura utilizada por la rama Normal.

Ejecute nuevamente PIL y compruebe que aparecen diferencias entre las dos ramas.

Esta prueba no forma parte de los resultados nominales y debe utilizarse únicamente como diagnóstico.

Después de realizarla:

1. elimine completamente la perturbación;
2. restaure la conexión original;
3. ejecute nuevamente `Run SIL/PIL`; y
4. vuelva a ejecutar `A11_comparar_Normal_PIL.m`.

Solo esta última ejecución nominal debe utilizarse como resultado final de la verificación.

Actividad 19: comparación Normal–PIL

Ejecute `A11_comparar_Normal_PIL.m` sobre los resultados nominales del banco de pruebas. Entregue:

- el máximo absoluto y RMSE para Q_1 y T_1;
- el resultado de la comprobación de igualdad exacta;
- las figuras Normal–PIL;
- las figuras de las diferencias PIL–Normal;
- el resultado global `PASS/FAIL`; y
- una explicación de qué evidencia aporta esta comparación respecto al código ejecutado en el procesador.

Si el resultado es `FAIL`, no cierre la etapa PIL hasta identificar la causa de las diferencias.

23 Cierre de la etapa PIL

Una ejecución PIL satisfactoria, seguida de una comparación *back-to-back* con resultado `PASS`, proporciona evidencia de que el código generado del controlador conserva el comportamiento de su modelo de referencia cuando se ejecuta sobre el procesador utilizado durante la prueba.

En esta etapa se ha verificado una cadena diferente a la utilizada durante SIL.

En SIL,

$$\text{modelo} \longrightarrow \text{código generado} \longrightarrow \text{compilación y ejecución en la computadora host.}$$

En PIL,

$$\text{modelo} \longrightarrow \text{código generado} \longrightarrow \text{compilación para el procesador objetivo} \longrightarrow \text{descarga} \longrightarrow \text{ejecución en el Arduino.}$$

23.1 Alcance de la verificación

Un resultado `PASS` proporciona evidencia de que, bajo las condiciones utilizadas:

- el código de `Controller.slx` puede compilarse para el Arduino Uno R4 Minima;
- la aplicación puede descargarse correctamente al procesador;
- la comunicación PIL entre Simulink y el Arduino funciona;
- las entradas `T_ref` y `T1` pueden transferirse al código ejecutado en el procesador;
- la salida `Q1` puede regresar a Simulink;
- el estado interno del controlador evoluciona correctamente durante toda la ejecución; y
- los resultados satisfacen el criterio de equivalencia establecido respecto a la ejecución Normal.

23.2 Aspectos que PIL no verifica

La prueba realizada no debe interpretarse como una validación completa del sistema físico.

Durante PIL:

- la planta FOPDT continúa siendo simulada;
- el retardo continúa siendo simulado;
- `T_amb` continúa siendo una señal de Simulink;
- no se mide la temperatura mediante el sensor de la TCLab;
- no se accionan físicamente los calentadores;
- no se verifican los canales de entrada/salida de la TCLab;
- no se comprueba una ejecución determinista en tiempo real; y

- el Arduino Uno R4 Minima actúa únicamente como procesador para la verificación PIL.

Por tanto, PIL debe interpretarse como una prueba del código y de su ejecución sobre un procesador físico, no como una prueba completa del controlador conectado a la planta real.

23.3 Conservación de la evidencia

Conserve como mínimo:

- `TCLab_Test_SIL.slx`;
- `TCLab_Test_PIL.slx`;
- `Controller.slx`;
- `A10_comparar_Normal_SIL.m`;
- `A11_comparar_Normal_PIL.m`;
- los resultados SIL;
- `TCLab_Test_PIL_results.mat`;
- las figuras de comparación;
- las métricas obtenidas; y
- evidencia de la compilación, descarga y ejecución PIL.

Registre la configuración validada mediante el punto de control

`tclab-pil-validated-v1`

Actividad 20: cierre de la verificación PIL

Prepare una síntesis final de la verificación realizada.
Explique:

- qué diferencia existe entre las ejecuciones Normal, SIL y PIL;
- qué parte del sistema fue ejecutada físicamente durante PIL;
- qué evidencia proporciona un resultado `PASS`;
- por qué la prueba realizada no constituye todavía una implementación completa del controlador sobre la TCLab; y
- cuáles aspectos del sistema físico permanecen fuera del alcance de esta verificación.

Conserve todos los archivos y resultados como evidencia del flujo de verificación basado en modelos.

24 Implementación física del controlador

Una vez finalizadas las etapas de verificación MIL, SIL y PIL, el controlador se implementará sobre la planta física utilizando un Arduino Leonardo y la TCLab.

A diferencia de las etapas anteriores, la planta ya no estará representada mediante un modelo de Simulink. El controlador se ejecutará físicamente en el microcontrolador y cerrará el lazo directamente con el calentador y el sensor de temperatura de la TCLab.

La arquitectura general de esta etapa será

`T_ref`
↓
`Controller_implementacion`
↓
`Q1`
↓
PWM D3
↓
TCLab física
↓
Sensor `T1`
↓
Analog Input A0
⟶ realimentación al controlador

La ejecución se realizará mediante *External Mode*, utilizando la opción `Monitor & Tune` de Simulink. Esta modalidad permite ejecutar el código en el Arduino mientras Simulink mantiene comunicación con la tarjeta para monitorizar señales y registrar los resultados experimentales.

Planta física

Durante esta etapa no debe incluirse ningún modelo FOPDT ni otro modelo de la planta dentro del lazo cerrado. La dinámica observada corresponderá exclusivamente a la TCLab física.

24.1 Arquitectura de ejecución con `Monitor & Tune`

En esta etapa se utilizará `Monitor & Tune`, también denominado *External Mode*, para ejecutar el controlador sobre el Arduino Leonardo mientras se mantiene comunicación con Simulink.

Es importante distinguir entre la representación gráfica del modelo y el lugar donde se ejecutan sus operaciones. Cuando se inicia `Monitor & Tune`, Simulink genera el código correspondiente al modelo principal y a los modelos referenciados que forman parte de su jerarquía, compila la aplicación y la descarga al hardware objetivo.

En este caso, el modelo principal será

`TCLab_Implementation.slx`,

y dentro de él se incorporará

`Controller_implementacion.slx`

mediante un bloque `Model`.

Por tanto, durante la ejecución, las operaciones que forman parte del lazo de control se realizan físicamente en el Arduino Leonardo. En cada periodo de muestreo, el microcontrolador realiza conceptualmente la secuencia

lectura de A0
↓
conversión de la lectura a temperatura
↓
filtrado de T_1
↓
ejecución del controlador PI–SIMC
↓
cálculo y limitación de Q_1
↓
conversión a PWM
↓
escritura en D3.

Así, Simulink no lee la entrada analógica para posteriormente calcular el controlador en la computadora. La adquisición, el procesamiento de la señal, el cálculo del controlador y la actuación se ejecutan en el Arduino.

La computadora mantiene únicamente un canal de comunicación con la aplicación que está ejecutándose en el hardware. Este enlace permite, entre otras funciones,

- observar señales internas mediante bloques como `Scope`;
- registrar señales experimentales mediante *signal logging*; y
- modificar durante la ejecución aquellos parámetros que sean ajustables mediante External Mode.

Por ejemplo, cuando las señales `T_ref`, `T1` y `Q1` se marcan para registro, esta selección no determina qué partes del modelo se convierten en código. Estas señales se marcan únicamente para que sus valores puedan transmitirse desde el Arduino hacia Simulink durante la ejecución.

La generación de código está determinada por la jerarquía del modelo principal. En esta práctica, la aplicación descargada al Leonardo incluirá los bloques de adquisición y actuación contenidos en `TCLab_Implementation.slx`, así como el código correspondiente al modelo referenciado `Controller_implementacion.slx`.

Ejecución en el hardware

Durante `Monitor & Tune`, el lazo de control se ejecuta en el Arduino Leonardo. Simulink actúa como interfaz de monitorización, registro y ajuste de parámetros.
Los bloques utilizados únicamente para visualización, como el `Scope`, se ejecutan en la computadora y no forman parte del cálculo del lazo de control realizado por el microcontrolador.

24.2 Representación numérica para Arduino Leonardo

El controlador utilizado durante SIL y PIL fue validado utilizando señales y parámetros de tipo `double`. Sin embargo, al seleccionar el Arduino Leonardo como hardware objetivo, Simulink advierte que la representación numérica del tipo `double` en la computadora host y en el procesador AVR no posee la misma precisión.

En particular, durante las pruebas de implementación se observó la advertencia

`Simulink double = 64 bits`

y

`target AVR double = 32 bits.`

Por esta razón, la versión destinada al Leonardo utilizará explícitamente aritmética de precisión simple mediante el tipo `single`.

El modelo

`Controller.slx`

debe conservarse sin modificaciones, ya que representa la versión utilizada y validada previamente mediante SIL y PIL. Para la implementación física se creará una copia denominada

`Controller_implementacion.slx.`

Esta separación permite preservar la referencia previamente validada y mantener las modificaciones específicas del hardware final en un modelo independiente.

24.3 Preparación de `Controller_implementacion.slx`

Abra `Controller.slx` y guarde una copia con el nombre

`Controller_implementacion.slx.`

En esta copia, configure los bloques de entrada `T_ref` y `T1` con

`Data type = single.`

Los parámetros `Kp_SIMC` y `Ki_SIMC` deben mantenerse como objetos `Simulink.Parameter` y conservar

`Storage Class = ExportedGlobal.`

Cambie su tipo a `single` y defina también sus valores explícitamente como números de precisión simple:

```
Kp_SIMC = single(2.93829548166286)
Ki_SIMC = single(0.015058760578832)
```

Los límites de la acción de control deberán definirse igualmente como

```
Q_min = single(0)
Q_max = single(90)
```

Si `Q_min` y `Q_max` son variables ordinarias almacenadas en el *Model Workspace*, pueden modificarse desde MATLAB mediante

```
mw = get_param('Controller_implementacion','ModelWorkspace');

assignin(mw,'Q_min',single(0));
assignin(mw,'Q_max',single(90));
```

El periodo de muestreo se mantiene como

$$T_s = 1\,\text{s}.$$

No es necesario modificar manualmente los tipos internos del bloque `Discrete PID Controller`. Los productos proporcional e integral, el estado del integrador, la suma PI y la salida de saturación pueden conservar sus reglas de herencia.

Después de actualizar el modelo mediante `Ctrl+D`, utilice

`Debug > Information Overlays > Base Data Types`

para comprobar que las entradas, la señal de error, las operaciones internas del PI y la salida `Q1` se resuelven como `single`.

Configure además

`Hardware board = Arduino Leonardo`

y conserve

`System target file = ert.tlc`.

El modelo debe actualizarse sin errores ni advertencias antes de continuar.

Actividad: preparación del controlador para implementación

Prepare una versión del controlador destinada específicamente al Arduino Leonardo.

1. Conserve `Controller.slx` sin modificaciones.
2. Cree `Controller_implementacion.slx` como copia del modelo validado.
3. Configure `T_ref` y `T1` como señales `single`.
4. Configure `Kp_SIMC` y `Ki_SIMC` como parámetros `single`, manteniendo `Storage Class = ExportedGlobal`.
5. Configure `Q_min = single(0)` y `Q_max = single(90)`.
6. Mantenga $T_s = 1\,\text{s}$.
7. Seleccione `Arduino Leonardo` como hardware objetivo.
8. Active `Base Data Types` y compruebe que el camino interno del controlador, incluido el estado del integrador, se resuelva como `single`.
9. Actualice el modelo y compruebe que no existan errores ni advertencias.

No modifique `Controller.slx`. Esta versión continuará siendo la referencia correspondiente a las etapas de verificación anteriores.

24.4 Creación de `TCLab_Implementation.slx`

Cree un nuevo modelo de Simulink y guárdelo con el nombre

`TCLab_Implementation.slx`.

Este modelo se utilizará para integrar el controlador con las entradas y salidas físicas de la TCLab. A diferencia de las etapas MIL, SIL y PIL, en esta etapa no se incluirá ningún modelo de la planta, ya que el lazo se cerrará directamente sobre el sistema físico.

Configure el modelo con

Hardware board:	`Arduino Leonardo`
Solver type:	`Fixed-step`
Solver:	`discrete (no continuous states)`
Fixed-step size:	`1 s`
Stop time:	`inf`

En

`Hardware Implementation > Target hardware resources > Host-board connection`

compruebe los puertos utilizados por el Arduino Leonardo.

El Leonardo puede presentar un puerto durante la operación normal y otro durante el funcionamiento del *bootloader*. Los números asignados dependen de cada computadora, por lo que deben identificarse en cada estación de trabajo y no deben copiarse directamente de otra computadora.

Si la detección automática no permite establecer la conexión, configure manualmente ambos puertos.

Finalmente, actualice el modelo mediante `Ctrl+D` y compruebe que no se presenten errores ni advertencias antes de comenzar a incorporar los bloques de adquisición, control y actuación.

24.5 Adquisición de la temperatura `T1`

El sensor de temperatura `T1` de la TCLab se adquiere mediante la entrada analógica

`A0`.

El bloque `Analog Input` entrega una lectura entera sin signo en el intervalo

$$0 \leq N_{\text{ADC}} \leq 1023.$$

La conversión utilizada para obtener la temperatura en grados Celsius es

$$T_1 = \frac{500}{1023} N_{\text{ADC}} - 50.$$

Implemente por tanto la cadena

`Analog Input A0`
↓
`Data Type Conversion`
↓
`single`
↓
`Gain = single(500/1023)`
↓
`Bias = single(-50)`.

La señal resultante corresponde a la temperatura medida en °C.

24.6 Filtrado de la temperatura

La salida del convertidor A/D posee una resolución finita y puede producir pequeñas variaciones de una muestra a otra. Estas variaciones se trasladan directamente al error del controlador y pueden producir cambios innecesarios en la acción de control.

Para suavizar la medición, inserte un bloque

`Moving Average`

después de convertir la lectura del ADC a temperatura.

Configure

Method:	`Exponential weighting`
Forgetting factor:	`single(0.5)`
Simulate using:	`Code generation`

La arquitectura de adquisición quedará entonces

A0
↓
`uint16`
↓
`single`
↓
conversión a °C
↓
`Moving Average`
↓
`T1 [single]`.

La salida filtrada debe utilizarse tanto para la realimentación del controlador como para la monitorización del experimento.

24.7 Integración del controlador

Agregue al modelo un bloque `Model` y configure

`Model name = Controller_implementacion`.

El bloque deberá presentar las entradas

`T_ref`, `T1`

y la salida

`Q1`.

Conecte la temperatura filtrada a la entrada `T1`.

Para las comprobaciones iniciales utilice un bloque `Constant` para generar la referencia y defina su valor explícitamente como `single`.

La estructura será

$$\left.\begin{matrix} T_{\text{ref}} \\ T_1 \end{matrix}\right\} \longrightarrow \texttt{Controller_implementacion} \longrightarrow Q_1.$$

24.8 Actuación sobre el calentador

El calentador 1 de la TCLab se controla mediante la salida PWM

`D3.`

Aunque el controlador ya contiene una saturación interna entre 0 y 90 %, incorpore una segunda saturación directamente antes de la cadena de actuación:

```
Lower limit = single(0)
```

```
Upper limit = single(90).
```

Esta saturación constituye una protección adicional en la interfaz con el actuador.

El bloque PWM utiliza valores enteros entre 0 y 255. La conversión entre la acción de control expresada en porcentaje y el valor enviado al bloque PWM se realizará mediante

$$N_{\mathrm{PWM}} = Q_1 \left(0.9 \frac{255}{100} \right).$$

Implemente la cadena

```
Q1 [single]
      ↓
Saturation [0,90]
      ↓
Gain = single(0.9*255/100)
      ↓
Data Type Conversion
      ↓
    uint8
      ↓
PWM D3.
```

Antes de ejecutar el modelo, compruebe que la salida del controlador sea la única señal conectada a esta cadena de actuación.

Actividad: construcción del lazo físico

Construya `TCLab_Implementation.slx` utilizando las entradas y salidas físicas de la TCLab.

1. Cree `TCLab_Implementation.slx` a partir de `TCLab_IO_Test.slx`.
2. Configure el Arduino Leonardo, el solver discreto y $T_s = 1\,\text{s}$.
3. Configure los puertos de operación normal y *bootloader* correspondientes a su computadora.
4. Implemente la adquisición de `T1` desde `A0`.
5. Convierta la lectura del ADC a grados Celsius utilizando `single(500/1023)` y `single(-50)`.
6. Incorpore el filtro exponencial con factor de olvido `single(0.5)`.
7. Inserte un bloque `Model` que referencie `Controller_implementacion.slx`.
8. Conecte la temperatura filtrada a la entrada `T1` del controlador.
9. Conecte la salida `Q1` a la cadena de actuación PWM.
10. Incorpore una saturación externa entre 0 y 90 % antes de la conversión a PWM.
11. Configure la salida física en `D3`.
12. Actualice el modelo mediante `Ctrl+D` y compruebe que no existan errores ni advertencias.

No ejecute todavía `Monitor & Tune` hasta comprobar la arquitectura completa y los límites de la acción de control.

24.9 Monitorización durante el experimento

Agregue un bloque `Scope` para observar las variables principales durante la ejecución.

Configure el Scope con dos paneles.

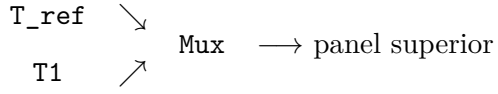
En el panel superior se mostrarán

$$T_{\text{ref}} \qquad \text{y} \qquad T_1.$$

En el panel inferior se mostrará

$$Q_1.$$

Para agrupar las dos señales de temperatura puede utilizar un bloque `Mux`:



La señal `Q1` se conecta directamente al segundo puerto del Scope.

Configure

Layout:	`2 x 1`
Time span:	`600 s`
Overrun action:	`Scroll`.

Una ventana de 600 s permite observar el transitorio térmico y, al mismo tiempo, mantener suficiente resolución temporal para inspeccionar la respuesta.

Las conexiones destinadas al Scope deben realizarse mediante ramificaciones de las señales existentes y no deben introducir bloques en serie dentro del camino de control.

24.10 Registro de las señales experimentales

Durante el experimento se registrarán

`T_ref`, `T1`, `Q1`.

Asigne estos nombres explícitamente a las señales y active

`Log Selected Signals`

para cada una de ellas.

En

`Model Settings > Data Import/Export`

compruebe que

Signal logging:	activado
Signal logging name:	`logsout`
Save format:	`Dataset`
Dataset signal format:	`timeseries`

Después de detener normalmente la ejecución, las señales registradas podrán recuperarse desde el objeto `logsout`.

El procesamiento posterior debe realizarse utilizando los nombres de las señales y no su posición dentro del `Dataset`. De esta forma, cualquier señal adicional registrada accidentalmente no modifica el análisis de `T_ref`, `T1` y `Q1`.

24.11 Comprobación previa a la ejecución

Antes de energizar el calentador, revise nuevamente toda la arquitectura.

Compruebe que:

- el hardware seleccionado sea el Arduino Leonardo;

- el modelo utilice un solver discreto con $T_s = 1\,\text{s}$;
- `T1` se adquiera desde `A0`;
- la señal de temperatura sea de tipo `single`;
- la señal filtrada sea la que realimenta al controlador;
- `Controller_implementacion.slx` sea el modelo referenciado;
- `Q1` sea de tipo `single`;
- la saturación interna y la saturación externa limiten Q_1 al intervalo 0–90 %;
- la conversión hacia PWM utilice `single(0.9*255/100)`;
- la entrada del bloque PWM sea `uint8`;
- el calentador 1 esté conectado al pin `D3`; y
- las señales conectadas al Scope sean únicamente ramificaciones para monitorización.

Seguridad antes de ejecutar

No inicie `Monitor & Tune` si la lectura de temperatura es incoherente, si la salida del controlador no está limitada o si existe alguna duda sobre la conexión hacia el bloque PWM.
Durante cualquier prueba, detenga inmediatamente la ejecución si la temperatura o la acción de control presentan un comportamiento inesperado.

24.12 Ejecución mediante `Monitor & Tune`

La primera ejecución debe realizarse con una referencia conservadora.

Configure inicialmente

```
T_ref = single(25)
```

y ejecute

`Monitor & Tune`.

Durante los primeros segundos compruebe que:

- la lectura de `T1` sea consistente con la temperatura real de la TCLab;
- la referencia permanezca en el valor configurado;
- la acción `Q1` permanezca dentro del intervalo 0–90 %; y
- no aparezcan errores de comunicación ni comportamientos anormales.

Una vez comprobado el funcionamiento correcto del lazo con una referenciacercana a la temperatura ambiente, detenga la ejecución.

Configure entonces

```
T_ref = single(60)
```

y ejecute nuevamente `Monitor & Tune`.

Esta referencia corresponde al intervalo de operación utilizado durante la identificación y el diseño del controlador. Debido al error inicial de temperatura, es posible que la acción de control alcance temporalmente el límite superior de 90 %. Este comportamiento es esperado durante la etapa inicial de calentamiento.

Observe cómo Q_1 disminuye progresivamente conforme T_1 se aproxima a la referencia y permita que el experimento continúe el tiempo suficiente para evaluar el comportamiento del lazo alrededor de $T_{\text{ref}} = 60\,^{\circ}\text{C}$.

Observe el calentamiento de la planta y permita que la respuesta evolucione durante un intervalo suficientemente largo para apreciar el acercamiento a la referencia. Como referencia de visualización, el Scope se encuentra configurado con una ventana de 600 s.

Durante la prueba observe simultáneamente:

- el seguimiento de T_{ref} por parte de T_1;
- la reducción progresiva de Q_1 conforme disminuye el error de temperatura;
- la ausencia de saturación sostenida injustificada; y
- el comportamiento de la medición filtrada alrededor del punto de operación.

Detenga normalmente la ejecución para finalizar el registro de las señales.

Actividad: ejecución del controlador sobre la TCLab

Ejecute el controlador PI–SIMC sobre la TCLab física mediante `Monitor & Tune`.

1. Realice la lista de comprobación previa antes de energizar el calentador.
2. Ejecute primero el lazo con `T_ref = single(25)`.
3. Compruebe que `T1` sea una lectura físicamente razonable y que $0 \leq Q_1 \leq 90\,\%$.
4. Detenga la ejecución si se observa cualquier condición anormal.
5. Una vez validada la prueba inicial, configure `T_ref = single(60)`.
6. Ejecute nuevamente `Monitor & Tune`.
7. Observe en el Scope la referencia, la temperatura y la acción de control durante el transitorio.
8. Mantenga la ejecución el tiempo suficiente para observar el acercamiento de la temperatura a la referencia.
9. Detenga normalmente la ejecución y compruebe que `logsout` se haya creado en MATLAB.
10. Verifique que el registro contenga las señales `T_ref`, `T1` y `Q1`.

La ejecución física constituye una validación experimental del lazo completo. A diferencia de MIL, SIL y PIL, la temperatura observada en esta etapa corresponde directamente a la planta real.

24.13 Visualización de los resultados

Para visualizar los resultados obtenidos mediante `Monitor & Tune` se suministra el script

`A12_visualizar_Implementacion_TCLab.m`.

El script recupera las señales directamente desde

`logsout`

utilizando sus nombres:

```
T_ref
T1
Q1
```

y genera una figura con dos paneles.

El panel superior presenta

$$T_{\text{ref}} \quad \text{y} \quad T_1,$$

mientras que el panel inferior presenta

$$Q_1.$$

Ejecute

```
A12_visualizar_Implementacion_TCLab
```

después de detener el experimento.

La figura obtenida permitirá analizar visualmente el seguimiento de referencia, la dinámica térmica de la planta y la evolución de la acción de control.

Actividad: documentación del experimento físico

Utilice los datos obtenidos durante `Monitor & Tune` para documentar el comportamiento del lazo cerrado.

1. Compruebe que `logsout` contenga las señales `T_ref`, `T1` y `Q1`.
2. Ejecute `A12_visualizar_Implementacion_TCLab.m`.
3. Compruebe que la figura muestre en el panel superior T_{ref} y T_1, y en el panel inferior Q_1.
4. Analice el seguimiento de la referencia y describa el comportamiento transitorio observado.
5. Analice la evolución de Q_1 y compruebe que permanezca dentro del intervalo permitido.
6. Identifique cualquier diferencia relevante entre el comportamiento experimental y el comportamiento observado previamente durante las etapas de simulación.
7. Conserve la figura y los datos registrados como evidencia de la implementación física.

24.14 Cierre de la implementación física y operación autónoma

Al completar esta etapa, el controlador deja de operar únicamente contra una planta simulada y pasa a controlar directamente el sistema térmico real.

El lazo cerrado implementado está compuesto por

$$T_{\text{ref}} \rightarrow \text{controlador en Arduino Leonardo} \rightarrow Q_1 \rightarrow \text{calentador TCLab} \rightarrow T_1 \rightarrow \text{controlador}.$$

Durante la ejecución mediante `Monitor & Tune`, las operaciones que forman parte del lazo de control se ejecutan sobre el Arduino Leonardo. Esto incluye la adquisición de `T1`, su conversión y filtrado, la ejecución del controlador PI–SIMC, el cálculo y limitación de `Q1` y la generación de la señal PWM en `D3`.

La computadora no participa en el cálculo del lazo de control. La comunicación con Simulink mediante *External Mode* se utiliza para monitorizar las señales, registrar los resultados experimentales y, cuando corresponda, modificar parámetros durante la ejecución.

Por tanto, la correcta ejecución mediante `Monitor & Tune` demuestra que el controlador generado puede integrarse con las entradas y salidas físicas de la TCLab y operar en lazo cerrado sobre la planta real.

La aceptación de esta etapa debe basarse en la evidencia experimental obtenida: el comportamiento de T_1, la evolución de Q_1, el seguimiento de la referencia, el cumplimiento de los límites del actuador y la ausencia de condiciones inseguras durante la prueba.

¿Qué sigue después de `Monitor & Tune`?

Una vez validado el funcionamiento sobre la planta física, el siguiente paso posible consiste en desplegar la aplicación para que el Arduino pueda ejecutar el controlador de manera autónoma, sin mantener una sesión de `Monitor & Tune`.

En Simulink, este flujo puede realizarse mediante

`Build, Deploy & Start.`

En este caso, Simulink genera y compila el código correspondiente a `TCLab_Implementation.slx` y a los modelos referenciados que forman parte de su jerarquía, descarga la aplicación al Arduino Leonardo e inicia su ejecución.

La arquitectura del lazo no cambia:

$$T_{\text{ref}} \rightarrow \text{controlador} \rightarrow Q_1 \rightarrow \text{TCLab} \rightarrow T_1 \rightarrow \text{controlador}.$$

La diferencia fundamental es que desaparece el enlace de supervisión mediante External Mode. El controlador continúa ejecutándose en el Arduino, pero ya no se dispone directamente de las herramientas de monitorización proporcionadas por Simulink, tales como

- visualización mediante `Scope`;
- registro de señales mediante `logsout`; y
- ajuste interactivo de parámetros durante la ejecución.

Si una aplicación autónoma requiere monitorización o almacenamiento de datos, estas funcionalidades deben incorporarse explícitamente mediante mecanismos adicionales de comunicación o registro.

Además, una aplicación autónoma requiere considerar cuidadosamente las condiciones de arranque. Una vez desplegado el programa, el Arduino puede comenzar a ejecutar el controlador después de ser energizado o reiniciado.

Por ejemplo, si se almacena

$$T_{\text{ref}} = 60\,^{\circ}\text{C}$$

y la TCLab se encuentra inicialmente cerca de la temperatura ambiente, el error inicial puede hacer que el controlador demande temporalmente el límite superior

$$Q_1 = 90\,\%.$$

Aunque este comportamiento puede ser consistente con el diseño del controlador, una aplicación autónoma debe incorporar una estrategia apropiada de arranque y seguridad antes de operar sin supervisión.

Entre los aspectos que deberían considerarse se encuentran

- la referencia y condiciones iniciales de arranque;
- el comportamiento ante reinicios del microcontrolador;
- la detección de mediciones inválidas;
- los límites de seguridad del actuador; y
- los mecanismos de parada o desactivación.

Estos aspectos forman parte del diseño de una aplicación embebida *standalone* completa y quedan fuera del alcance de esta práctica.

Cierre del proyecto

Mediante `Monitor & Tune`, el controlador PI–SIMC diseñado y verificado durante las etapas anteriores se ejecuta finalmente sobre el Arduino Leonardo y controla directamente la TCLab real.
El paso hacia una aplicación completamente autónoma no requiere trasladar el cálculo del controlador al Arduino, pues este ya se ejecuta allí durante `Monitor & Tune`. Lo que cambia es la eliminación del enlace de supervisión con Simulink y, por tanto, la necesidad de incorporar explícitamente las funciones de arranque, monitorización y seguridad requeridas por una aplicación *standalone*.

25 Bibliografía

Referencias

[1] J. D. Hedengren, "Arduino Dynamic Temperature Modeling," *APMonitor: Dynamics and Control*. Disponible en: https://apmonitor.com/pdc/index.php/Main/ArduinoModeling. Consultado: 11 de agosto de 2026.

[2] J. D. Hedengren, "TCLab Linearize Energy Balance," *APMonitor: Dynamics and Control*. Disponible en: https://apmonitor.com/pdc/index.php/Main/TCLabLinearize. Consultado: 11 de agosto de 2026.

[3] J. D. Hedengren, "TCLab FOPDT Model," *APMonitor: Dynamics and Control*. Disponible en: https://apmonitor.com/pdc/index.php/Main/TCLabFOPDT. Consultado: 11 de agosto de 2026.

[4] S. Skogestad, "Simple analytic rules for model reduction and PID controller tuning," *Journal of Process Control*, vol. 13, no. 4, pp. 291–309, 2003. doi: 10.1016/S0959-1524(02)00062-8.

[5] MathWorks, "Anti-Windup Control Using a PID Controller," *Simulink Documentation*. Disponible en: https://www.mathworks.com/help/simulink/slref/anti-windup-control-using-a-pid-controller.html. Consultado: 12 de agosto de 2026.

[6] MathWorks, "`stepinfo`: Rise time, settling time, and other step-response characteristics," *Control System Toolbox Documentation*. Disponible en: https://www.mathworks.com/help/control/ref/dynamicsystem.stepinfo.html. Consultado: 12 de agosto de 2026.

[7] MathWorks, "Model-Based Design Positions Startups for Success," video. Disponible en: https://uk.mathworks.com/videos/model-based-design-positions-startups-for-success-1707204629512.html. Consultado: 11 de agosto de 2026.

[8] MathWorks, "What Is Hardware-in-the-Loop (HIL)?" Disponible en: https://www.mathworks.com/discovery/hardware-in-the-loop-hil.html. Consultado: 1 de septiembre de 2026.

[9] MathWorks, "Source Code Generation," *Embedded Coder Documentation*. Disponible en: https://www.mathworks.com/help/ecoder/source-code-generation-and-verification.html. Consultado: 1 de septiembre de 2026.

[10] MathWorks, "Software-in-the-Loop Simulation," *Embedded Coder Documentation*. Disponible en: https://www.mathworks.com/help/ecoder/software-in-the-loop-sil-simulation.html. Consultado: 1 de septiembre de 2026.

[11] MathWorks, "Processor-in-the-Loop Simulation," *Embedded Coder Documentation*. Disponible en: https://www.mathworks.com/help/ecoder/processor-in-the-loop.html. Consultado: 1 de septiembre de 2026.

[12] MathWorks, "Arduino Hardware," *Simulink Support Package for Arduino Hardware Documentation*. Disponible en: https://www.mathworks.com/help/simulink/supportpkg-arduino.html. Consultado: 1 de septiembre de 2026.

[13] MathWorks, "Prepare, Configure, and Run Simulink Model for Different User Workflows on Arduino," *Simulink Support Package for Arduino Hardware Documentation*. Disponible en: https://www.mathworks.com/help/simulink/supportpkg/arduino_ref/prepare-configure-and-run-simulink-model-for-different-user-workflows-on-arduino.html. Consultado: 1 de septiembre de 2026.